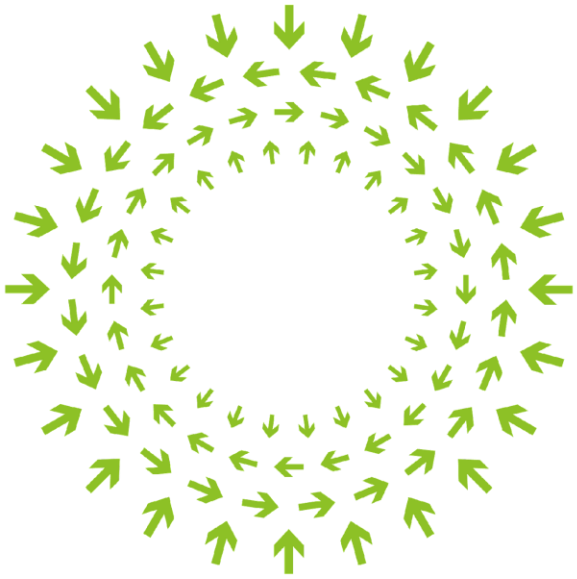


OPEN

Compute Project

Modular Plug-and-Play FRU, Discovery & Boot Design Specification

Part of the

Modular Hardware System Family of Specifications

Version 0.7.0 RC1

10/07/2026

M-PNP.FRU_DISCOVERY_BOOT Authors/Contributors:

Advanced Micro Devices, Inc: Duk Kim, Justin King

American Megatrends International, LLC: Chitrak Gupta, Rama Bisa

Dell Technologies, Inc.: Jeff Kennedy, Tim Lambert

Hewlett Packard Enterprise Company: Andrew Cartes, Phillip Leech, Bill Scherer, Mike Witkowski, Jeff Wolford, Melvin Benedict, Arun Koshy

Intel Corporation: Kasper Wszolek, Javier Lasa, Eduardo Estrada, Nilan Naidoo

Jabil, Inc.: Todd Rosedahl, Justin Thaler

Table of Contents

1. License 9
   1.1. Open Web Foundation (OWF) CLA 9
   1.2. Acknowledgements 10
2. Version Table 10
3. Overview 14
4. MHS and Related Family of Specifications 14
   4.1. Related OCP Specifications 15
   4.2. Related Industry Specifications 15
5. Terminology 17
   5.1. M-PNP Specific Terminology 17
   5.2. M-PNP Self-Describing Attributes Terminology Definition 19
6. Scope 19
   6.1. Supplementary Materials List 20
   6.2. Supplementary FRU and JSON Generation Tools and Materials List 21
   6.3. Typical OCP Sections Not Applicable 22
7. DC-SCM FRU Device Considerations 22
8. Component FRU Information Device Requirements and Considerations 22
   8.1. DC-SCM Mandated HPM FRU Information Device Requirements 22
   8.2. Additional HPM FRU Information Device Requirements and Preferences 22
9. HPM FRU Information Usage And Discovery Process 24
   9.1. M-PNP HPM FRU Information Overview 24
      9.1.1. M-PNP HPM FRU IPMI Header 25
      9.1.2. M-PNP HPM FRU IPMI Chassis Area Record 25
      9.1.3. M-PNP HPM FRU IPMI Board Area Record 25
      9.1.4. M-PNP HPM FRU IPMI Product Info Area Record 25
      9.1.5. M-PNP HPM FRU HPM FRU Multi-record Area Record 25
      9.1.6. HPM FRU M-PNP DMTF FRU Format OEM IPMI Multi-Record Section 26
      9.1.7. M-PNP HPM FRU OCP M-PNP DMTF FRU Format Section 27
   9.2. HPM FRU Write Protection Requirements 31
   9.3. M-PNP HPM Virtual FRU Device Considerations 32
   9.4. HPM Self-Description Requirements 32
   9.5. HPM Discovery Process 32
      9.5.1. System Discovery Method Overview 32

9.5.2. Discovery Phase 0: SCM Power and Initialization 33
9.5.3. SCM initialization 33
9.5.4. SCM BMC Early Boot Process (Boot Phase 0) 33
9.5.5. Discovery Phase 1: HPM Discovery 34
9.5.6. Early HPM Discovery Process 35
9.5.6.1. FRU Information Device Non-local Extended JSON File BIOS Firmware Volume Usage Guidance and Considerations 39
9.5.6.2. File Structure for Firmware Volume Navigation 39
9.5.7. Establishing Connection from SCM to HPM Primary HPM Programmable Device 41
9.5.8. Discovering the Primary HPM Programmable Device 42
9.5.9. Static HPM Primary Programmable Device Data Collection 43
9.5.10. Dynamic and Extended Primary HPM Programmable Device Data Collection 45
9.5.11. High-level discovery of the HPM 46
9.5.12. Platform Model Type Determination Considerations 49

10. Peripheral Device FRU Information Usage and Discovery Process 50

10.1. Backplane Peripheral Requirements and Considerations 50
10.1.1. Common Backplane Discovery and Configuration Capabilities Requirements 50
10.1.2. Common Backplane Peripheral Self-Description Requirements 51
10.1.3. UBM Backplane Specific Requirements 53
10.1.4. Example UBM Backplane Block Diagram 53
10.1.5. UBM Backplane NVMe Drive Hot Plug Requirements 54
10.1.6. UBM Backplane FlexIO Assignments 55
10.1.6.1. UBM Backplane MXIO FlexIO Signals Definition 55
10.1.7. UBM Backplane NVMe Drive Hot Plug Behavioral Modes 56
10.1.8. Non-UBM Backplane Type 1 (GPIO Expander on Backplane) NVMe Drive Hot Plug Requirements 57
10.1.9. Non-UBM Backplane Type 1 (GPIO Expander on Backplane) FlexIO Assignments 57
10.1.9.1. Non-UBM Backplane Type 1 (GPIO Expander on Backplane) MXIO FlexIO Signals Definition 58
10.1.10. Non-UBM Backplane Type 2 (GPIO Expander Functionality on HPM) NVMe Drive Hot Plug Requirements 59
10.1.11. Non-UBM Backplane Type 2 (GPIO Expander on HPM) FlexIO Assignments 59
10.1.11.1. Non-UBM Backplane Type 2 (GPIO Expander on HPM) MXIO FlexIO Signals Definition 60
10.1.12. HPM SoC to Primary HPM Programmable Device Interconnection Options For NVMe Host Plug Support 61
10.2. Riser Peripheral Requirements and Considerations 61
10.2.1. Common Riser Discovery and Configuration Capabilities Requirements 61
10.2.2. Common Riser Peripheral Self-Description Requirements 63
10.2.3. Example Riser Block Diagram 65

10.3. Power Distribution Board (PDB) Peripheral Requirements and Considerations 65
10.3.1. Common PDB Discovery and Configuration Capabilities Requirements 66
10.3.2. Common PDB Peripheral Self-Description Requirements 67
10.3.3. Example Power Distribution Board (PDB) Block Diagram 70
10.4. Control Panel Board Peripheral Requirements and Considerations 71
10.4.1. Common Control Panel Discovery and Configuration Capabilities Requirements 71
10.4.2. Common Control Panel Peripheral Self-Description Requirements 73
10.4.3. Example Control Panel Block Diagram 75
10.4.4. Control Panel M-PESTI Virtual Wires Considerations 76
10.5. PCIe Switch Peripheral Requirements and Considerations 76
10.5.1. Common PCIe Switch Discovery and Configuration Capabilities Requirements 76
10.5.2. Common PCIe switch Peripheral Self-Description Requirements 78
10.5.3. Example PCIe switch Block Diagram 80
10.6. Remote Cooling Subsystem Peripheral Requirements and Considerations 80
10.6.1. Common Remote Cooling Subsystem Discovery and Configuration Capabilities Requirements 81
10.6.2. Common Remote Cooling Subsystem Peripheral Self-Description Requirements 82
10.6.3. Example Remote Cooling Subsystem Peripheral Block Diagram 84
10.7. PCIe Re-timer CEM Passthrough Peripheral Requirements and Considerations 84
10.7.1. Common PCIe Re-timer CEM Passthrough Peripheral Discovery and Configuration Capabilities Requirements 85
10.7.2. Common PCIe re-timer CEM passthrough Peripheral Self-Description Requirements 86
10.7.3. Example PCIe Re-timer CEM passthrough peripheral Block Diagram 89
10.8. OCP NIC Interposer Peripheral Requirements and Considerations 90
10.8.1. Common OCP NIC Interposer Peripheral Discovery and Configuration Capabilities Requirements 90
10.8.2. Common OCP NIC interposer Peripheral Self-Description Requirements 91
10.8.3. Example OCP NIC interposer peripheral Block Diagram 94
10.9. General Peripheral Requirements and Considerations 95
10.9.1. Common Discovery and Configuration Capabilities Requirements 95
10.9.2. Common Peripheral Self-Description Requirements 96
10.10. Discovery Phase 2: Non-endpoint Peripherals Discovery 98
10.10.1. Primary HPM Programmable Device Autonomous System Discovery: Non-Endpoint Peripherals 99
10.10.2. Establishing Connection to Non-Endpoint Peripherals 100
10.10.3. High-level Discovery of Non-Endpoint Peripherals 102
10.11. Peripheral Device Class Definitions 104
10.11.1. Peripheral Device Class Table 104
10.11.2. M-PESTI Payload and Snoop Device Considerations 105
10.12. Peripheral M-XIO FlexIO And M-PIC PICPWR Alternate Functions Definitions and Default Assignments 105

10.12.1. M-XIO FlexIO Alternate Functions Definitions Table 105
10.12.2. M-PIC PICPWR Sideband Alternate Functions Definitions 106
10.12.3. Control Panel Sideband Alternate Functions Definitions Table 107
10.13. Peripheral Device FRU Information 107
10.13.1. M-PNP Peripheral FRU IPMI Common Header 108
10.13.2. M-PNP Peripheral FRU IPMI Chassis Info Area Record 108
10.13.3. M-PNP Peripheral FRU IPMI Board Info Area Record 109
10.13.4. M-PNP Peripheral FRU IPMI Product Info Area Record 109
10.13.5. M-PNP Peripheral FRU IPMI Multi-record Area Record 109
10.13.6. M-PNP Peripheral DMTF FRU Format OEM IPMI Multi-Record FRU M-PNP Multi-record Section 110
10.13.7. Peripheral FRU DMTF FRU Format OEM Multi-record Header 111
10.13.8. Peripheral FRU M-PNP FRU Format OEM Multi-record Contents 111
10.13.9. M-PNP Peripheral FRU OCP M-PNP DMTF FRU Format Section 112
10.13.10. M-PNP Peripheral DMTF FRU File Entry 1: Local Self-Describing JSON Data 113
10.13.11. M-PNP Peripheral DMTF FRU File Entry 2: Non-Resident Self-Describing JSON Data 113
10.13.12. M-PNP Peripheral DMTF FRU File Entry 3: Resident Self-Describing JSON Data For Peripheral PLD Initial Factory Image 114
10.13.15. M-PNP Peripheral DMTF FRU File Entries (N – O + 1) to N: Optional OEM Files 114
10.13.16. Peripheral FRU Self-Describing Attributes Considerations and Guidelines 114
10.13.17. Peripheral FRU Information Authenticity Requirements 115
10.13.18. Peripheral FRU Write Protection Requirements 115
10.14. Generalized Nested Peripherals Guidance 116
10.15. Discovery Phase 3: Endpoint Peripheral Discovery Method 117
10.15.1. Primary HPM Programmable Device Autonomous System Discovery: Endpoint Peripherals 118
10.15.2. Establishing Connection to Endpoint Peripheral Devices 118
10.15.3. Endpoint Peripheral High-level Discovery 119
10.15.4. Granting Full Operational Power to the System (Transition To S0) 120
10.15.5. Considerations Regarding Operational Power For Peripherals Connected Via Standard Interfaces 122
10.15.6. Considerations Regarding Main Power to Endpoint Peripherals Connected Via Non-Standard / OEM Connectors 122
10.15.7. SCM Final Discovery State Checkpoint 122
10.15.8. Runtime Rediscovery Considerations 123
11. FRU Digital Signature 123
11.1. FRU Information Authenticity Requirements 123
11.2. FRU Digital Signature Requirements 123
12. Component Self-Describing JSON Description 124
12.1. Self-Describing JSON Overview 124

12.2. JSON Schema Reference Documentation 126
12.3. Common Bus Reference Diagrams 126
12.3.1. I2C Bus Reference Diagrams 126
12.3.2. I3C Bus Reference Diagrams 127
12.3.3. JTAG Bus Reference Diagrams 128
12.3.4. PCIe Bus Reference Diagrams 129
12.3.5. USB Reference Diagrams 130

13. Boot Process Requirements 132
13.1. Boot Process System-Level Discovery 132
13.1.1. Secure Discovery Process Enablement 132
13.2. BMC Usage of Primary HPM Programmable Device Power State Guidance and Considerations 133
13.3. BMC – BIOS Communications Guidance 133
13.3.1. BMC – BIOS Communications Requirements 133
13.3.2. Early Boot Communications Data Payloads Transport Guidance 134

14. Specification Compliance Checklist 136

15. Trademarks 136

16. Future Material 136

Table of Figures

Table 1: Standards Terminology 17
Table 2: M-PNP Specific Terminology....17
Table 3: Supplementary Material 20
Table 3: Supplementary FRU and JSON Generation Tools and Materials 21
Table 4: M-PNP FRU Format File Descriptor GUIDs Table 28
Table 5: Common Backplane Requirements 50
Table 6: Backplane Peripheral Self-describing JSON Attributes....52
Table 7: UBM Backplane Specific Requirements 53
Table 8: UBM Backplane FlexIO Assignments 55
Table 9: UBM Backplane MXIO FlexIO Signals Definition 56
Table 10: Non-UBM Backplane MXIO FlexIO Assignments 58
Table 11: Non-UBM Backplane MXIO FlexIO Signals Assignment Description 58
Table 12: Non-UBM Backplane Type 2 (GPIO Expander on HPM) MXIO FLEXIO Assignments 60
Table 13: Non-UBM Backplane Type 2 MXIO FlexIO Signals Definition....60
Table 14: Riser Peripheral Requirements....62
Table 15: Riser Peripheral Self-Describing JSON Attributes 63
Table 16: PDB Requirements....66
Table 17: PDB Self-Describing JSON Attributes 67
Table 18: Control Panel Discovery Requirements 71
Table 19: Control Panel Self-Describing JSON Attributes 73
Table 20: Example Control Panel M-PESTI Virtual Wire Definitions....76
Table 21: PCIe Switch Peripheral Requirements....76
Table 22: PCIe Switch Peripheral Self-Describing JSON Attributes 78
Table 23: Remote Cooling Subsystem Peripheral Requirements 81
Table 24: Remote Cooling Peripheral Self-Describing JSON Attributes 82
Table 25: PCIe Re-timer CEM passthrough Peripheral Requirements 85
Table 26: PCIe Re-timer CEM passthrough Peripheral Self-Describing JSON Attributes 87
Table 27: OCP NIC Interposer Peripheral Requirements 90
Table 28: OCP NIC interposer Peripheral Self-Describing JSON Attributes 92
Table 29: Common Peripheral Discovery Requirements 95
Table 30: Common Peripheral Self-Describing JSON Attributes 96
Table 31: Peripheral Device Class Table 104
Table 32: MXIO FlexIO Alternate Functions Table 105
Table 33: M-PIC PICPWR Sideband Alternate Functions Definitions Table 106
Table 34: M-PIC Panel Control Sideband Alternate Functions Definitions Table 107
Table 35: Peripheral FRU IPMI Multi-Record Area Record Table....109
Table 83: BIOS - BMC Communications Requirements 133
Table 84: Early Boot Payload Requirements....134
Table 85: Required and Optional JSON Files Exposed Via PLDM File Transfer 135
Table 86: Compliance checklist table. 136

1. License

1.1. Open Web Foundation (OWF) CLA

Contributions to this Specification are made under the terms and conditions set forth in Open Web Foundation Modified Contributor License Agreement ("OWF CLA 1.0") ("Contribution License") by:

- Advanced Micro Devices, Inc
- Ampere Computing
- American Megatrends International (AMI)
- Arm Holdings PLC
- Dell, Inc.
- Google LLC
- Hewlett Packard Enterprise Company
- Intel Corporation
- Jabil Inc.
- Lattice Semiconductor Corporation
- Meta Platforms, Inc.
- Microsoft Corporation
- Nvidia Corporation

Usage of this Specification is governed by the terms and conditions set forth in **Open Web Foundation Modified Final Specification Agreement ("OWFa 1.0") ("Specification License").**

You can review the applicable OWFa1.0 Specification License(s) referenced above by the contributors to this Specification on the OCP website at http://www.opencompute.org/participate/legal-documents/. For actual executed copies of either agreement, please contact OCP directly.

Notes:

1. The above license does not apply to the Appendix or Appendices. The information in the Appendix or Appendices is for reference only and non-normative in nature.

NOTWITHSTANDING THE FOREGOING LICENSES, THIS SPECIFICATION IS PROVIDED BY OCP "AS IS" AND OCP EXPRESSLY DISCLAIMS ANY WARRANTIES (EXPRESS, IMPLIED, OR OTHERWISE), INCLUDING IMPLIED WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, OR TITLE, RELATED TO THE SPECIFICATION. NOTICE IS HEREBY GIVEN, THAT OTHER RIGHTS NOT GRANTED AS SET FORTH ABOVE, INCLUDING WITHOUT LIMITATION, RIGHTS OF THIRD PARTIES WHO DID NOT EXECUTE THE ABOVE LICENSES, MAY BE IMPLICATED BY THE IMPLEMENTATION OF OR COMPLIANCE WITH THIS SPECIFICATION. OCP IS NOT RESPONSIBLE FOR IDENTIFYING RIGHTS FOR WHICH A LICENSE MAY BE REQUIRED IN ORDER TO IMPLEMENT THIS SPECIFICATION. THE ENTIRE RISK AS TO IMPLEMENTING OR OTHERWISE USING THE SPECIFICATION IS ASSUMED BY YOU. IN NO EVENT WILL OCP BE LIABLE TO YOU FOR ANY MONETARY DAMAGES WITH

RESPECT TO ANY CLAIMS RELATED TO, OR ARISING OUT OF YOUR USE OF THIS SPECIFICATION, INCLUDING BUT NOT LIMITED TO ANY LIABILITY FOR LOST PROFITS OR ANY CONSEQUENTIAL, INCIDENTAL, INDIRECT, SPECIAL OR PUNITIVE DAMAGES OF ANY CHARACTER FROM ANY CAUSES OF ACTION OF ANY KIND WITH RESPECT TO THIS SPECIFICATION, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), OR OTHERWISE, AND EVEN IF OCP HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

1.2. Acknowledgements

With the hope of making this specification useful for the entire OCP community, we acknowledge and appreciate the contributions, review, and feedback of various individuals and companies that participated in MHS.

2. Version Table

Date	Version #	Description
4/10/2024	0.04	Cleanup. Not done.
7/9/2024	0.11	Merge discovery and FRU and Boot into this doc. Review.
8/20/2024	0.12	Cleanup and improved wordsmithing and content clarifications in many sections. Added content regarding FRU information device format. Many formatting changes and moving of sections. Added content regarding backplanes and risers. Added HPM minimum self-describing JSON section. Added self-describing JSON for connectors. Digital certificate initial content. Placeholder conformance checklist. Initial table of figures.
9/19/2024	0.30	Cleanup of many sections. Addressed review comments. Addressed yellow highlighted content. Updated related specs list. Wording / verbiage modifications. Added HPM Standby / S5 SCM interfaces list. Removed UBM backplane interoperability statement. Added a statement in section 10.4.13 regarding peripheral self-describing considerations and guidelines.
9/27/2024	0.3.0 RC1	Minor cleanup. Addressed review comments.
5/1/25	0.3.1	Added peripheral device class table and alternate FlexIO and sideband assignments table.
5/30/25	0.3.2	Updated FRU format to use updated DSP0220 release spec.

		Updated chapter 12 self-describing JSON attributes sections and combined the HPM and peripheral JSON attributes sections due to shared schema. Updated chapter 13 self-describing JSON attributes to reflect updated schema. Added suggested control panel alternative function signal. Additional cosmetic changes throughout the document.
7/10/25	0.4.0	Updated these sections: 4.2 Related Industry Specifications 6.1. Supplementary Materials List Major changes in chapters: 9. HPM FRU Information Usage And Discovery Process 10. Peripheral Device FRU Information Usage and Discovery Process 12. Component Self-Describing JSON Attributes 13. Common Self-Describing JSON Attributes Minor format or wording changes in these sections: 5. Terminology 11. FRU Digital Certificate and Signature
8/12/25	0.5.0	Updated these sections: 4.2 Related Industry Specifications 6.0 Scope 8.1 DC-SCM Mandated HPM FRU Information Device Requirements 9.1. M-PNP HPM FRU Information Overview 9.4. HPM Self-Description Requirements 9.5. HPM Discovery Process 12. Component Self-Describing JSON Attributes 13. Common Self-Describing JSON Attributes 15. FRU Data Generation Tools and Collaterals Significant changes in chapters: 10. Peripheral Device FRU Information Usage and Discovery Process 11. FRU Digital Certificate and Signature 14. Boot Process Requirements
9/10/25	0.5.0 RC1	Minor grammar and formatting changes throughout the document. Added missing figure and table captions. Cleanup of footnotes. Minor language clarifications. Updated these sections: 6.2. Supplementary FRU and JSON Generation Tools and Materials List

		10.1.2. Common Backplane Peripheral Self-Description Requirements 10.2.2. Common Riser Peripheral Self-Description Requirements 10.3.2. Common PDB Peripheral Self-Description Requirements 10.4.2. Common Control Panel Peripheral Self-Description Requirements 10.5.2. Common Peripheral Self-Description Requirements 10.9.5. M-PNP Peripheral FRU IPMI Multi-record Area Record 14.3.2. Early Boot Communications Data Payloads Transport Guidance Removed chapter 15 as this content was a duplicate of section 6.2.
9/18/25	0.5.0 RC2	Minor formatting changes throughout the document. Section 10.1.2, fixed footnotes. Section 10.2.2, added the statement regarding the ‘C’ conditional option for the table. Section 10.9.5, moved the VendorID attribute before DeviceID in the table. Section 11.3, minor grammatical fix. Section 13.2.6, fixed the footnote ordering so note is at the top of list before ‘ibid’.
3/18/26	0.5.1	Added clarifying statements in section 10.1. Added sections 10.1.5 to 10.1.12 regarding backplane hot plug support guidance.
5/06/26	0.5.2	Added sections on PCIe switch, cooling subsystem, PCIe re-timer, and OCP NIC interposer peripheral boards.
7/01/26	0.5.3	Added sections 10.14. Generalized Nested Peripherals Guidance and updated section 14.3.2. Early Boot Communications Data Payloads Transport Guidance.
8/12/26	0.5.4	Updated the discovery process sections 9.5, 10.10 and 10.15 to address error reporting and failure paths. Additional clarifications to text. Fixed the figure captioning auto-incrementing. Added missing table captions.
8/20/26	0.5.5	Added clarification language in rediscovery process. Removed digital certificate. Added other language clarifications regarding the platform model type determination, NVME hot plug clarifications, and PDB as an MPESTI target.
9/03/26	0.6.0	Added terminology clause for ISO/EAC directives. Adjusted formatting. Improved / clarified language in places. Added runtime considerations guidance for rediscovery. Removed all JSON attributes tables and added referencing schema spec document. Added BIOS firmware volume usage guidance. FRU signature guidance. Clarifying SoC ramp power state use case.

9/25/26	0.7.0 Draft	Added missing industry specifications in section 4.2. Defined Hardware ID in table 2. Added note in section 5.2 regarding referencing the HW and FPGA schema documentation. Added bullet point to in-scope in section 6. Added HW and FPGA schema documentation to supplementary materials list. Added qualifier definition in section 9.4. Language clarifications in section 9.5. Changed to 'Preferred' and 'Legacy' for the 'plug and play' level options in section 9.5.7 and removed option B since it was left vague and undefined. Fixed security specification references in 9.5.10. Provided additional guidance in section 10 first and second paragraph. Added clarifying language and notes in peripheral requirements tables in section 10. Added clarifying language and guidance for riser, PDB, Control Panel, PCIe Switch, Cooling and OCP NIC interposer example block diagram in section 10.2.3, 10.3.3, 10.4.3, 10.5.3, 10.6.3, 10.8.3. Added clarifying language for general peripheral requirements section 10.9. Improved figure 27 in chapter 10.10. Added clarifying language to first paragraph in 10.10.1. Added vendor defined device class range to peripheral device class table. Added note to table 32 for FlexIO alternate functions. Clarifying language to section 10.12.3. Replaced figure 29 to remove incorrect address and I2C bus information. Added reserved range to table 35. Clarified language for File Entry 2 in section 10.13.9. Added patch file location examples in section 10.13.12. Removed specific peripheral FRU write protection signal implementation example in 10.13.18 and added note that implementation is vendor defined. Changed I2C 7-bit address to 8-bit. Clarified language in sections 10.15.1 and 10.15.4. Grammar and spelling typo fixes throughout the document. Updated table of contents.
10/7/26	0.7.0 RC1	Replaced reference to FPGA throughout the document and changed to HPM Primary Programmable Device. Changed reference to M-PNP.FPGA specification to M-PNP.HPM and Peripheral Programmable Device specification. Updated terminology in table 2. Updated supplementary material in table 3. Updated diagrams to change from FPGA to HPM Primary Programmable Device or added a note regarding the usage of an FPGA in the diagram.

3. Overview

This specification defines and standardizes common elements needed for the Host Processor Module (HPM) and peripheral Field Replaceable Unit (FRU) component contents as well as common discovery of HPM and peripherals and boot guidance for MHS-based systems.

This specification outlines what is required for a server and its constituent components to be considered M-PNP compliant.

There are three main areas of focus within this specification.

- Discovery of system components in a deterministic standardized manner to support the plug and play vision of interoperability.
- FRU and extended FRU information use to support discovery and management of the system in a standardized manner.
- Requirements regarding the system boot process to enable plug and play.

The system, as mentioned above, is comprised of all the constituent components within the server. This includes all peripherals as well as Host Processor Modules (HPMs).

4. MHS and Related Family of Specifications

The Modular Hardware System (MHS) Rev 1.0 family of specifications are written to enable interoperability between key elements of datacenter and enterprise infrastructure by providing consistent interfaces and form factors among modular building blocks. At the time of this publication there are the following specification workstreams:

- **M-FLW** (**M**odular Hardware System **F**ull **W**idth Specification) – Host Processor Module (HPM) form factor specification optimized for using the full width of a Standard EIA-310 Rack mountable server. The specification is not limited to use within the EIA-310 Rack but is used to serve as a template for a common target where the design is expected to be utilized.
- **M-DNO** (**M**odular Hardware System Partial Width **D**e**n**sity **O**ptimized Specification) – Host Processor Module (HPM) specification targeted to partial width (i.e., ½ width or ¾ width) form factors. Such form factors are often depth challenged and found not only in enterprise applications but also in Telecommunications, Cloud and Edge Deployments. While the EIA-310 Rack implementation is chosen as a key test case for use, the specification is not limited to use within the EIA-310 Rack but is used to serve as a template for a common target where the design is expected to be utilized. Note that the M-SDNO (scalable density optimized) HPM specification also exists.
- **M-CRPS** (**M**odular Hardware System **C**ommon **R**edundant **P**ower **S**upply Specification) – Specifies the power supply solutions and signaling expected to be utilized by MHS compatible systems.
- **M-PIC** (**M**odular Hardware System **P**latform **I**nfrastructure **C**onnectivity Specification) – Specifies common elements needed to interface a Host Processor Module (HPM) to the platform/chassis infrastructure elements/subsystems. Examples include power management, control panel and cooling amongst others.
- **M-XIO** (**M**odular Hardware System E**x**tensible **I**/**O**) – Specifies the high-speed connector hardware strategy. An M-XIO source connector enables entry and exit points between

sources such as Motherboards, Host Processor Modules & RAID Controllers with peripheral subsystems such as PCIe risers, backplanes, etc. M-XIO includes the connector, high speed and management signal interface details and supported pinouts.

- **M-PESTI** (**M**odular Hardware System **Pe**ripheral **S**ideband **T**unneling **I**nterface) – Specifies a standard method for discovery of subsystems, self-describing attributes, and status (i.e., versus a priori knowledge/hard coding firmware and BIOS for fixed/limited configurations). Examples: vendor/module class, physical connectivity descriptions, add-in card presence, precise source to destination cable coupling determination.
- **M-SIF** (**M**odular Hardware System **S**hared **Inf**rastructure Specification) – Specifies data and management plane interfaces along with power and cooling connections that are expected to be present within a modular platform utilizing shared elements.
- **DC-SCM** (Data Center Secure Control Module Specification) **2.1 -** Specification for a control module containing the system BMC. This module implements a Data Center Secure Control Interface (DC-SCI) used for interfacing with a Host Processor Module (HPM).
- **HPM Common Circuits Type 1**- Specifies the set of HPM common circuits as interfaced by the SCM and its BMC as categorized into types. Type 1 refers to a common single node platform.
- **M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE (M**odular-**P**lug-a**n**d-**P**lay) **-** Specifies Primary HPM and Peripheral Programmable Device requirements and guidelines for common BMC interfacing and descriptive information.
- **M-PNP Interoperability (M**odular-**P**lug-a**n**d-**P**lay) **–** Specifies the requirements and conformance criteria a server system and constituent components shall meet to satisfy all criteria to be Modular Plug and Play (M-PNP) conformant.

To access additional MHS specifications, please visit the OCP Server WIKI at Server/MHS - OpenCompute.

4.1. Related OCP Specifications

The list below contains the set of related OCP specifications referenced in this document.

- **OCP NIC 3.0 Design Specification -** Specifies a standard for Network Interface Cards (NIC)s.

4.2. Related Industry Specifications

The list below contains the set of related industry specifications referenced in this document.

- **DMTF PCMI DSP0220 v1.0 - Field Replaceable Unit (FRU)** Data Specification. https://www.dmtf.org/sites/default/files/standards/documents/DSP0220_1.0.pdf

- **DMTF DSP0236 v1.3, Management Component Transport Protocol (MCTP)** Base Specification:
  https://www.dmtf.org/sites/default/files/standards/documents/DSP0236_1.3.pdf
- **DMTF PMCI DSP0257 v2.0 - PLDM** Platform Level Data Model (PLDM) for FRU Data Specification:
  https://www.dmtf.org/sites/default/files/standards/documents/DSP0257_2.0.pdf
- **DMTF PLDM for File Transfer Specification (DSP0242):**
  https://www.dmtf.org/dsp/DSP0242
- **DMTF MMBI Specification DSP0282 v1.0** Memory-Mapped Buffer Interface (MMBI) Specification:
  https://www.dmtf.org/sites/default/files/standards/documents/DSP0282_1.0.pdf
- **IPMI Platform Management FRU Information Storage Definition Specification v1.0 Document Revision 1.3** Specification:
  https://www.intel.com/content/dam/www/public/us/en/documents/specification-updates/ipmi-platform-mgt-fru-info-storage-def-v1-0-rev-1-3-spec-update.pdf

5. Terminology

The terms in the following table have definitions according to ISO/IEC Directives, Part 2, Clause 7. Those meanings are intended where these terms appear.

Table 1: Standards Terminology

Standardized Term	Meaning
Shall	Indicates a requirement or something that must be done.
Shall Not	Indicates something that must not be done.
May	Used to indicate that something is permitted.
Need Not	Not required.
Should	Indicates a recommendation, which is optional but still a good idea.
Should Not	Not recommended.
Can	Used to indicate that something is possible, for example, that an organization or individual is able to do something
Cannot	See ISO/IEC Directives, Part 2, Clause 7.
Normative	See ISO/IEC Directives, Part 2, Clause 7.
Informative	See ISO/IEC Directives, Part 2, Clause 7.

5.1. M-PNP Specific Terminology

For the purposes of this document, the following additional terms and definitions apply.

Table 2: M-PNP Specific Terminology

Standardized Term	Meaning	Alternative Terms

FRU	Field Replaceable Unit	
FRU Information	Data reflecting an IPMI Storage FRU standard data format for including records such as board or product information plus sometimes a required record with self-describing local attributes and references for additional extended attributes.	
FRU Information Device	A real or virtual EEPROM, or in the future a programmable device such as an MCU that contains the necessary local attributes	
Local Attributes	HW description attributes stored in the device being described. e.g., in an HPM or peripheral EEPROM. More explanation provided in section 5.1.	
Extended Attributes	HW description attributes not stored in the device being described. e.g., BMC or BIOS ROM on behalf of HPM or peripheral. More explanation provided in section 5.1.	
Root Of Trust (RoT)	An optional security processor responsible for attesting the BMC, BIOS and/or other firmware images on the system. See DC-SCM specification for more details.	Protected RoT, Secure RoT, Platform RoT, External RoT, Hardware RoT
Hardware ID	Used as a cache / lookup key as a tuple of Vendor ID, HPM or Device ID, Class, Subclass, and Serial Number. {Vendor ID, HPM / Device ID, HPM Type | Device Class, HPM | Device Subtype, HPM / Peripheral Component Serial Number}	
HPM Primary Programmable Device	The primary programmable device that the BMC interfaces with for HPM management. Typically implemented as a Field Programmable Gate Array (FPGA) or Custom Programmable Logic Device (CPLD).	FPGA, CPLD
Non-endpoint Peripheral	An MHS peripheral component that has one or more connectors that support a connection to an endpoint peripheral such as a PCIe CEM card or OCP NIC board. Examples include plug-in card risers, storage backplanes, PCIe switch boards, and others.	
Endpoint Peripheral	A peripheral that does not have any connectors to additional plug-in or cabled components. Examples include PCIe Card Electromechanical (CEM) and OCP NIC boards.	

5.2. M-PNP Self-Describing Attributes Terminology Definition

This section generally classifies and defines what self-describing attributes are and what they are used for regarding the enabling of component interoperability. Self-describing attributes are used in the component discovery process and are required for promoting plug-and-play standardization.

Local attributes are defined as self-describing hardware attributes that reside inside the FRU Information Device located on the device needing to be discovered and configured. It is preferred to minimize the FRU Information local self-describing attributes to only the most essential static information.

Extended attributes are defined as hardware description attributes that reside off the device needing to be discovered and configured, such as in BMC, BIOS, or other storage areas.

Requirements:

- A common set of local attributes will be encoded as part of an IPMI multi-record in IPMI format to enable unique component identification.
- A minimum set of local attributes will be encoded as JSON to provide required discovery information, such as I2C trees and other essential data for plug-and-play enablement.
- All Extended Attributes shall be in JSON and not IPMI or table structures such as CSV.
- Extended attributes will use specific unique identification values to enable a unique mapping between a specific component's FRU and off component extension files.
- Candidate extended attributes include those that are not critical to minimum operation (e.g., thermal algorithms are more conservative if not present) or that are excessively large.

Refer to the Hardware and Programmable Device schema documentation for reference regarding specific local and extended attributes requirements.

6. Scope

This document defines technical specifications for the M-PNP.FRU_DISCOVERY_BOOT Design Specification used in the MHS family to support interoperability between MHS components. This section broadly outlines what is in and out of scope for this specification. In addition, this section lists a set of supplementary materials.

Any supplier seeking OCP recognition for a hardware product based on this spec shall be fully compliant with all features or requirements listed in chapter 14, "Specification Compliance Checklist", of this specification.

In-Scope:

- FRU information device requirements for HPM and peripherals
- Multi-record base for HPM and peripherals
- Self-describing JSON information schemas
- Definition of Local Attributes (e.g., Vendor ID, Device Class, Device ID, Major HW topologies, etc.)
- Definition of Extended Attributes
- Discovery process
- HPM and peripherals M-PNP support guidance and requirements

- Boot process requirements
- Security requirements and recommendations
- Supplementary materials list.

Out of Scope:
- DC-SCM 2.1 or newer HPM FRU information requirements multi-record definition
- Any relation to endpoints, end form factors (See PCI-SIG, SNIA, NVMe-MI, OCP NIC, etc.)
- OEM specific local or extended attributes definition

6.1. Supplementary Materials List

This section outlines a list of non-exclusive materials considered to be supplementary to this design specification. Revision and documentation control will be outside the scope of this document. Changes to those materials will not be tracked within this document.

Table 3: Supplementary Material

Material Name	Description
HPM and Peripherals Self-Describing File JSON Schema	Schema for self-describing informational files specific to the HPM MHS component and peripheral components such as risers and backplanes.
HPM and Peripherals Self-Describing File JSON Schema Documentation Files: hardware-component-documentation.md hardware-component-documentation.pdf hardware-component- developer.md hardware-component- developer.pdf hardware-component-changelog.md	HPM and Peripherals Self-Describing File JSON Schema Documentation both in markdown and PDF format. The document changelog is provided in markdown format.
HPM Self-Describing File Mock-up Example(s)	HPM JSON mock-ups with example data for reference.
Peripheral Self-Describing File Mock-up Example(s)	Peripheral JSON mock-ups with example data for reference.
HPM and Peripheral Programmable Device Self-Describing JSON Information File Schema	Schema for the HPM and Peripheral Programmable Device self-describing JSON information file.
HPM and Peripheral Programmable Device Self-Describing File JSON Schema Documentation Files:	HPM and Peripheral Programmable Device Self-Describing File JSON Schema Documentation both in markdown and PDF format. The

programmable-device-documentation.md programmable-device--documentation.pdf programmable-device--developer.pdf programmable-device--developer.md programmable-device--changelog.md	document changelog is provided in markdown format.
HPM and Peripheral Programmable Device Self-Describing JSON Information File Example	HPM and Peripheral Programmable Device image deliverable created at build time by vendor which provides self-describing JSON for HPM and Peripheral Programmable Device IP block CSRs, virtual wire definitions, and other items.

6.2. Supplementary FRU and JSON Generation Tools and Materials List

This section outlines a list of non-exclusive materials considered to be supplementary to this design specification for the use in aiding the HPM and peripherals vendor and system designer for creating the FRU image and producing a self-describing JSON description of the set of HPM and / or peripherals.

Table 4: Supplementary FRU and JSON Generation Tools and Materials

Material Name	Description
M.PNP FRU Packaging Tool	Example packaging tool for packing the DMTF FRU format (DSP0220) and the legacy IPMI format.

Delivery of these materials is outside the scope of this specification. Tools may be delivered via OCP GitHub or packaged along with the point releases of the M.PNP specification materials as supplementary documents and / or other materials.

See section 6.1, “Supplementary Materials List”, for an additional list of supplementary materials.

6.3. Typical OCP Sections Not Applicable

This is a design specification, requiring other MHS specifications to fully define a system. The following typical Sections of an OCP specification are not included because they are not applicable to this specification.

Rack Compatibility
Physical Spec
Rear Side Power, I/O, Expansion
Mechanical
Onboard Power System
Environmental Regulations/Requirements
Prescribed Materials
Software Support

7. DC-SCM FRU Device Considerations

Out of scope of this specification is any board or system FRU information requirements contained on a DC-SCM or on end form factors (like PCIe CEM add-in cards).

8. Component FRU Information Device Requirements and Considerations

8.1. DC-SCM Mandated HPM FRU Information Device Requirements

See latest DC-SCM Specification (minimum revision 2.1) for hardware requirements including the required HPM FRU I2C bus number through the DC-SCI, power source/state, target bus frequency, target device addressing, as well as device access requirements (e.g., 2-byte internal addresses).

8.2. Additional HPM FRU Information Device Requirements and Preferences

This section outlines the specific requirements and provides general guidance for FRU information devices utilized on the HPM.

It is recognized that a FRU Information Device may be via a physical EEPROM or a virtual EEPROM emulated by programmable logic or a microcontroller. If the FRU Information Device is emulated in another programmable device, that device is recommended to support secure boot, and such emulation shall follow the same timings and logical requirements of a physical EEPROM.

See PCI Express Base Specification >= 6.2 for FRU Information Device selection criteria, inclusive of 2 byte addressing, etc. for more details.

9. HPM FRU Information Usage And Discovery Process

This section outlines the HPM FRU information requirements to support M-PNP interoperability.

9.1. M-PNP HPM FRU Information Overview

This section provides a high-level overview of the HPM FRU contents. The following figure provides such an overview, and the figure below contains the FRU Information expected in an HPM FRU Information Device. Solid lines indicate required sections, and dotted lines mean those sections are optional. The blue coloring indicates sections that will use a format specified by the DMTF standards body within DSP0220 for housing the self-describing attributes, digital signature files, and OEM specific optional files.

The following sections provide further details regarding the records contained within the block diagram.

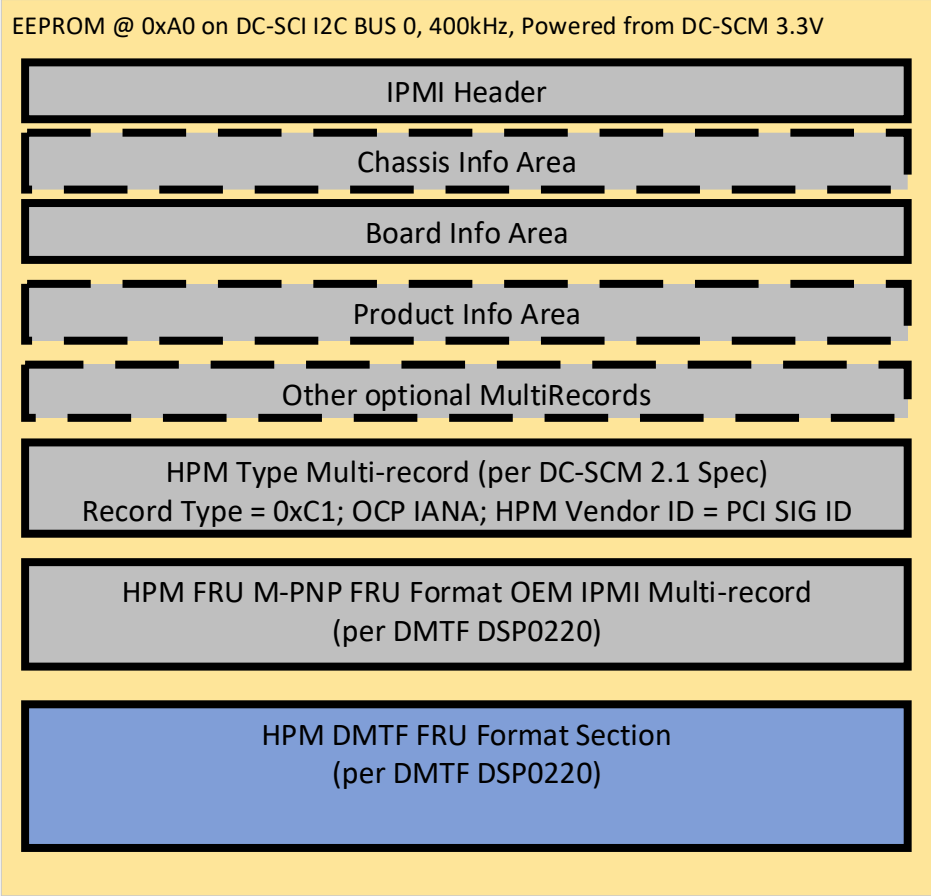


Figure 1: High Level HPM FRU Information Block Diagram

9.1.1. M-PNP HPM FRU IPMI Header

This is the industry standard IPMI header as defined within the *[IPMI Platform Management FRU Information Storage Definition]* specification.

9.1.2. M-PNP HPM FRU IPMI Chassis Area Record

This is the industry standard IPMI chassis area record as defined within the *[IPMI Platform Management FRU Information Storage Definition]* specification. This is an optional section.

9.1.3. M-PNP HPM FRU IPMI Board Area Record

This is the industry standard IPMI board area record as defined within the *[IPMI Platform Management FRU Information Storage Definition]* specification. This is a mandatory section.

9.1.4. M-PNP HPM FRU IPMI Product Info Area Record

This is the industry standard IPMI product area record as defined within the *[IPMI Platform Management FRU Information Storage Definition]* specification. This is an optional section.

9.1.5. M-PNP HPM FRU HPM FRU Multi-record Area Record

This record is defined within the DC-SCM 2.1 or newer specification (see specification for details). This is a mandatory section.

Not only is this multi-record is required as per the DC-SCM 2.1 or newer specification for compliant HPMs, but it is also required for plug-and-play interoperability and will be an M-PNP conformance requirement.

9.1.6. HPM FRU M-PNP DMTF FRU Format OEM IPMI Multi-Record Section

This record format is defined within the DMTF as it uses the proposed DMTF OEM IPMI multi-record format as per the DMTF FRU format (see DSP0220). This is a mandatory section.

The high-level diagram below outlines the M-PNP multi-record.

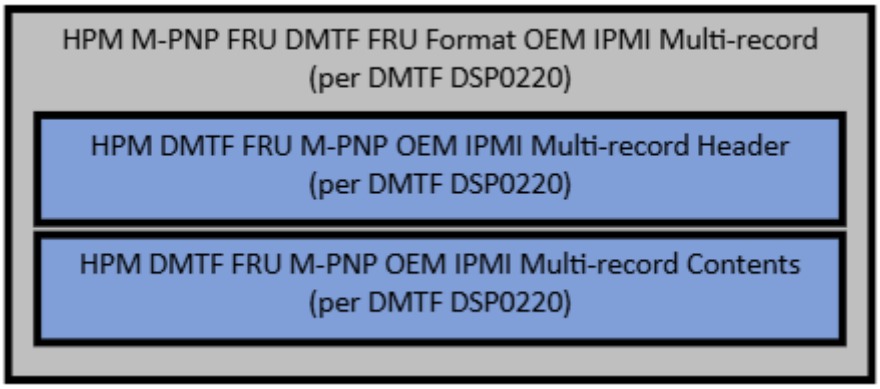


Figure 2: HPM M-PNP FRU DMTF FRU Format OEM Multi-record Block Diagram

There are several sub-sections within this multi-record which are detailed below.

- **HPM DMTF FRU Format OEM Multi-Record Header:** Follows the IPMI specification multi-record header format as outlined in DSP0220.
- **HPM DMTF FRU Format OEM Multi-Record Contents:** Follows the DMTF FRU Format OEM multi-record contents specification as outlined in DSP0220.

For more information regarding this format, please consult DSP0220.

9.1.6.1. HPM FRU M-PNP DMTF FRU Format OEM Multi-record Header

This format is defined in DSP0220 and contains the header for this multi-record and follows the standard IPMI multi-record header format.

9.1.6.2. HPM FRU DMTF M-PNP FRU Format OEM Multi-record Contents

This format is defined in DSP0220 and contains the pointer to the starting offset for the DMTF FRU Files Format section of the FRU image.

9.1.7. M-PNP HPM FRU OCP M-PNP DMTF FRU Format Section

This section outlines the OCP M-PNP standards specific data block. This data block follows the DMTF FRU Record format defined in DSP0220 and contains M-PNP specific files. An overview of the DMTF FRU record format is shown in the diagram below. Solid lines indicate required sections, and dotted lines mean those sections are optional.

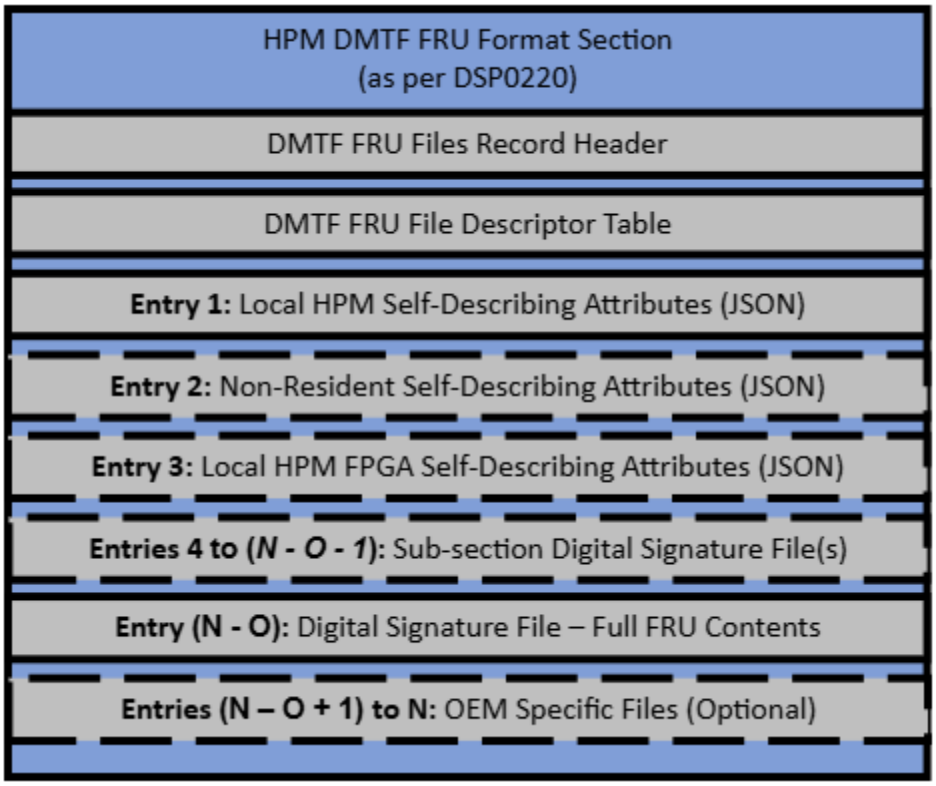


Figure 3: Block diagram of the M-PNP HPM DMTF FRU Format Section

Below contains an overview description of each section of the M-PNP Standards Specific Data Block. For these entries, *N* is the total number of files in the File Descriptor Table, *S* is the number of sub-section signature files, and *O* is the number of OEM files. For this table, $N = S + O + 4$.

- **FRU Files Record Header**: Contains the header and the offset to the start of the file descriptor table (see DSP0220 for details).

- **File Descriptor Table:** Contains entries for each file within the multi-record. Each entry has a pointer offset and size of the file. Each entry can have an optional hash in SHA 256 format (see DSP0220 for details).
- **File Entry 1:** Required self-describing FRU local attributes in JSON format for the HPM.
- **File Entry 2:** If present, this entry provides information indicating that there exists non-resident (meaning not within the FRU information device) extended FRU attributes in a JSON file stored in a separate location. This entry provides sufficient information as to where it is located and how one would navigate to collect that non-resident file.
- **File Entry 3:** Optional resident Primary HPM Programmable Device JSON attributes in JSON file stored as initial factory version of the image JSON within the FRU information device applied during the manufacturing process. See M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification for details.
- **File Entries 4 to (*N – O - 1*)**: Contains S signatures for individual data files (optional).
- **File Entry (N - O)**: Contains a mandatory signature that covers the entire FRU contents (static data only) including static IPMI records.
- **File Entries (N – O + 1) to N:** Contains O number of optional data files in any format desired by system implementor.

For more information regarding this format, please consult DMTF DSP0220 specification.

9.1.7.1. HPM FRU M-PNP DMTF FRU Format Section Header

This section contains the format and description for the DMTF FRU Files Format section header. For more information regarding this format, please consult DSP0220.

9.1.7.2. HPM FRU M-PNP Multi-record File Descriptor Table

This section contains the format and description for the M-PNP multi-record file descriptor table. For more information regarding this format, please consult DSP0220.

9.1.7.3. M-PNP Multi-record HPM FRU Format File Descriptor GUIDs

This section contains the set of file descriptor GUIDs used by M-PNP conformant HPMs.

Table 5: M-PNP FRU Format File Descriptor GUIDs Table

GUID Name	GUID	Description

HPM and Peripherals Self-describing Attributes (JSON) File	b0f252dc-31ad-413c-a42c-3bc97ccdd361	This is the GUID that references the M-PNP self-describing attributes file format which uses the self-describing attributes JSON schema for the HPM and peripherals. There may be two files of this type with the first (smaller) being the local attributes and the second (larger) being extended attributes.
HPM and Peripheral Programmable Logic Devices Self-describing Attributes (JSON) File	572b3d99-d422-4cfe-a898-aa5fc43a46c6	This is the GUID that references the M-PNP self-describing attributes file format which uses the self-describing attributes JSON schema for the primary HPM programmable device and other programmable devices.
Digital Signature	befea991-9f2b-4467-81b9-4835b1ad90f6	This GUID references the M-PNP FRU digital signature file and associated format.

9.1.7.4. M-PNP HPM DMTF FRU Standards Specific Data File Entry 1: Local Self-Describing JSON Data

This section contains the format and description for the M-PNP initial multi-record file entry. This entry will always contain the minimum required local self-describing JSON data payload. Additional JSON data payloads can be optionally stored and provided within subsequent file entries. Examples of additional payloads could include any extended attributes that a system integrator may want to store locally on the FRU information device.

Section 12, “Component Self-Describing JSON Attributes,” contains the definition of the JSON self-describing data and the HPM minimum required data set for local storage in the FRU information device.

9.1.7.5. M-PNP HPM DMTF FRU Standards Specific Data File Entry 2: Non-Resident Self-Describing JSON Data

This section contains the description for the M-PNP HPM non-resident JSON self-describing attributes file. This file typically contains the additional extended attributes that a system integrator may want to store outside the FRU information device.

9.1.7.6. M-PNP HPM DMTF FRU Standards Specific Data File Entry 3: Resident Self-Describing JSON Data for Primary HPM Programmable Device Initial Factory Image

This section contains the description for the M-PNP Primary HPM Programmable Device JSON self-describing attributes file associated with the initial factory image of the Primary HPM Programmable Device. The initial Primary HPM Programmable Device factory image shall have an associated JSON file which contains the full JSON set of attributes within a single file. This file may be stored and accessed by the BMC via accessing Primary HPM Programmable Device main memory using a well-defined access mechanism (see M-PNP.HPM and Peripheral Programmable Device specification for details).

In order to reduce the memory footprint associated with the storage of the full Primary HPM Programmable Device self-describing JSON file, it is allowed to store the initial version of the Primary HPM Programmable Device JSON file within the HPM FRU EEPROM. The figure below depicts this implemented using an FPGA device as the primary programmable logic device. This file is intended to provide an initial baseline of the Primary HPM Programmable Device JSON description which can be used in conjunction with Primary HPM Programmable Device provided patch files to construct the current JSON representation of the Primary HPM Programmable Device image. The Primary HPM Programmable Device memory will only store the patch files when the PD image is updated. This is used as a means to reduce the overall required Primary HPM Programmable Device memory footprint for JSON file storage as only the patch files need to be stored in Primary HPM Programmable Device memory. The system implementor may implement this as a memory saving feature..

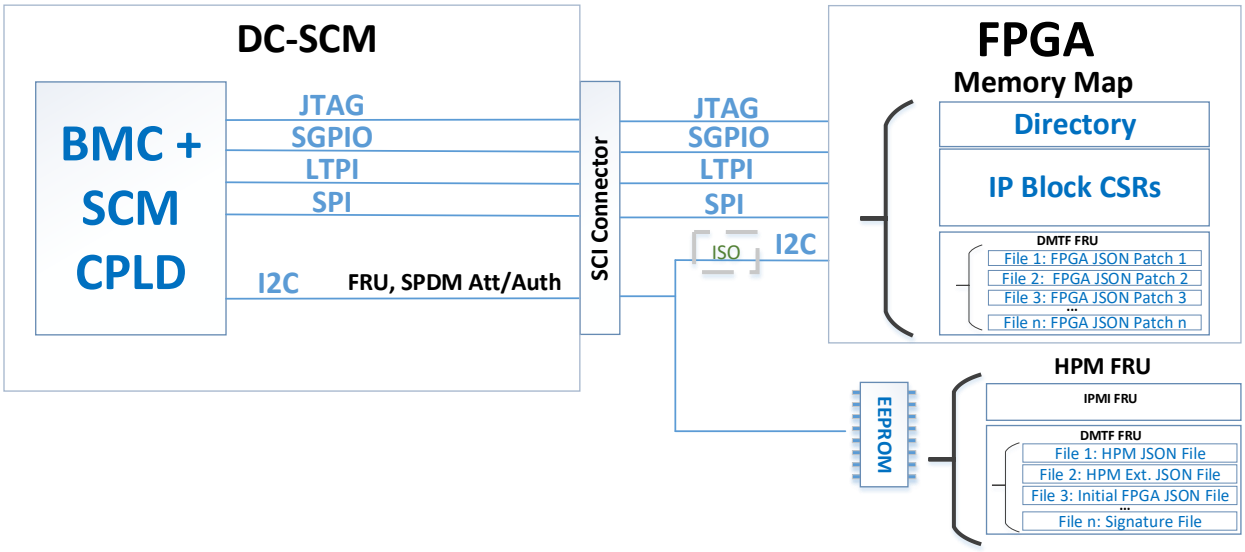


Figure 4: Example Primary HPM Programmable Device JSON Patch Optional Delivery

See M-PNP.HPM and Peripheral Programmable Device specification for further details.

9.1.7.7. HPM DMTF FRU M-PNP File Entries 4 to (N – O - S): Subsection Digital Signatures

This section contains digital signatures for the individual FRU sub-sections. See section 11, “FRU Digital Signature”, for more details regarding the FRU digital signature.

9.1.7.8. HPM DMTF FRU M-PNP File Entries (N - O): FRU Digital Signature

This section contains the digital signature for the entire FRU information device contents. It is required that the signature represents the static information contained in the FRU image. An implementation may cover dynamic information, but any dynamic re-signing of the contents is outside the scope of this specification. See section 11, “FRU Digital Signature”, for more details regarding the FRU information device and the FRU digital signature.

9.1.7.9. HPM DMTF FRU M-PNP File Entries (N – O + 1) to N: OEM Files

Subsequent file sections are optional and may be used to store extended attributes or other OEM data not specified within this document.

9.2. HPM FRU Write Protection Requirements

In many applications, HPM FRU information is intentionally not to be modifiable by a system BMC and thus the FRU information device may be hard strapped for write protection.

In some applications, there is a desire for hybrid FRU, meaning a partial area that is static and write protected and a partial area where the system can write. The usage of a hybrid FRU information device with mutable sections is outside of the scope of this specification.

It is recommended but not required that the HPM Primary HPM Programmable Device connects an override signal to the HPM FRU write protect input. The circuit should default to write protected.

BMC may instruct the HPM Primary HPM Programmable Device to override the WP signal via a BMC memory map bit with controls such as automatic reassertion when a BMC reset is detected (to prevent accidental writes).

9.3. M-PNP HPM Virtual FRU Device Considerations

This section provides details regarding the usage of virtual FRU support through EEPROM device emulation.

If a virtual FRU is used, updates to the virtual FRU emulation may require a firmware update. From the BMC perspective, it should be transparent as to whether the FRU device is a physical or virtual storage device. Handshaking or other synchronization mechanisms to ensure availability during a firmware update process for the virtual device is out of scope for this specification.

9.4. HPM Self-Description Requirements

It is required that the HPM provides a self-describing JSON file within the HPM EEPROM information device to support the platform discovery process. Please see section 12, “Component Self-Describing JSON Attributes”, for more details on the specific structure and attributes for the HPM JSON file. See the JSON schema documentation for additional details regarding specific Hardware or Primary HPM Programmable Device JSON attributes.

Step notation: Each process step is prefixed by one or more bracketed qualifiers.

Power-state qualifiers name the system power state(s), as defined in the M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification, in which the step is permitted to execute: `PreSTBY`, `STBY (Hold-Off)`, `STBY (Run)`, `S5 (Hold-Off)`, `S5 (Run)`, `S0 (Hold-Off)`, `S0 (Run-BIOS-Boot)`, `S0 (Run-OS)`. Where several are listed, the step may execute in any of them; a step shall not execute in a state not listed.

Applicability qualifiers: `Required` (default when omitted) — the step shall be performed; `Optional` — the step may be omitted, and any later step that depends on its result shall then be treated as having failed; `Security` — the step is required only when the platform security profile requires it.

9.5. HPM Discovery Process

The following section outlines the discovery process for the HPM.

9.5.1. System Discovery Method Overview

The diagram below outlines the overall system discovery methodology.



Figure 5: Breadth-first Discovery Process Overview

The discovery process is generally breadth-first, spreading from the SCM to the HPM and its control plane including the Primary HPM Programmable Device, to intermediate boards such as risers and backplanes, and then collecting the options that form the leaves of the overall system topology tree.

System boot is partially intertwined with the discovery process, but most of the discovery process is completed before system boot is performed.

9.5.2. Discovery Phase 0: SCM Power and Initialization

In Phase 0, the SCM and its BMC boot and initialize themselves.

9.5.3. SCM initialization

This section contains the process steps in the early SCM initialization.

- [**G3**] No System or PSU Input power. The only power available at this point is from the battery backup (such as a coin cell)
- [**PreSTBY**] DC-SCM fully seated enables the HPM eFuse (if present), providing power to the DC-SCM. Pre standby power available.
    - Assumption: System designer has ensured that there is sufficient power to power everything that needs standby power in the HPM
    - HPM designs shall document how much standby power they require; SCM designs shall document how much standby power they can provide.
- [**PreSTBY**] SCM AUX VRs regulate, including VCC_SCM_HPM_FRU
- Note: Any failure to properly power the SCM at this state is a hardware failure. Error indication and remediation are defined by vendor policy and not within the scope of this specification.

9.5.4. SCM BMC Early Boot Process (Boot Phase 0)

This section contains the process steps for the early SCM boot process of the BMC.

- [**PreSTBY, Optional, Security**] Use Silicon Root of Trust to validate SCM BMC firmware.
- [**PreSTBY, Optional, Security**] SCM BMC now trusts itself.
    - SCM BMC may start in the, “PRE-STBY”, power state and will wait to transition to “STBY HOLD-OFF” power state until it asserts the HPM_STBY_EN signal.
- [**STBY**] SCM BMC completes the minimum initialization required for Phase 1.
- [**STBY**] Interface with SCM CPLD
    - Interface and protocol are out of scope for this specification.
    - Establishment of trust from SCM BMC to its CPLD is out of scope for this specification.

- Note: Any errors during the SCM early boot process at this stage shall be considered to be a hardware failure. The BMC should report and log that this failure occurred. The format and informational details regarding reporting and logging are out of scope of this specification. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.

9.5.5. Discovery Phase 1: HPM Discovery

In Phase 1, the zone of discovery expands from just the SCM out to the HPM board and its onboard components (EEPROM, Primary HPM PD, etc.). The figure below illustrates this with the primary HPM programmable device implemented within an FPGA device.

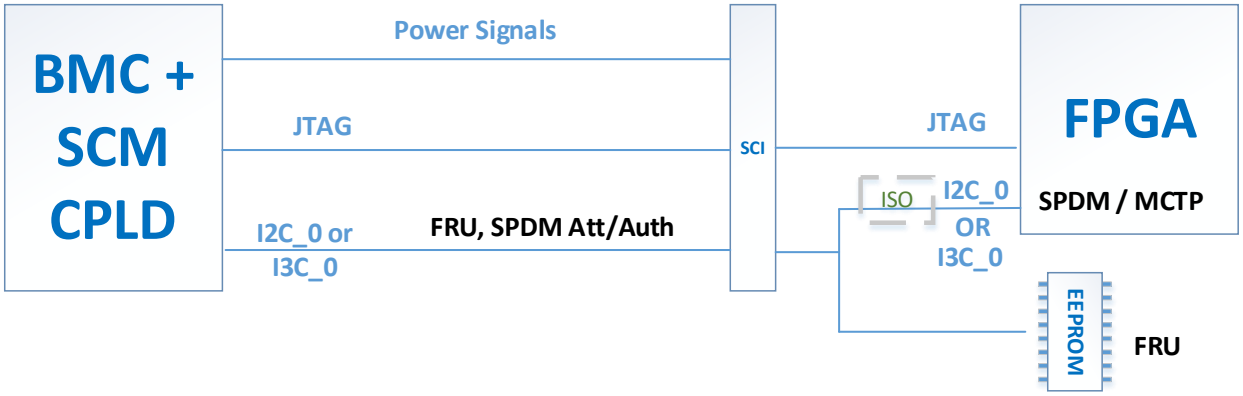


Figure 6: Example Early HPM Discovery Block Diagram

See the DC-SCM version 2.1 or newer specification for details regarding these interfaces including which power states they must be operational in. For connection to the PDB, the interfaces provided by the PDB management connector to the HPM shall be powered during standby. See the M-PIC version 1.1 or newer specification for details.

9.5.6. Early HPM Discovery Process

This section outlines the early HPM discovery process. The following diagram outlines the key steps in the process.



Figure 7: Early HPM Discovery Process Flow Diagram

[1]

The steps illustrated in the flow diagram, with references to the appropriate system power state, are as follows below:

- [**PreSTBY**] SCM BMC boots (preceded by establishment of a root of trust (RoT) if it exists) to a point where it is capable of accessing the HPM FRU on I2C bus 0.
- [**PreSTBY**] Retrieve HPM IPMI FRU from EEPROM or EEPROM-like device.
    - fixed location: I2C bus 0 address 0xA0 (8-bit write address; 7-bit address 0x50), as mandated by the DC-SCM 2.x specification.
    - it is required that the I2C bus 0 be operational during the pre-standby power state including the FRU information device.
    - A failure to properly read the FRU EEPROM at this stage shall be considered to be a discovery process failure. The BMC shall halt the discovery process. The BMC should report and log that this failure occurred. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.
- [**PreSTBY, Optional, Security**] Provisionally trust the FRU data until we can establish trust using standard security mechanisms.
    - If HPM DMTF FRU has at least one digital signature file, then the BMC can retrieve these to perform authentication of IPMI and DMTF FRU data.
    - The BMC can optionally use SPDM as an additional mechanism to attest to the HPM FRU data. The inability to establish an SPDM session shall be considered a security failure. The BMC should report and log that this failure occurred. The vendor may continue the platform discovery process according to their policy.
    - Note: A failure to properly read, verify, and utilize the signature of the FRU EEPROM at this stage shall be considered to be a discovery process failure. The BMC should report and log that this failure occurred. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.
- [**PreSTBY**] SCM verifies compatibility with HPM.

[1] *Note: Electrical handshaking is detailed in DC-SCM specification version 2.1 or newer. See DC-SCM 2.1 or newer specification for details.

- Detection of an incompatible HPM by the BMC shall result at this stage shall be considered to be a discovery process failure. The BMC should report and log that this failure occurred. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification. Continue electrical handshaking with HPM:
    - [**PreSTBY**] SCM asserts SCM_HPM_STBY_EN
        - SCM BMC transitions to the, “STBY (HOLD-OFF)”, power state (see M-PNP.HPM and Peripheral Programmable Device specification for details)
        - Failure to properly assert this signal by the SCM at this stage i shall be considered to be a hardware failure. The BMC shall halt the discovery process. The BMC should report and log that this failure occurred. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.
        - Note: Specific timeout logic for this handshake signal is based on vendor policy and outside the scope of this specification.
    - [**STBY**] HPM asserts HPM_SCM_STBY_RDY when all HPM AUX Rails are within regulation.
        - Failure to properly assert this signal by the HPM at this stage shall be considered to be a hardware failure. The BMC shall halt the discovery process. The BMC should report and log that this failure occurred. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.
        - Note: Specific timeout logic for this handshake signal is based on vendor policy and outside the scope of this specification.
    - [**STBY, Optional, Security**] SCM attests or otherwise verifies the authenticity of the Primary HPM Programmable Device.
        - This discovery process step is outlined in detail in section 9.5.6, “Static Primary HPM Programmable Device Data Collection.”
        - Further details regarding this process can be found in the M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification.
        - Failure to authenticate the Primary HPM Programmable Device image by the BMC at this stage shall be considered to be a security failure. The BMC shall halt the discovery process. The BMC should report and log that this failure occurred. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.
    - [**STBY**] SCM de-asserts SCM_HPM_STBY_RST_N
        - SCM BMC transitions to the, “**STBY (RUN)** “, power state (see MPNP. HPM and Peripheral Programmable Device specification for details)
        - Failure to properly assert this signal by the BMC at this stage shall be considered to be a hardware failure. The BMC shall halt the discovery process. The BMC should report and log that this failure occurred. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.
        - Note: Specific timeout logic for this handshake signal is based on vendor policy and outside the scope of this specification.

    - [**STBY (Hold-Off), STBY (Run)**] standby-classified interfaces may engage (see interfaces located in DC-SCM 2.1 or newer specification). HPMs and SCMs may choose to use SCM_HPM_STBY_RST_N as the qualifier for engaging standby RUN allowed DC-SCI signals.
        - SCM BMC may transition to the **S5 (HOLD OFF**) power state provided all of the standby powered requirements for on-board HPM devices are met. The BMC may enable the PSU 12V primary voltage rail based on device power needs in order to proceed to S5 (Hold Off). See the M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification for further power sequencing details.
        - Note: Failure by the HPM to transition to S5 (HOLD OFF at this stage shall be considered to be a hardware failure. The BMC shall halt the discovery process. The BMC should report and log that this failure occurred. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.
- [**STBY (Hold-Off), Optional**] BMC retrieves local HPM JSON attributes files from wherever it is located using one of the following methods, with required power level as noted:
    - Via PLDM File Transfer (DSP0242) using methods defined in PLDM FRU Access (DSP0257 v2.0) if device is powered and accessible in standby
    - From the HPM FRU or a location indicated by it:
        - The beginning of the DMTF FRU Files Record may be identified by getting its offset from the DMTF record in the IPMI FRU previously retrieved.
        - The File Descriptor Table is contained within the FRU Files Record. The File Descriptor in slot 1 of the table represents the local JSON attributes.
        - If the Resident bit in the Flags field of the File Descriptor is set to 1b, the file is present in the EEPROM or EEPROM-like device from which the IPMI data was retrieved.
            - The Offset field in the File Descriptor is the location of the beginning of the FRU data.
        - Else, if the Resident bit in the Flags field of the File Descriptor is set to 0b, the file is not present in the EEPROM or EEPROM-like device from which the IPMI data was retrieved.
            - The two high-order bits in the Context field of the File Descriptor describe the location of the FRU data:
            - Value 11b indicates that the data is present in a virtual EEPROM emulated by the Primary HPM Programmable Device. The remainder of the Context field is the actual offset within the virtual EEPROM.
            - Value 10b indicates that the data is present in a BIOS firmware volume (FV). The remainder of the Context field identifies the specific BIOS firmware volume via mechanism outlined in section 9.5.6.1, “FRU Information Device Non-local Extended JSON File BIOS Firmware Volume Usage Guidance and Considerations”.
            - Values 01b and 00b are reserved for future use.
        - The Size field in the File Descriptor is the length of the FRU data.

    - From a cache or store in the SCM tied to the hardware ID of the HPM or other identifying information.
    - From a networked resource, based on the hardware ID of the HPM or other identifying information.
        - Specific locations, transport mediums, protocols, and other networking specific details are vendor defined and out of scope for this specification.
    - Note: Failure to retrieve the required local HPM JSON file by the BMC at this stage shall be considered to be a discovery process failure. The BMC shall halt the discovery process. The BMC should report and log that this failure occurred. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.
- [**STBY (Hold-Off), STBY (Run), Optional**] BMC retrieves extended HPM JSON attributes file from wherever it is located using one of the following methods, with required power level as noted:
    - Via PLDM File Transfer (DSP0242) using methods defined in PLDM FRU Access (DSP0257 v2.0)
    - From the HPM FRU or a location indicated by it:
        - The beginning of the DMTF FRU Files Record may be identified by getting its offset from the DMTF record in the IPMI FRU previously retrieved.
        - The File Descriptor Table is contained within the FRU Files Record. The File Descriptor in slot 2 of the table represents the extended JSON attributes.
        - If the Resident bit in the Flags field of the File Descriptor is set to 1b, the file is present in the EEPROM or EEPROM-like device from which the IPMI data was retrieved.
            - The Offset field in the File Descriptor is the location of the beginning of the FRU data.
        - Else, if the Resident bit in the Flags field of the File Descriptor is set to 0b, the file is not present in the EEPROM or EEPROM-like device from which the IPMI data was retrieved.
            - The two high-order bits in the Context field of the File Descriptor describe the location of the FRU data:
            - Value 11b indicates that the data is present in a virtual EEPROM emulated by the Primary HPM Programmable Device. The remainder of the Context field is the actual offset within the virtual EEPROM.
            - Value 10b indicates that the data is present in a BIOS firmware volume (FV). The remainder of the Context field identifies the specific BIOS firmware volume via the mechanism outlined in section 9.5.6.1.
            - Values 01b and 00b are reserved for future use.
        - The Size field in the File Descriptor is the length of the FRU data.
    - From a cache or store in the SCM tied to the hardware ID of the HPM or other identifying information.
    - From a networked resource, based on the hardware ID of the HPM or other identifying information.

- Note: Failure to retrieve the optional extended HPM JSON file by the BMC at this stage shall be considered to be a discovery process failure. The BMC should report and log that this failure occurred. The format and informational details regarding reporting and logging are out of scope of this specification. The BMC may continue the platform discovery process according to their policy. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.

Please refer to the power state transition diagram in the M-PNP HPM and Peripheral Programmable Device Design Specification for further details.

9.5.6.1. FRU Information Device Non-local Extended JSON File BIOS Firmware Volume Usage Guidance and Considerations

This section provides guidance and considerations regarding the usage of the BIOS firmware volume for storage of non-local extended JSON self-describing attributes files.

If it is indicated that extended non-local JSON is stored within a BIOS firmware volume, the GUID of this file and firmware volume will be provided in the DTMF FRU format file table. The BMC shall retrieve the BIOS firmware volume via the BIOS image using standard practices outlined in the UEFI Platform Initialization (PI) Specification. See UEFI specification volumes 3, “Shared Architectural Design”, and 4, “Firmware Volume/File System”, for further details..

For details regarding GUID storage within the DMTF FRU format, see the DMTF DSP0220 specification for further information.

9.5.6.2. File Structure for Firmware Volume Navigation

The following section outlines the necessary details provide navigational reference material to determine the location of non-resident files.

The BIOS firmware volume uses a directory structure that depends on GUIDs to uniquely identify the volume, file structure, and file within the BIOS image.

The UEFI Platform Initialization (PI) Specification Volume 3 (Shared Architectural Elements) defines the architecture for Firmware Volumes (FVs) and Firmware File Systems (FFSs) and how to package arbitrary files into the BIOS FV, See that specification for further details.

In order to reference the non-resident file within a FV, the following information shall be packaged in the DMTF FRU format as a specific JSON file.

- FV GUID: Each volume begins with an EFI_FIRMWARE_VOLUME_HEADER containing a unique filesystem GUID, signature (_FVH), length, and block maps.

- FV File GUID: A unique 128-bit ID. This is how the FFS identifies your file inside the ROM.
- Packaged File GUID: This is the file that is packaged within the DMTF FRU format within the FV file.

The following diagram describes the relationship of these GUIDs to the associated objects.

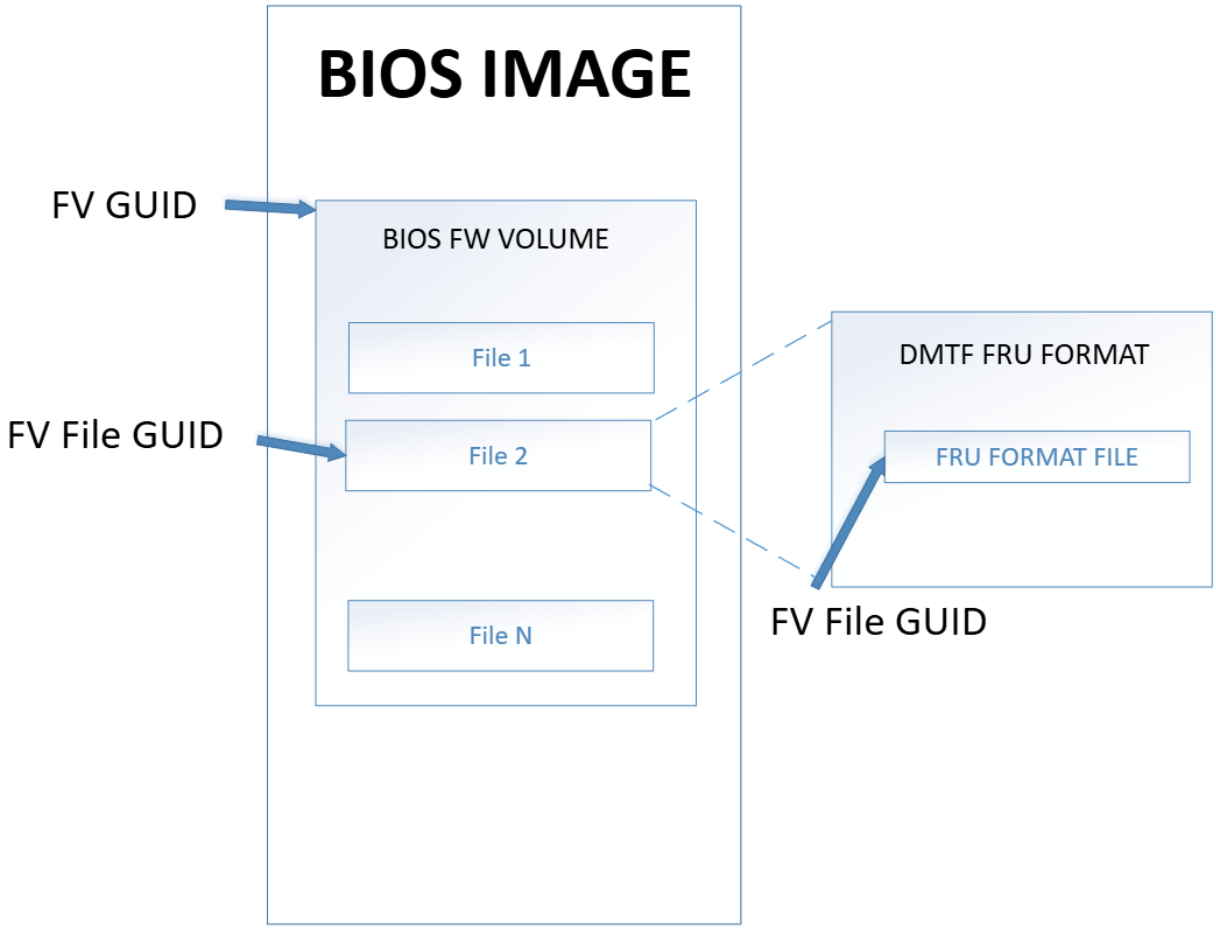


Figure 8: Firmware Volume Conceptual Diagram

The file packaged into the firmware volume shall use the DMTF FRU file format and it shall be built using the EFI_FV_FILETYPE_RAW file type to indicate it is a binary packed file.

See UEFI BIOS specifications for further details.

9.5.7. Establishing Connection from SCM to HPM Primary HPM Programmable Device

This section outlines the process by which the SCM discovers the Primary HPM Programmable Device. The following diagram outlines the key steps in the process.

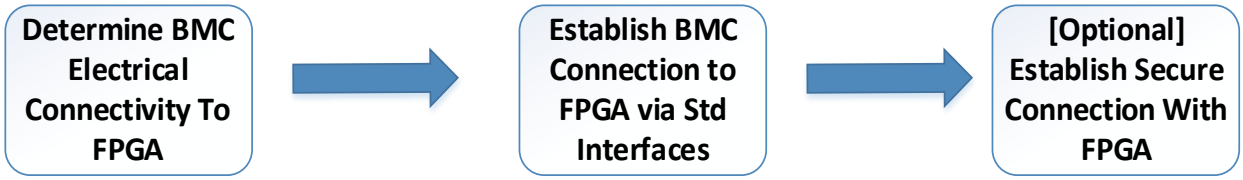


Figure 9: Primary HPM Programmable Device Connectivity Establishment Process Diagram

The steps illustrated in the flow diagram are as follows:

- [**STBY (Hold-Off), STBY (RUN), Optional**] Get the electrical connectivity to the HPM Primary PD from the HPM FRU
    - [**Preferred**] Comes from HPM extended FRU data.
        - In Devices section, find device of Type: Primary HPM Programmable Device with Primary: true.
        - The 'ConnectedBuses' JSON property for this device gives the bus and signal information for connection. See JSON schema documentation for further details.
    - [**Legacy**] Using HPM Vendor and Product IDs, use look up table in BMC to determine what HPM components are connected to the Primary HPM Programmable Device through SCI
        - BMC uses bus discovery processes for LTPI and SGPIO to determine versioning and capabilities of components.
            - Note: This interface may not be available and ready if the PD is still held in reset due to security and attestation requirements.
        - If SPI is connected to the Primary HPM Programmable Device, then treating the PD as a SPI device provides random access to PD memory.
            - Note: This interface may not be available and ready if the PD is still held in reset due to security and attestation requirements.
    - Note: Failure to retrieve the necessary information needed by the BMC to determine the electrical connectivity to the Primary HPM Programmable Device at this stage shall be considered to be a discovery process failure. The BMC shall halt platform discovery. The BMC should report and log that this failure occurred. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.
- [**STBY (Run)**] Establish electrical connection to Primary HPM Programmable Device via bus-specific initiation and transactional protocols.
- [**STBY (Run), Optional, Security**] Primary HPM Programmable Device is known to be either at a fixed location (such as a particular address in I2C bus 0) or at a location

identified by the HPM FRU. Further details can be found in the M-PNP. HPM and Peripheral Programmable Device specification.

- [**STBY (Run), Optional, Security**] Establish SPDM connection to HPM's Primary Programmable Device. Further details can be found in the M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification. T
    - The inability to establish an SPDM session shall be considered a security failure. The BMC should report and log that this failure occurred. The BMC may continue the platform discovery process according to vendor policy.
- [**STBY (Run), Optional, Security**] SCM now trusts HPM's Primary Programmable Device
- Note: Failure to communicate with the Primary HPM Programmable Device at this stage shall be considered to be a discovery process failure. The BMC shall halt platform discovery. The BMC should report and log that this failure occurred. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.

9.5.8. Discovering the Primary HPM Programmable Device

This section outlines the process by which the Primary HPM Programmable Device is discovered by the SCM BMC. The following block diagram illustrates the interconnections used in the discovery process with the primary HPM programmable device implemented using an FPGA.

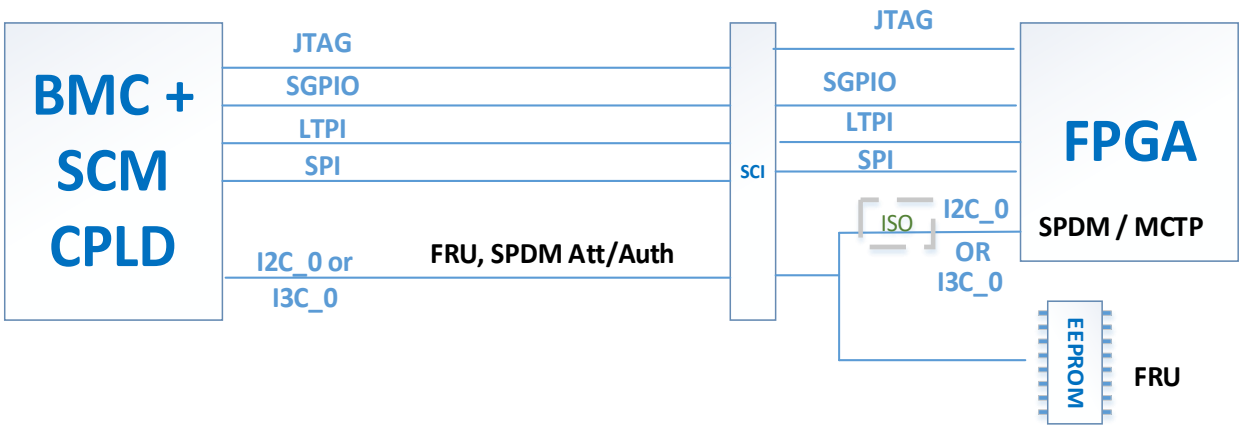


Figure 10: Primary HPM Programmable Device Discovery Block Diagram

The following diagram outlines the key steps in the process.

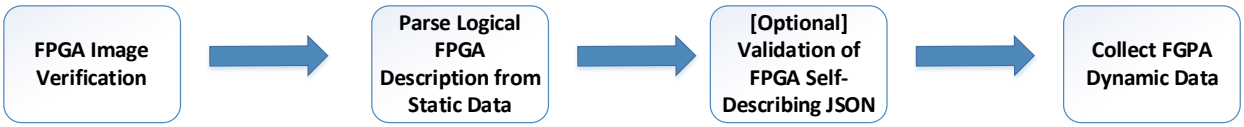


Figure 11: Primary HPM Programmable Device Discovery Process Diagram

9.5.9. Static HPM Primary Programmable Device Data Collection

This section contains the process steps for collecting the static Primary HPM Programmable Device descriptive data used by the BMC during the discovery process of the system.

- [**STBY (Hold-Off)**] Prior to releasing reset to the HPM, which includes the Primary HPM Programmable Device (see section 9.5.6), BMC shall use JTAG or another low-level interface to get identifying information and to perform image verification. It is required that this interface be available while the Primary HPM Programmable Device primary user defined logic is held in reset. See M-PNP M-PNP.HPM and Peripheral Programmable Device design specification for further details.
    - Available low-level interfaces, such as JTAG, shall be identified in the HPM JSON file.
    - Failure to communicate with the Primary HPM Programmable Device to collect and verify compatible identifying information and image at this stage via one or more low-level interfaces shall be considered to be a discovery process failure. The BMC shall halt platform discovery. The BMC should report and log that this failure occurred. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.
- [**STBY (Hold-Off)**] BMC verifies that it is compatible with the Primary HPM Programmable Device code version(s).
    - A verification failure at this stage by the BMC shall be considered to be a discovery process failure. The BMC shall halt the discovery process. The BMC should report and log that this failure occurred. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.
- [**STBY (Hold-Off), STBY (RUN)**] Get the logical description of the Primary HPM Programmable Device (VGPIOs, mem map, etc.). This can happen in several different ways:
    - [**Offline, Non-Self Describing**] Option A: The use of a GUID provided by the Primary HPM Programmable Device that corresponds to a table in firmware. This table may be contained by the BMC or in an accessible memory device on the SCM. There may be a single GUID that represents the entire Primary HPM

Programmable Device contents and there may be supplementary GUIDs that represent multiple IP blocks contained by the Primary HPM Programmable Device. Details on how the SCM BMC recognizes and interprets the GUID are out of scope for this document.

- [**Offline, Partial Self Describing**] Option B: File provided to BMC at factory build time that describes the Primary HPM Programmable Device + any 'patch' files that are provided in the Primary HPM Programmable Device update package.
  - This set of files still results in a complete JSON description of the Primary HPM Programmable Device.
  - If the SCM is replaced in the field, then it is required to go back through a (re)deployment process, and it has to get the matching Primary HPM Programmable Device image with the description file from some repo or pre-loaded.
  - See M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE design specification for further details.
- [**STBY (RUN), Full Self Describing**] Option C: Primary HPM Programmable Device contains a self-describing JSON file resident and accessible in an extended memory space within the PD memory map packaged and created at build time and provided as part of its FRU data:
  - The Primary HPM Programmable Device DMTF FRU data is located within the Primary HPM Programmable Device main memory at a location specified by its directory (as specified in M-PNP HPM and Peripheral Programmable Device Design Specification)
    - Access to the Primary HPM Programmable Device main memory is via an interface as specified in the HPM JSON's PD device entry (e.g. LTPI)
    - This allows a signature to be carried with the JSON data as needed per security policy.
    - This enables updates to the JSON data to be stored as a series of patches, without requiring replication of the entire JSON file data.
  - The PD's DMTF FRU files table slot 1 represents the Primary HPM Programmable Device JSON data.
  - If the Resident bit in the Flags field of the File Descriptor Data entry is set to 1b, the PD JSON file is available in Primary HPM Programmable Device main memory.
    - The Offset field of the File Descriptor Data entry describes the location of the FRU data in the Primary HPM Programmable Device main memory.
    - The Size field in the File Descriptor Data entry is the length of the FRU data.
  - If the Resident bit in the Flags field of the File Descriptor Data entry is set to 0b, the Primary HPM Programmable Device JSON file is not present in PD main memory.
    - The two high-order bits in the Context field of the File Descriptor Data entry describe the location of the FRU data:
      - Value 11b indicates that the data is present in EEPROM or EEPROM-like device that contains the HPM FRU data.

Slot 3 in the File Descriptor Data table for the HPM FRU details the location of the Primary HPM Programmable Device FRU data. The remainder of the Context field is the offset where the FRU data is located.
- Value 10b indicates that the data is present in a BIOS firmware volume (FV). The remainder of the Context field is the offset within the FV image where the FRU data is located.
- Values 01b and 00b are reserved for future use.

- The Size field in the File Descriptor Data entry is the length of the FRU data.

- If SCM is field replaced, then it just goes and reads it from the Primary HPM Programmable Device directly.
- Primary HPM Programmable Device patch files contain information on updating the PD FRU data, corresponding to each update. Each subsequent file in the File Descriptor Data table corresponds to a patch that shall be applied in order to reconstruct the current JSON data. The location of patch data may be found in the same manner as the location of the base PD JSON file.

- Note: Failure to communicate with the Primary HPM Programmable Device and collect and utilize the PD JSON file and required information at this stage shall be considered to be a discovery process failure. The BMC shall halt the discovery process. The BMC should report and log that this failure occurred. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.

See M-PNP.HPM and Peripheral Programmable Device Design Specification for further details regarding static data collection.

9.5.10. Dynamic and Extended Primary HPM Programmable Device Data Collection

This section contains the process steps for collecting the dynamic and extended Primary HPM Programmable Device descriptive data used by the BMC during the discovery process of the system.

- [**Security, Optional**] As per system integrator and end-user security policy, the platform may require secure measurement to attest to the platform firmware and configuration data.
  - Measurement may occur each time you AC power cycle the system.
  - At a minimum, a standard digital signature is required to perform the measurement operation for each measured entity, such as firmware images.
  - As per European Union cyber security act and CNSA 2.0, it may be required to provide a Post Quantum Security (PQC) level supporting digital signature as well as a standard digital signature.
  - Secure measurements may occur during several power states based on device and component power needs.

- [**STBY (Run), Optional, Security**] Establish MCTP connection to HPM's Primary Programmable Device (if no secure connection established)
    - MCTP is required for PLDM. MCTP may also be required for SPDM if the vendor discovery policy requires an SPDM connection.,
    - Failure to communicate using MCTP shall be considered to be a discovery process failure if required by vendor policy. The BMC should report and log that this failure occurred. The vendor may continue the platform discovery process according to their policy.
- [**STBY (Run), Optional, Security**] Establish SPDM connection to Primary HPM Programmable Device.
    - The inability to establish an SPDM session shall be considered a security failure if required by vendor policy. The BMC should report and log that this failure occurred. The vendor may continue the platform discovery process according to their policy.
- [**STBY (Run), Optional, Security**] SCM/BMC now trusts the Primary HPM Programmable Device.
- [**STBY (Run)**] BMC retrieves dynamic Primary HPM Programmable Device data via CSRs for each IP Block
- [**STBY (Run)**] BMC parses dynamic data to complete Primary HPM Programmable Device discovery
    - IP blocks
    - memory map and register definition.
    - bus mappings
    - virtual wire mappings for the internal Primary HPM Programmable Device signals to external bus transport and other uses

- Note: Failure to retrieve and/or process the optional extended Primary HPM Programmable Device JSON file by the BMC at this stage shall be considered to be a discovery process failure. The BMC should report and log that this failure occurred. The BMC may continue the platform discovery process according to vendor policy. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.

See M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification for further details.

9.5.11. High-level discovery of the HPM

This section outlines the process by which the HPM is discovered by the SCM BMC once HPM Programmable Device, here shown as an FPGA, discovery is complete. The following block diagram illustrates the interconnections used in the discovery process and is used for reference during the high-level discovery process with the primary HPM PD implemented using an FPGA.

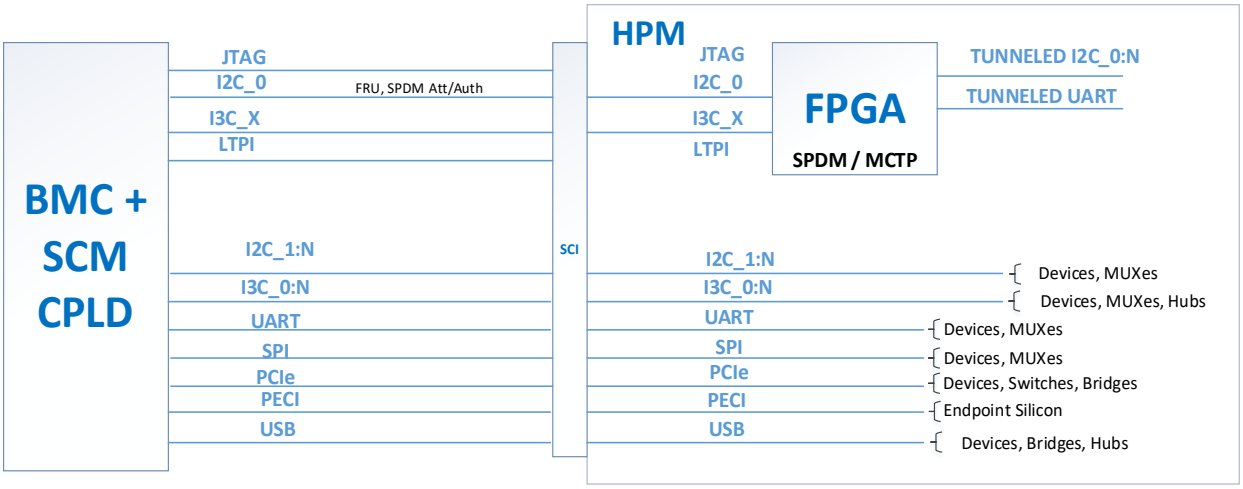


Figure 12: Example High-level HPM Discovery Block Diagram

The following diagram outlines the key steps in the process.

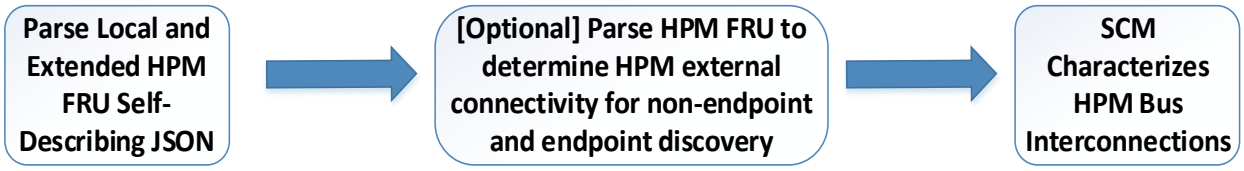


Figure 13: High Level HPM Discovery Process Diagram

The steps illustrated in the flow diagram are as follows:

- [**STBY (Run)**] Identification of HPM inventory using both local and extended FRU data using trusted information.
  - Trust is facilitated using SPDM / security to provide assurance.
  - NOTE: This is dependent on functionality in the HPM Primary Programmable Device (M-PESTI, MUXES, etc.)
- [**STBY (Run), Optional**] Parse HPM Extended FRU (JSON) retrieved from HPM Primary Programmable Device to identify:
  - Location and quantities of expansion slots/connectors.
  - Location and quantities of M-XIO and M-PIC PICPWR connectors.
  - Location, quantity, and type of PDB management connectors and other ancillary board connectors such as front panel control boards.
  - Location, type, and quantity of expected on-board HPM non-programmable device presence and identification.
- [**STBY (Run)**] BMC performs additional bus characterization (e.g., presence and configuration of devices behind a MUX) using bus-specific protocols on the HPM that are active during the standby, STBY (RUN), power state. A list of possible buses that the BMC may characterize is as follows:

  - I2C buses other than bus 0 (which was discovered in an earlier process step)
  - I3C buses
  - UART
  - USB
  - SPI
  - eSPI
  - LTPI
  - SGPIO
  - JTAG
  - PCIe (including CXL support if supported)

  This is not a comprehensive list and others may also be characterized at this stage.

- [**S5 (Hold-Off)**] Transition to S5 (Hold-Off) necessary for on-board device identification and enablement of security features (attestation / authentication) as per security policies.
  - HPM JSON file will provide specific power state guidance for buses and device power state needs.
  - Trust is facilitated using SPDM / security to provide assurance.
  - NOTE: This is dependent on functionality in the Primary HPM Programmable Device (M-PESTI, MUXES, etc.)
- [**S5 (Hold-Off), S5 (Run)**] BMC performs additional bus characterization (e.g., presence and configuration of devices behind a MUX) using bus-specific protocols on the HPM that are active during the S5 (Hold-Off) or S5 (RUN) power states. A list of possible buses that the BMC may characterize is as follows:
  - I2C buses other than bus 0 (which was discovered in an earlier process step)
  - I3C buses
  - UART
  - USB
  - SPI
  - eSPI
  - LTPI
  - SGPIO
  - JTAG
  - PCIe (including CXL support if supported)

  This is not a comprehensive list, and other bus interfaces may also be characterized at this stage.

- [**S0 (Hold-Off)**] Transition to S0 (Hold-Off) may be necessary for on-board device identification and enablement of security features (attestation / authentication) as per security policies.
  - HPM JSON file will provide specific power state guidance for buses and device power state needs.
  - Trust is facilitated using SPDM / security to provide assurance.
  - NOTE: This is dependent on functionality in the Primary HPM Programmable Device (M-PESTI, MUXES, etc.)

- [**S0 (Hold-Off), S0 (Run)**] BMC performs additional bus characterization (e.g., presence and configuration of devices behind a MUX) using bus-specific protocols on the HPM that are active during the S0 (Hold-Off) or S0 (RUN) power states. A list of possible buses that the BMC may characterize is as follows:
    - I2C buses other than bus 0 (which was discovered in an earlier process step)
    - I3C buses
    - UART
    - USB
    - SPI
    - eSPI
    - LTPI
    - SGPIO
    - JTAG
    - PCIe (including CXL support if supported)

  This is not a comprehensive list, and other bus interfaces may also be characterized at this stage.

- Note: Failure to complete a platform inventory process the using the HPM and Primary HPM Programmable Device JSON files by the BMC at this stage is a shall be considered to be a discovery process failure. The BMC should report and log that this failure occurred. The vendor may continue the platform discovery process according to their policy. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.

- Note: Loss of bus communications during runtime after the discovery process is completed for any bus is considered to be hardware failure. The BMC should report and log that this failure occurred. The vendor may continue operations according to their policy. Any further actions including rediscovery and remediation are policy decisions left to the vendor and are out of scope for this specification.

See M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification for further details regarding system power state transitions.

9.5.12. Platform Model Type Determination Considerations

When using a specific type of HPM-SCM pair in multiple system models, some additional information may be needed to determine the final model number. As there are many ways to accomplish this, a specific method is out of scope of this specification and left to the implementer.

10. Peripheral Device FRU Information Usage and Discovery Process

Peripheral types such as backplanes and risers that couple to MHS compliant modules (HPM, PDB, etc.) shall support a FRU Information device on a PICPWR I2C bus. A primary FRU Information Device is required to respond to only I2C static address 0xA6 (8-bit) except in the case of nested peripherals or specialized peripherals such as PCIe Retimer and OCP NIC interposer boards. Please see section 10.14, “Generalized Nested Peripheral Guidance”.

FRU Information devices and the associated bus segments are required to support a minimum of 100Khz with 400KHz[2] preferred during the discovery phase. M-XIO (high speed I/O) connections on peripheral cards do not require a FRU Information device. It is required that the HPM isolate each PICPWR connector through non-programmable, GPIO controlled MUXes to prevent address collision. See the latest M-PIC specification for further details regarding PICPWR requirements.

10.1. Backplane Peripheral Requirements and Considerations

This section outlines specific requirements and considerations for backplane components used within M-PNP conformant servers.

This specification outlines interoperability for backplanes that support only one level of bifurcation. Cable splitting is supported for a single x8 MXIO connector to two x4 cables but further bifurcation (e.g., x8 to four x2 cables) is outside the scope of this specification and left to the vendor to define.

10.1.1. Common Backplane Discovery and Configuration Capabilities Requirements

This section outlines any common requirements for backplanes class peripherals.

Table 6: Common Backplane Requirements

Requirement	REQ	OPT	Description
PICPWR I2C connected FRU information device address shall be assigned 0xA6.	Y		It is required that the backplane provides a FRU EEPROM or EEPROM-like information device at address 0xA6.
FRU EEPROM / Information Device backplane connection via PICPWR I2C bus.	Y		It is required that the FRU EEPROM be on a PICPWR I2C bus.
FRU EEPROM / Information Device not behind a MUX	Y		It is required that the backplane provides a FRU EEPROM or EEPROM-like information device shall not be behind a MUX on the backplane.

[2] Note: The M-PIC specification requires 100Khz support on I2C but does recommend 400Khz operation. Please refer to the M-PIC specification for further information.

Self-describing JSON file contents in FRU EEPROM / Information Device	Y		The local self-describing file shall be present in the FRU EEPROM. See section 10.1.2, “Common Backplane Peripheral Self-Description Requirements”, for details.
Programmable Logic / MCU JSON File if populated	Y*		It is required that the backplane provides a JSON self-describing information file for all programmable devices contained on the backplane. Guidance on how this is accomplished is contained within the MPNP. Primary HPM Programmable Device specification and schema documentation. The location where this file can be collected shall be provided within the FRU EEPROM JSON file.
Source-to-Destination Routing Detection	Y		It is required that the backplane provide Source-to-destination (S2D) routing detection between the PICPWR connector and the MXIO connector(s) CBL_PRES_N signal.
Operational Power State Requirements	Y		It is required that the backplane indicates whether it needs to be operational only in S0 or whether it needs to be powered during S5 or Standby power states. This shall be within the self-describing JSON file.
Maximum Power Requirements	Y		It is required that the backplane provides a maximum power requirement encoded as an attribute within the self-describing JSON file.
M-PESTI Virtual Wires Definition		Y	If the backplane supports PESTI virtual wires, then it shall provide a definition of those virtual wires within the PDB programmable logic (PLD) JSON file.
M-PESTI MUX / Switch Support		Y	It is not required for peripherals to support M-PESTI MUXes.
FlexIO and Sideband Definition		Y	If it is not the defined default for that device class, then it is required to have the definition in the JSON file.
SPDM Support		Y	Peripherals may optionally support SPDM for enhanced security. The JSON file shall contain the I2C / I3C address if SPDM is supported using these buses.

*Note: The JSON file shall be provided only if the peripheral contains one or more programmable devices.

10.1.2. Common Backplane Peripheral Self-Description Requirements

This section outlines the required self-description needed to support backplane peripheral component interoperability.

The following table provides the required and optional self-describing data set needed to promote M-PNP peripheral interoperability. The column, “R”, refers to required attributes, and the column, “L”, refers to local attributes. The allowable values for these columns are Y for yes, N for no, and C for conditional (see descriptions for explanation as to specific conditions).

Table 7: Backplane Peripheral Self-describing JSON Attributes

Data	R	L	Description	Section Reference
Device Class and Subclass	Y	Y	Classification of peripheral encoded as two attributes: Device Class, and Device Subclass.	10.7
Vendor ID	Y	Y	Required general attributes.	12.2.3
Device	Y	Y	List of all devices present in the peripheral including connections to peripheral buses and connectors.	12.2.5
Buses	Y	C[3] (partial)	Array structure of all buses supported by peripheral if present.	12.2.8
Connectors	Y	C[4] (partial)	Listing of all connectors and bus mappings to connectors supported by the peripheral if present.	12.2.4
Flex IO Signals Mappings	Y	Y	Present if unique, non-default assignments are required. These are captured in the MXIO connectors section.	13.12.4
Sideband Signals Mappings	Y	Y	Present if unique, non-default assignments are required. These are captured in the PICPWR connectors section.	13.12.5
M-PESTI Virtual Wires Definition	C	Y	It is required that the PLD JSON file provides the M-PESTI virtual wires definition if the peripheral supports the M-PESTI active phase.	13.6

[3] Note: These sections of attributes require only a sub-set to be local within the FRU information device.
[4] ibid.

M-PESTI MUX / Switch Support	C	Y	JSON file shall indicate if a M-PESTI MUX / Switch is present.	13.6
SPDM Support		Y	Backplanes that support SPDM are required to indicate which interface supports this security feature.	

It is required that implementors provide the device class and subclass of the peripheral. The peripheral shall provide a JSON file, and it required to describe all bus interconnections between devices on the peripheral and connectors and device addresses for each bus device. This is needed for the BMC to properly create the bus topology for each bus. This is accomplished using the JSON descriptive attributes within the self-describing JSON file. See chapter 12, “Component Self-Describing JSON Attributes”, for further details.

10.1.3. UBM Backplane Specific Requirements

This section outlines any specific requirements for Universal Backplane Management (UBM) supporting backplanes.

Table 8: UBM Backplane Specific Requirements

Requirement	REQ	OPT	Description
Universal Backplane Management (UBM) SFF-TA-1005 Specification	Y		Backplanes are required to implement the Industry Standard Universal Backplane Management (UBM) SFF-TA-1005 specification.
UBM FRU Information Device(s) Assigned AEh on MXIO I2C Bus	Y		It is required that the UBM FRU on MXIO connectors is assigned an I2C address of AEh. This is separate from the PICPWR I2C FRU information device.

10.1.4. Example UBM Backplane Block Diagram

The block diagram below contains an example implementation for a UBM backplane.

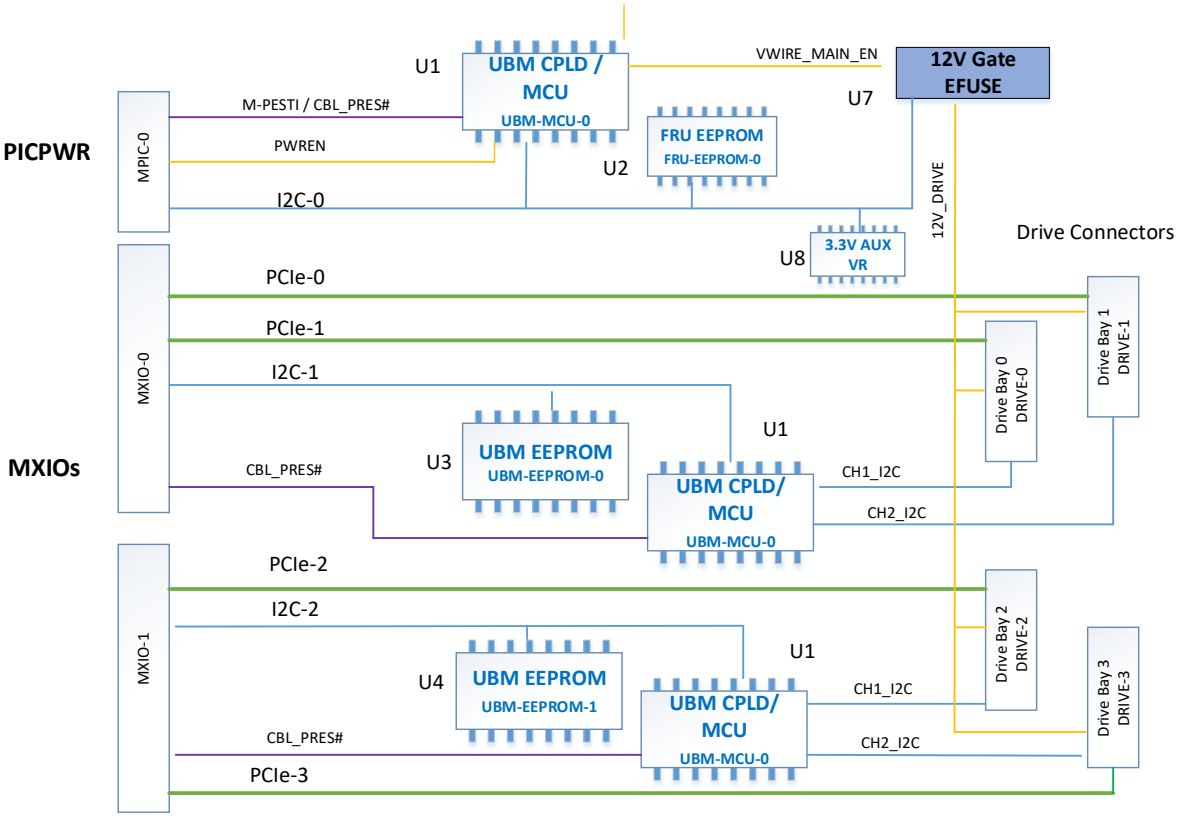


Figure 14: Example UBM Backplane Block Diagram

This example shows the UBM MCU shows as three separate entities for ease of illustration but is, in fact, single component with reference designator, U1. An implementor can use separate MCU and CPLD components but handing MPESTI/CBL_PRES# source-to-destination routing enablement would need to be coordinated between the CPLD connected to the PICPWR and the MCU devices connected to CBL_PRES# on the MXIO connectors. How this is accomplished is outside the scope of this specification.

This diagram is meant to provide simple example, and it is not required for vendors to implement it as shown. Additional examples may be provided in the additional resources package. See section 6.1, “Supplementary Materials List”, for additional resources.

10.1.5. UBM Backplane NVMe Drive Hot Plug Requirements

This section outlines the requirements for UBM backplane NVMe drive hot plug support.

The following diagram below provides a high-level block diagram depicting the UBM backplane and signal interconnections between it and other key components supporting NVMe drive hot plug support. Here the HPM primary PD implemented using an FPGA device.

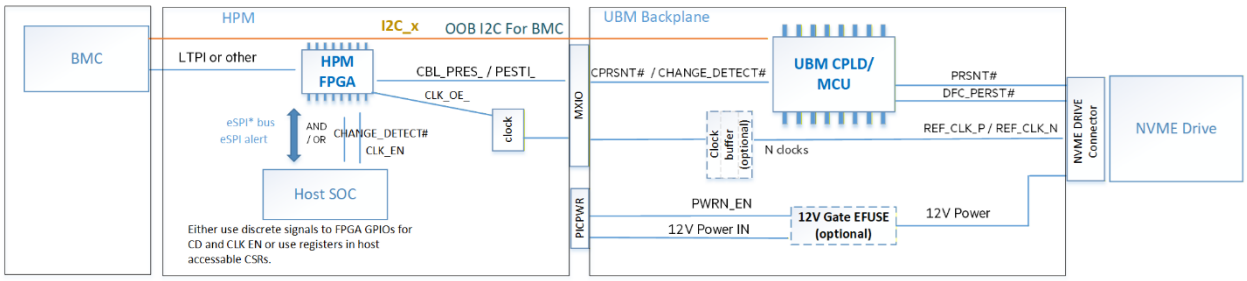


Figure 15: UBM Backplane NMVe Hot Plug Support Diagram

The following sections contain tables that outline the MXIO FlexIO requirements for supporting NVMe hot plug functionality.

Please refer to the specific NVMe drive specifications for further information regarding hot-plug requirements.

10.1.6. UBM Backplane FlexIO Assignments

This section provides the MXIO FlexIO assignments for the UBM backplane required and recommended for NVMe drive hot plug support.

UBM Backplane: Device Class = 0x02 Subclass = 0x01

Table 9: UBM Backplane FlexIO Assignments

FlexIO Assignment	MXIO Pin	Default Assignment	Preferred Assignment	HPM SHALL Support	IF Supported SHALL be on	Power State Valid
CBL_PRES_A_PESTI_A_N	B12	CPRSNT_A_N	FLEXIO_0A	Y	FLEXIO_0A	S5 / S0
CPRSNT_B_N	B29	NC	FLEXIO_1A	N	FLEXIO_1A	S5 / S0
I2C_RESETA_N	B30	NC	FLEXIO_2A	N	FLEXIO_2A	S5 / S0
REFCLK_B_P	B26	NC	FLEXIO_3A	N	FLEXIO_3A	S5 / S0
REFCLK_B_N	B27	NC	FLEXIO_4A	N	FLEXIO_4A	S5 / S0
SMSCL_B	A26	NC	FLEXIO_5A	N	FLEXIO_5A	S5 / S0
SMSDA_B	A27	NC	FLEXIO_6A	N	FLEXIO_6A	S5 / S0

10.1.6.1. UBM Backplane MXIO FlexIO Signals Definition

This section provides detailed descriptions for the UBM Backplane FlexIO MXIO signals.

Table 10: UBM Backplane MXIO FlexIO Signals Definition

Signal	Required	Description
CBL_PRES_A_PESTI_A_N	Y	Cable Presence A / PESTI A. This is required.
CPRSNT_B_N	N	Optional secondary cable presence signal. Only required if HPM / system integrator requires support for splitting the MXIO from x8 to two independent x4 cables.
I2C_RESETA_N	N	Optional independent I2C bus reset signal.
REFCLK_B_P	N	Clock signal differential pair. Only required to support if the clock device is on the HPM and not the backplane. The backplane JSON file will indicate the FLEXIO requirements.
REFCLK_B_N	N	Clock signal differential pair. Only required to support if the clock device is on the HPM and not the backplane. The backplane JSON file will indicate the FLEXIO requirements.
SMSCL_B	N	Optional secondary I2C interface. Only required if HPM / system integrator requires support for splitting the MXIO from x8 to two independent x4 cables.
SMSDA_B	N	Optional secondary I2C interface. Only required if HPM / system integrator requires support for splitting the MXIO from x8 to two independent x4 cables.

10.1.7. UBM Backplane NVMe Drive Hot Plug Behavioral Modes

This section describes two main UBM backplane behavioral modes: Autonomous and Managed.

Autonomous Mode

- Backplane will assert CPRSNT# when a drive is installed.
    - CPRSNT# acts as a clock request, and it is required that the host indicates this to the Primary HPM Programmable Device to enable REFCLK within 100us.
- The backplane then waits 100ms and then de-asserts DFC PERST# to the drive bay.
    - The DFC PERST# signal is between the UBM controller and the drive bay. It is not the same PERST# on the MXIO connector.

Managed Mode

- The Backplane will assert CPRSNT# when a drive is installed.
    - CPRSNT# is used as a change detection, CHANGE_DETECT, signal. It is required that the HOST to query the UBM controller to determine what has changed.

- The HOST will indicate to the HPM Primary Programmable Device to supply REFCLK signals to the backplane. After 100ms it will then send the UBM command to de-assert the appropriate DFC PERST#.

See the SNIA SFF-TA-1005 UBM specification for additional details.

10.1.8. Non-UBM Backplane Type 1 (GPIO Expander on Backplane) NVMe Drive Hot Plug Requirements

This section outlines the requirements for non-UBM, legacy backplane NVMe drive hot plug support with one or more GPIO expanders present on the backplane.

The following diagram below provides a high-level block diagram depicting the non-UBM backplane and signal interconnections between it and other key components supporting NVMe drive hot plug support. Here the HPM primary PD implemented using an FPGA device.

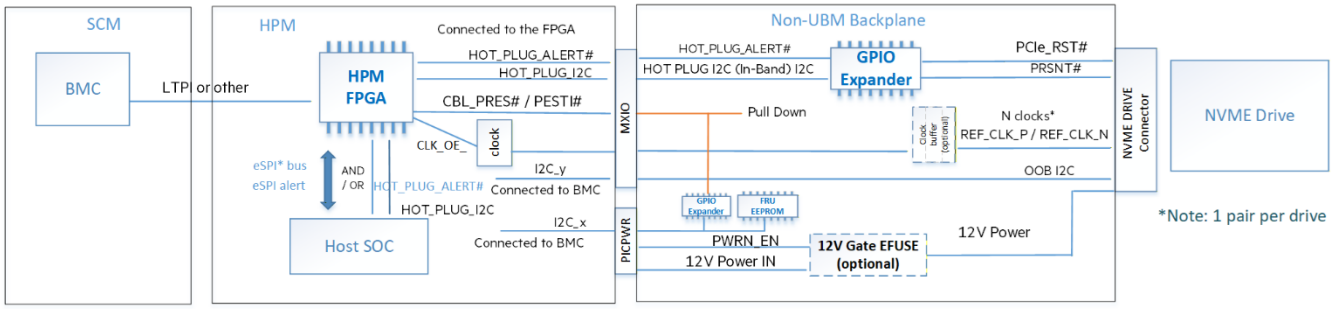


Figure 16: Non-UBM Backplane With IO Expander on Board NMVe Hot Plug Support Diagram

*Note: Could be another low-level access bus to Primary HPM Programmable Device e.g. I2C.

10.1.9. Non-UBM Backplane Type 1 (GPIO Expander on Backplane) FlexIO Assignments

The following table outlines the MXIO FlexIO requirements for supporting NVMe hot plug functionality.

Non-UBM Backplane Type 1: Device Class = 0x02 Subclass = 0x00

Table 11: Non-UBM Backplane MXIO FlexIO Assignments

FlexIO Assignment	MXIO Pin	Default Assignment	Preferred Assignment	HPM SHALL Support	IF Supported SHALL be on	Power State Valid
CBL_PRES_A_PESTI_A_N	B12	CPRSNT_A_N	FLEXIO_0A	Y	FLEXIO_0A	S5 / S0
HOT_PLUG_ALERT_N	B29	NC	FLEXIO_1A	Y*	FLEXIO_1A	S5 / S0
VENDOR DEFINED	B30	NC	FLEXIO_2A	N	FLEXIO_2A	S5 / S0
REFCLK_B_P	B26	NC	FLEXIO_3A	N	FLEXIO_3A	S5 / S0
REFCLK_B_N	B27	NC	FLEXIO_4A	N	FLEXIO_4A	S5 / S0
HOT_PLUG_SCL	A26	NC	FLEXIO_5A	Y*	FLEXIO_5A	S5 / S0
HOT_PLUG_SDA	A27	NC	FLEXIO_6A	Y*	FLEXIO_6A	S5 / S0

*Note: Only if HPM supports non-UBM backplanes.

10.1.9.1. Non-UBM Backplane Type 1 (GPIO Expander on Backplane) MXIO FlexIO Signals Definition

This section provides detailed descriptions for the non-UBM Backplane FlexIO MXIO signals.

Table 12: Non-UBM Backplane MXIO FlexIO Signals Assignment Description

Signal	Required	Description
CBL_PRES_A_PESTI_A_N	Y	Cable Presence A / PESTI A. This is required.
HOT_PLUG_ALERT_N	Y	Required alert signal. This signal is sourced from the GPIO expander and represents an event source.
VENDOR DEFINED	N	Reserved for vendor use.
REFCLK_B_P	N	Clock signal differential pair. Only required to support if the clock device is on the HPM and not the backplane. The backplane JSON file will indicate the FLEXIO requirements.
REFCLK_B_N	N	Clock signal differential pair. Only required to support if the clock device is on the HPM and not the backplane. The backplane JSON file will indicate the FLEXIO requirements.
HOT_PLUG_SCL	Y	Required secondary I2C interface for connection to the backplane GPIO expander.
HOT_PLUG_SDA	Y	Required secondary I2C interface for connection to the backplane GPIO expander.

10.1.10. Non-UBM Backplane Type 2 (GPIO Expander Functionality on HPM) NVMe Drive Hot Plug Requirements

This section outlines the requirements for non-UBM backplane NVMe drive hot plug support with the GPIO expander functionality present on the HPM.

The following diagram below provides a high-level block diagram depicting the non-UBM backplane and signal interconnections between it and other key components supporting NVMe drive hot plug support. Here the HPM primary PD implemented using an FPGA device.

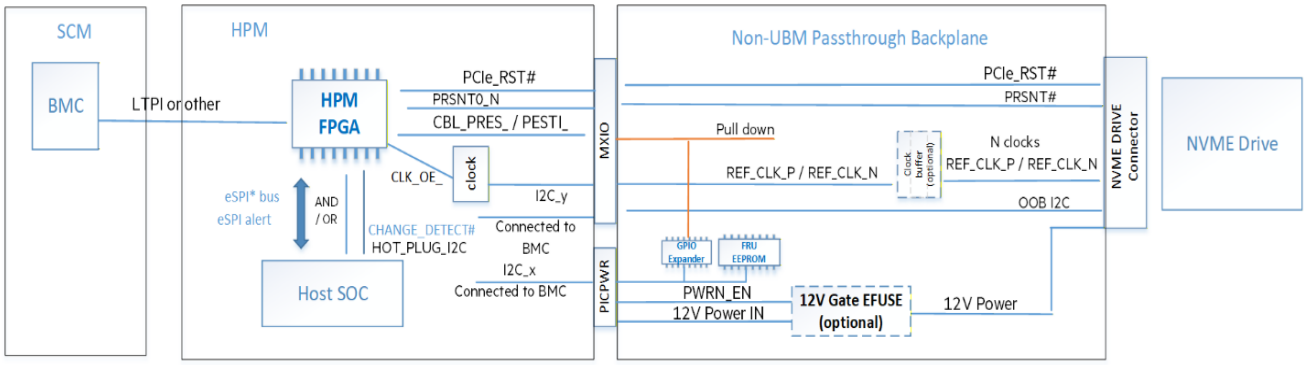


Figure 17: Non-UBM Backplane With IO Expander on HPM NMVe Hot Plug Support Diagram

*Note: Could be another low level access bus to Primary HPM Programmable Device e.g. I2C.

The following table outlines the MXIO FlexIO requirements for supporting NVMe hot plug functionality for non-UBM backplanes that have GPIO expander functionality implemented on the HPM.

10.1.11. Non-UBM Backplane Type 2 (GPIO Expander on HPM) FlexIO Assignments

The following table outlines the MXIO FlexIO requirements for supporting NVMe hot plug functionality for non-UBM type 2 backplanes.

Non-UBM Backplane Type 2 (GPIO Expander On HPM) Device Class = 0x02 Subclass = 0x02

Table 13: Non-UBM Backplane Type 2 (GPIO Expander on HPM) MXIO FLEXIO Assignments

FlexIO Assignment	MXIO Pin	Default Assignment	Preferred Assignment	HPM SHALL Support	IF Supported SHALL be on	Power State Valid
CBL_PRES_A_	B12	CBL_PRES_A_	FLEXIO_0A	Y	FLEXIO_0A	S5 / S0
PRSNT0_N	B29	NC	FLEXIO_1A	Y*	FLEXIO_1A	S5 / S0
PRSNT1_N	B30	NC	FLEXIO_2A	Y*	FLEXIO_2A	S5 / S0
REFCLK_B_P	B26	NC	FLEXIO_3A	N	FLEXIO_3A	S5 / S0
REFCLK_B_N	B27	NC	FLEXIO_4A	N	FLEXIO_4A	S5 / S0
VENDOR DEFINED	A26	NC	FLEXIO_5A	N	FLEXIO_5A	S5 / S0
VENDOR DEFINED	A27	NC	FLEXIO_6A	N	FLEXIO_6A	S5 / S0

*Note: Only if HPM supports non-UBM backplanes.

10.1.11.1. Non-UBM Backplane Type 2 (GPIO Expander on HPM) MXIO FlexIO Signals Definition

This section provides detailed descriptions for the non-UBM Backplane FlexIO MXIO signals with the GPIO expander on the HPM.

Table 14: Non-UBM Backplane Type 2 MXIO FlexIO Signals Definition

Signal	Required	Description
CBL_PRES_A_	Y	Cable Presence A / PESTI A. This is required.
PRSNT0_N	Y	Drive presence 0.
PRSNT1_N	Y	Drive presence 1.
REFCLK_B_P	N	Clock signal differential pair. Only required to support if the clock device is on the HPM and not the backplane. The backplane JSON file will indicate the FLEXIO requirements.
REFCLK_B_N	N	Clock signal differential pair. Only required to support if the clock device is on the HPM and not the backplane. The backplane JSON file will indicate the FLEXIO requirements.
VENDOR DEFINED	Y	Reserved for vendor use.
HOT_PLUG_SDA	Y	Required secondary I2C interface for connection to the backplane GPIO expander.

10.1.12. HPM SoC to Primary HPM Programmable Device Interconnection Options For NVMe Host Plug Support

This section describes different interconnection options for providing GPIO expander emulation functionality.

The following diagram outlines two different mechanisms for GPIO expander emulation by the Primary HPM Programmable Device for the host SoC. Here the HPM primary PD implemented using an FPGA device.

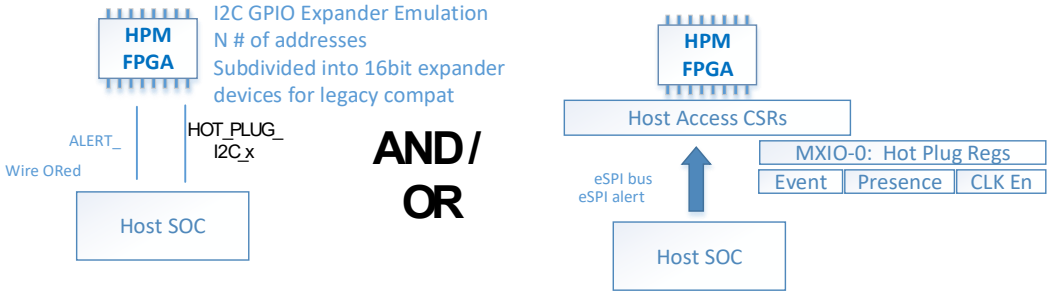


Figure 18: HPM SoC To Primary HPM Programmable Device Interconnection Diagram

The HPM and Primary HPM Programmable Device implementer can provide either or both of these interconnection options. It is required that at least one is implemented. It is required that the PD and HPM JSON file indicates that the interconnection option is present and supports NVMe hot plug functionality.

For specific Primary HPM Programmable Device register definitions, consult the Host Access CSR register definition in the M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification for details.

10.2. Riser Peripheral Requirements and Considerations

This section outlines specific requirements and considerations for system riser components used within M-PNP conformant servers.

10.2.1. Common Riser Discovery and Configuration Capabilities Requirements

This section outlines any common requirements for all riser class peripherals.

Table 15: Riser Peripheral Requirements

Requirement	REQ	OPT	Description
FRU EEPROM / Information Device address at 0xA6	Y		It is required that the riser provides a FRU EEPROM or EEPROM-like information device at address 0xA6.
FRU EEPROM / Information Device connection via PICPWR I2C bus.	Y		It is required that the FRU EEPROM be on a PICPWR I2C bus.
FRU EEPROM / Information Device not behind a MUX	Y		It is required that the riser provides a FRU EEPROM or EEPROM-like information device shall not be behind a MUX.
Self-describing JSON file contents in FRU EEPROM / Information Device	Y		The local self-describing file shall be present in the FRU EEPROM. See section 10.2.2, “Common Riser Peripheral Self-Description Requirements”, for details.
Programmable Logic / MCU JSON File if PLD is populated	Y*		If the riser has programmable logic, then it is required that the riser provides a JSON self-describing information file for all programmable devices contained on the riser. Guidance on how this is accomplished is contained within the M-PNP.HPM and Peripheral Programmable Device specification and schema documentation. The location where this file can be collected shall be provided within the FRU EEPROM JSON file.
Source-to-Destination Detection	Y		It is required that the riser provides Source-to-destination (S2D) routing detection between the PICPWR connector and the MXIO connector(s) CBL_PRES_N signal.
Operational Power State Requirements	Y		It is required that the riser indicate whether it needs to be operational only in S0 or whether it needs to be powered during S5 or Standby power states. This shall be within the self-describing JSON file.
Maximum Power Usage Requirements	Y		It is required that the riser provides a maximum power requirement encoded as an attribute within the self-describing JSON file.
M-PESTI Virtual Wires Definition		Y	The use of M-PESTI virtual wires is optional. If the riser supports M-PESTI virtual wires, then it shall provide a definition of those virtual wires within the riser programmable logic (PLD) JSON file.
M-PESTI MUX / Switch Support		Y	It is not required for peripherals to support M-PESTI MUXes.
Flex IO and Sideband		Y	It is required that all peripherals disclose specific usage of FlexIO and Sideband signaling in the FRU information device JSON file. This is only required

			where a peripheral of a given device class is not using a predefined default assignment for a given FlexIO or PICPWR sideband signal.
SPDM Support		Y	Peripherals may optionally support SPDM for enhanced security. The JSON file shall contain the I2C / I3C address if SPDM is supported using these buses.

*Note: The JSON file shall be provided only if the peripheral contains one or more programmable devices.

10.2.2. Common Riser Peripheral Self-Description Requirements

This section outlines the required and recommended self-description needed to support riser peripheral component interoperability. The column, “R”, refers to required attributes, and the column, “L”, refers to local attributes. The allowable values for these columns are Y for yes, N for no, and C for conditional (see descriptions for explanation as to specific conditions).

Table 16: Riser Peripheral Self-Describing JSON Attributes

Attribute Requirement	R	L	Description	Section Reference
Device Class and Subclass	Y	Y	Classification of peripheral encoded as two attributes: Device Class, and Device Subclass.	10.7
Vendor ID	Y	Y	Required general attributes.	12.2.3
Device	Y	Y	List of all devices present in the peripheral including connections to peripheral buses and connectors.	12.2.5
Buses	Y	C[5] (partial)	Array structure of all buses supported by peripheral if present.	12.2.8
Connectors	Y	C[6] (partial)	Listing of all connectors and bus mappings to connectors supported by the peripheral if present.	12.2.4

[5] Note: These sections of attributes require only a sub-set to be local within the FRU information device.
[6] ibid.

Flex IO Signals Mappings	Y	Y	Present if unique assignments are required.	13.12.4
Sideband Signals Mappings	Y	Y	Present if unique assignments are required.	13.12.5
M-PESTI Virtual Wires Definition	C	Y	It is required that the PLD JSON file provides the M-PESTI virtual wires definition if the peripheral supports the M-PESTI active phase.	13.6
M-PESTI MUX / Switch Support	C	Y	It is not required for peripherals to support M-PESTI MUXes. If supported, it shall be indicated in the JSON file.	13.6
SPDM Support		Y	Peripherals that support SPDM shall indicate which interface supports this security feature.	

It is required that implementors provide the device class and subclass of the peripheral. The peripheral shall provide a JSON file, and it is required to describe all bus interconnections between devices on the peripheral and connectors and device addresses for each bus device. This is needed for the BMC to properly create the bus topology for each bus. This is accomplished using the JSON descriptive attributes within the self-describing JSON file.

10.2.3. Example Riser Block Diagram

The block diagram below contains an example implementation for a common riser.

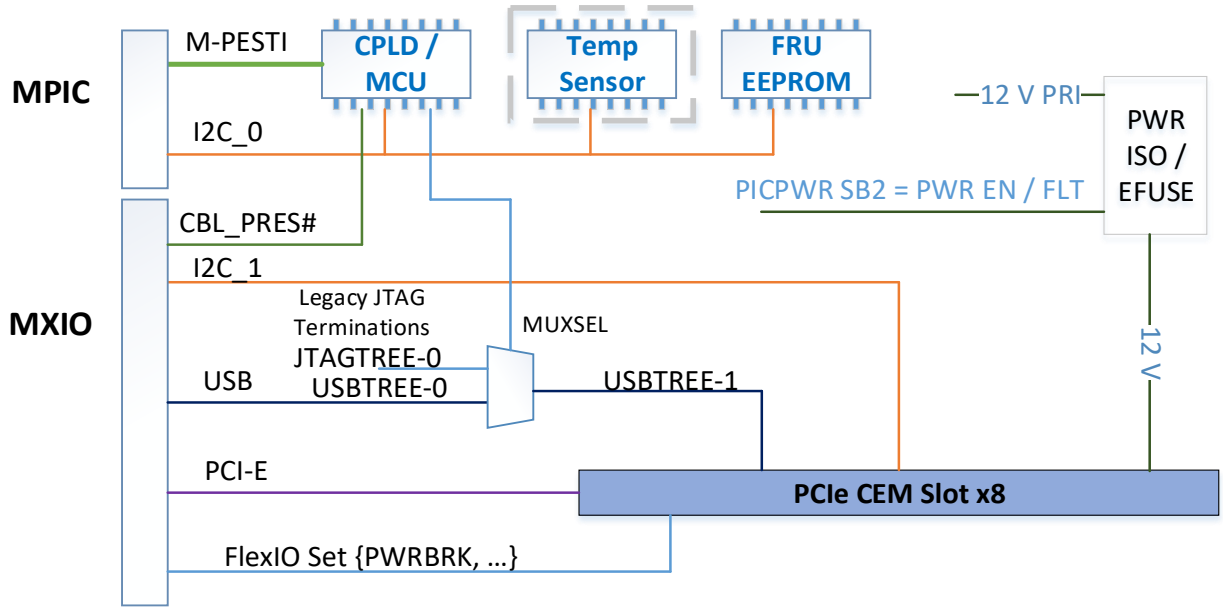


Figure 19: PCIe CEM Slot Riser single x8 slot (MPESTI Version) Example

The example riser block diagram illustrates the basic components for supporting a x8 PCIe CEM slot. The dashed box around the temperature sensing IC indicates that it is an optional component. This example indicates the use programmable logic, such as a CPLD or MCU, however, other embodiments exist that include the use of GPIO expanders or other devices to handle signaling on this peripheral.

This diagram is but one example, and it is not required for vendors to implement it as shown. Vendors may implement the required riser functionality using discrete logic, I2C expanders, etc. Additional examples may be provided in the additional resources package. See section 6.1, “Supplementary Materials List”, for additional resources.

10.3. Power Distribution Board (PDB) Peripheral Requirements and Considerations

This section outlines specific requirements and considerations for system power distribution board (PDB) components used within M-PNP conformant servers.

10.3.1. Common PDB Discovery and Configuration Capabilities Requirements

This section outlines any common requirements for all PDB class peripherals.

Table 17: PDB Requirements

Requirement	REQ	OPT	Description
FRU EEPROM address at 0xA6	Y		It is required that the PDB provides a FRU EEPROM or EEPROM-like information device at address 0xA6.
Type 1 Management Connector: FRU EEPROM connector to HPM to PDB PICPWR I2C bus.	Y[7]		If the PDB implements the type 1 connector, it is required that a PICPWR connector implements the FRU EEPROM. It cannot be on the PMBUS I2C bus.
Type 2 Management Connector: FRU EEPROM connected to I2C_MGMT1 I2C bus.	Y[8]		If the PDB implements the type 2 connector, it is required that the type 2 management connector implements the FRU EEPROM on the I2C_MGMT1 I2C bus. It cannot be on the PMBUS I2C bus.
FRU EEPROM not behind a MUX		Y	It is recommended that the PDB provides a FRU EEPROM or EEPROM-like information device shall not be behind a MUX on the backplane component.
FRU EEPROM on default channel 0 if behind a MUX		Y	It is allowable to have the FRU EEPROM behind a MUX. If it is behind a MUX, then it is required that SGPIO provide the 'MUX reset' GPIO bit to force all MUXes to the original power on default channel. It is required that SGPIO discovery payload indicates support for this feature if implemented. See M-PNP.HPM and Peripheral Programmable Device specification for SGPIO implementation details.
Self-describing JSON file contents in FRU EEPROM	Y		The local self-describing file shall be present in the FRU EEPROM. See section 10.3.2, "Common PDB Peripheral Self-Description Requirements", for details.
Programmable Logic / MCU JSON File	Y*		It is required that the PDB provide a JSON self-describing information file for all programmable devices contained on the riser. Guidance on how this is accomplished is contained within the M-PNP.HPM and Peripheral Programmable Device specification and schema documentation. The location where this file can be collected shall be provided within the FRU EEPROM JSON file.

[7] Note: It is required that the PDB either provide a type 1 or type 2 management connector to the HPM.

[8] Ibid.

M-PESTI Virtual Wires Definition		Y	The type 2 PDB management connector can optionally support M-PESTI as well as PICPWR connections from the HPM to the PDB. If the PDB supports PESTI virtual wires, then it shall provide a definition of those virtual wires within the PDB programmable logic (PLD) JSON file.
M-PESTI Fan-Out and MUX / Switch Support		Y	If the PDB implements M-PESTI connected to a PESTI bus initiated from the HPM, it shall describe whether it has a PESTI MUX / SWITCH in the JSON file. It shall provide the number of channels and the operational mode and method documented within the JSON file. See M-PNP Primary HPM Programmable Device specification for M-PESTI implementation details.
FlexIO and Sideband	Y		It is required that all peripherals disclose specific usage of FlexIO and Sideband signaling in the FRU information device JSON file. This is only required where a peripheral of a given device class is not using a predefined default assignment for a given FlexIO or PICPWR sideband signal.
SPDM Support		Y	Peripherals may optionally support SPDM for enhanced security. The JSON file shall contain the I2C / I3C address if SPDM is supported using these buses.

*Note: The JSON file shall be provided only if the peripheral contains one or more programmable devices.

10.3.2. Common PDB Peripheral Self-Description Requirements

This section outlines the required and recommended self-description needed to support PDB peripheral component interoperability. The column, “R”, refers to required attributes, and the column, “L”, refers to local attributes. The allowable values for these columns are Y for yes, N for no, and C for conditional (see descriptions for explanation as to specific conditions).

Table 18: PDB Self-Describing JSON Attributes

Data	R	L	Description	Section Reference
Vendor ID	Y	Y	Required general attributes.	12.2.3

Device Class and Subclass	Y	Y	Classification of peripheral encoded as two attributes: Device Class, and Device Subclass.	10.7
Device	Y	Y	List of all devices present in the peripheral including connections to peripheral buses and connectors.	12.2.5
Buses	Y	C[9] (partial)	Array structure of all buses supported by peripheral if present.	12.2.8
Connectors	Y	C[10] (partial)	Listing of all connectors and bus mappings to connectors supported by the peripheral if present.	12.2.4
Flex IO Signals Mappings	Y	Y	Present if unique assignments are required.	13.12.4
Sideband Signals Mappings	Y	Y	Present if unique assignments are required.	13.12.5
M-PESTI MUX / Switch Support	C	Y	JSON file shall indicate if a M-PESTI MUX / Switch is present.	13.6
M-PESTI Virtual Wires Definition	C	Y	It is required that the PLD JSON file provides the M-PESTI virtual wires definition if the peripheral supports the M-PESTI active phase.	13.6
SPDM Support		Y	Peripherals that support SPDM shall indicate which interface supports this security feature.	

It is required that implementors provide the device class and subclass of the peripheral. The peripheral shall provide a JSON file, and it required to describe all bus interconnections between devices on the peripheral and connectors and device addresses for each bus device. This is needed for the BMC to properly create the bus topology for each bus. This is accomplished using the JSON descriptive attributes within the self-describing JSON file.

[9] Note: These sections of attributes require only a sub-set to be local within the FRU information device.
[10] ibid.

10.3.3. Example Power Distribution Board (PDB) Block Diagram

The block diagram below contains an example implementation for a generic Power Distribution Board (PDB).

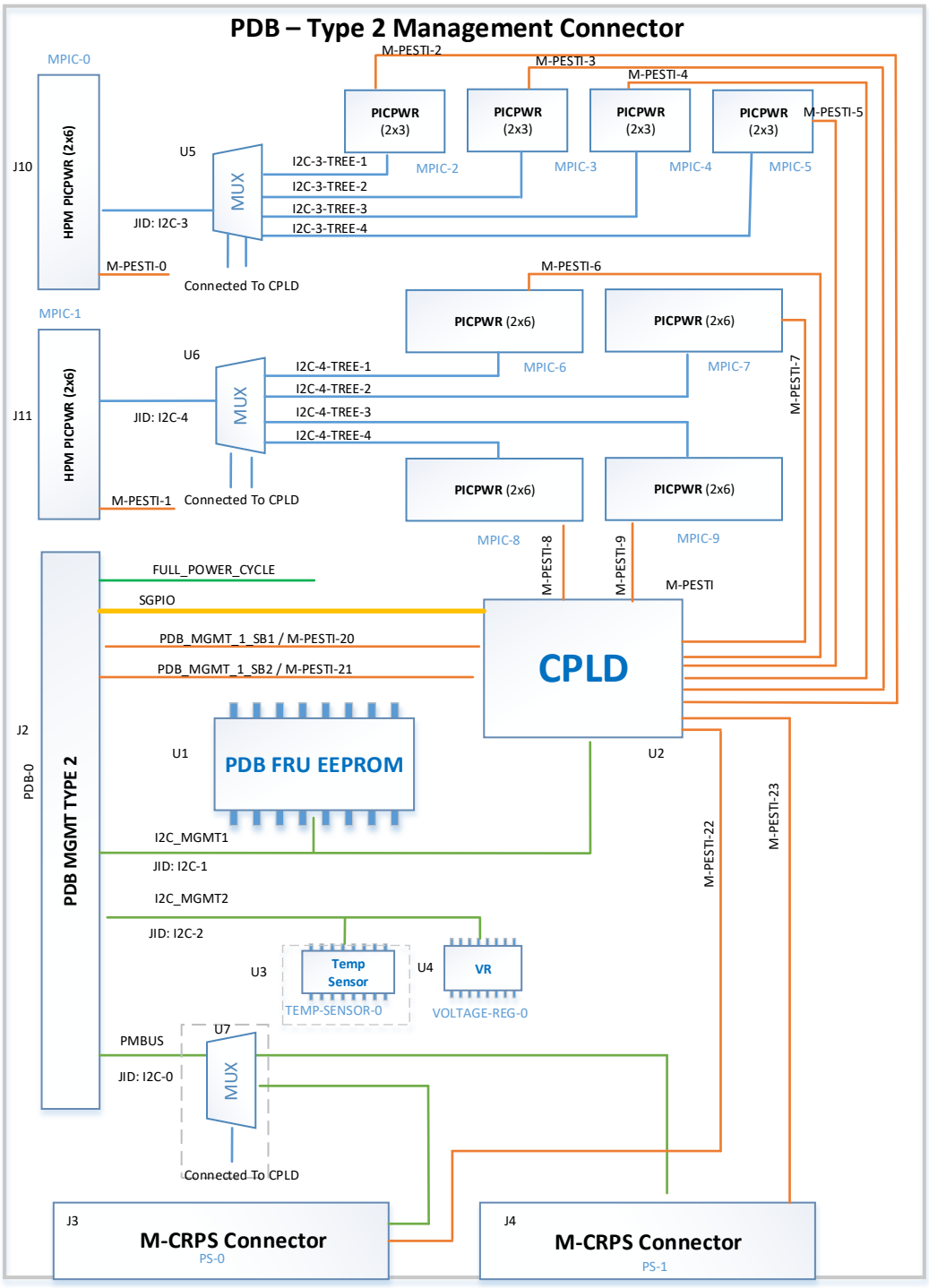


Figure 20: Power Distribution Board (PDB) Block Diagram Example

The example Power Distribution Board (PDB) block diagram illustrates the basic building blocks of a general two M-CRPS slot supporting PDB. This specific example incorporates the use of the M-PIC Type 2 Power Management Board connector. Additionally, this example could represent M-PESTI MUXing usage via the PDB CPLD, or it could represent that the CPLD embodies its own MPESTI IP block with its own M-PESTI initiator instances. The choice is left up to the PDB CPLD implementor. The dashed box around the temperature sensing IC indicates that it is an optional component.

Note: If the PDB is an M-PESTI target, then the PDB shall re-synchronize M-PESTI through SGPIO signaling upon loss of communications with the M-PESTI initiator. Please refer to the M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification for details.

This diagram is meant to provide simple example, and it is not required for vendors to implement it as shown. For instance, the PDB programmable device could implement a M-PESTI MUX to support the necessary M-PESTI targets connector to PICPWR or it could implement a true M-PESTI initiator instead. The specific implementation details are left to the vendor and are out of scope for this specification. Additional examples may be provided in the additional resources package. See section 6.1, “Supplementary Materials List”, for additional resources.

10.4. Control Panel Board Peripheral Requirements and Considerations

This section outlines specific requirements and considerations for system front panel control board components used within M-PNP conformant servers.

10.4.1. Common Control Panel Discovery and Configuration Capabilities Requirements

This section outlines the set of common requirements for all front panel control class peripherals.

Table 19: Control Panel Discovery Requirements

Requirement	REQ	OPT	Description
FRU EEPROM / Information Device address at 0xA6	Y		It is required that the front panel control provides a FRU EEPROM or EEPROM-like information device at address 0xA6.

FRU EEPROM / Information Device Control Panel Connection	Y		It is required that the FRU EEPROM be connected using the M-PIC Control Panel SMBUS/I3C bus.
Control Panel Sideband SB1 Used For Cable Presence or PESTI	Y		It is required that the control panel sideband SB1 assignment is connected to CBL_PRES_N for cable presence and / or MPESTI bus interface.
Control Panel Sideband SB2 Shal Be Assigned to PWR_EN.	Y		It is required that the front control panel sideband PCP_SB2 assignment is connected to active high PWR_EN.
FRU EEPROM / Information Device not behind a MUX	Y		It is required that the control panel FRU EEPROM or EEPROM-like information device shall not be placed behind a MUX.
Self-describing JSON file contents in FRU EEPROM / Information Device	Y		The local self-describing file shall be present in the FRU EEPROM. See section 10.2.2, “Common Control Panel Peripheral Self-Description Requirements”, for details.
Programmable Logic / MCU JSON File if a PLD is populated	Y*		It is required that the front control panel provide a JSON self-describing information file for all programmable devices contained on the peripheral. Guidance on how this is accomplished is contained within the M-PNP.HPM and Peripheral Programmable Device specification and schema documentation. The location where this file can be collected shall be provided within the FRU EEPROM JSON file.
Sideband PCP_SB3 and PCP_SB4] Definitions		Y	If the control panel uses PCP_SB3 and PCP_SB4 sideband signals then it shall provide a definition of those assignments within the front control panel JSON file.
M-PESTI Virtual Wires Definition		Y	If the control panel supports PESTI virtual wires, then it shall provide a definition of those virtual wires within the front control panel programmable logic (PLD) JSON file.
M-PESTI MUX / Switch Support		Y	It is not required for peripherals to support M-PESTI MUXes.
SPDM Support		Y	Peripherals may optionally support SPDM for enhanced security. The JSON file shall contain the I2C / I3C address if SPDM is supported using these buses.

*Note: The JSON file shall be provided only if the peripheral contains one or more programmable devices.

10.4.2. Common Control Panel Peripheral Self-Description Requirements

This section outlines the required and recommended self-description needed to support the front control panel peripheral component interoperability. The column, “R”, refers to required attributes, and the column, “L”, refers to local attributes. The allowable values for these columns are Y for yes, N for no, and C for conditional (see descriptions for explanation as to specific conditions).

Table 20: Control Panel Self-Describing JSON Attributes

Attribute Requirement	R	L	Description	Section Reference
Device Class and Subclass	Y	Y	Classification of peripheral encoded as two attributes: Device Class, and Device Subclass.	10.7
Vendor ID	Y	Y	Required general attributes.	12.2.3
Device	Y	Y	List of all devices present in the peripheral including connections to peripheral buses and connectors.	12.2.5
Buses	Y	C[11] (partial)	Array structure of all buses supported by peripheral if present.	12.2.8
Connectors	Y	C[12] (partial)	Listing of all connectors and bus mappings to connectors supported by the peripheral if present.	12.2.4
Flex IO Signals Mappings	Y	Y	Present if unique assignments are required.	13.12.4
Sideband Signals Mappings	Y	Y	Present if unique assignments are required.	13.12.5
M-PESTI Virtual Wires Definition	C	Y	It is required that the PLD JSON file provide the M-PESTI virtual wires definition if the peripheral supports M-PESI.	13.6

[11] Note: These sections of attributes require only a sub-set to be local within the FRU information device.
[12] ibid.

M-PESTI MUX / Switch Support	C	Y	JSON file shall indicate if a M-PESTI MUX / Switch is present.	13.6
SPDM Support		Y	Peripherals that support SPDM shall indicate which interface supports this security feature.	

It is required that implementors provide the device class and subclass of the peripheral. The peripheral shall provide a JSON file, and it required to describe all bus interconnections between devices on the peripheral and connectors and device addresses for each bus device. This is needed for the BMC to properly create the bus topology for each bus. This is accomplished using the JSON descriptive attributes within the self-describing JSON file.

10.4.3. Example Control Panel Block Diagram

The block diagram below contains an example implementation for a common control panel that supports M-PESTI.

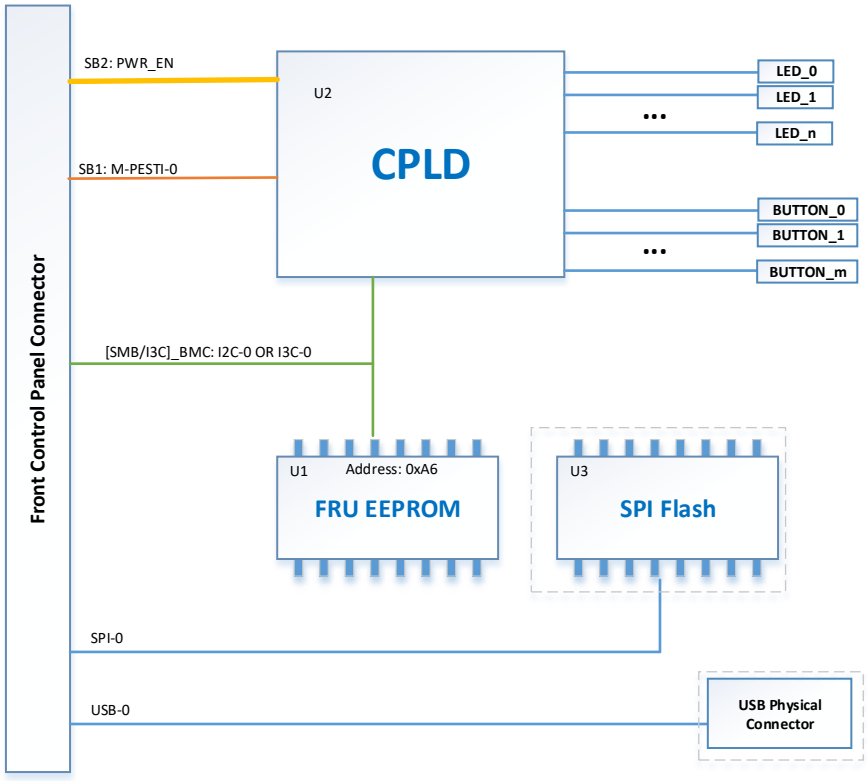


Figure 21: Control Panel Peripheral (MPESTI Supporting) Example

The example control panel block diagram illustrates the basic components for supporting simple human interface set of controls and indicators (e.g. LEDs). The dashed box around the set of LEDs and buttons indicates that it is an optional component. It is up to the implementor decide the type, usage, and number of human interface devices supported by the front control panel peripheral.

This diagram is but as one example, and it is not required for vendors to implement it as shown. Vendors may implement a control panel using an MCU, discrete logic, I2C expanders, etc. Additional examples may be provided in the additional resources package. See section 6.1, “Supplementary Materials List”, for additional resources.

10.4.4. Control Panel M-PESTI Virtual Wires Considerations

This section outlines a set of recommended virtual wires that implementors may use for control panel peripheral component designs to promote interoperability.

Table 21: Example Control Panel M-PESTI Virtual Wire Definitions

Virtual Wire	Direction	Description
Power Button	IN	Manual physical power button state input signal.
Power LED	OUT	Single LED indication for S0 power
Health LED Signal(s)	OUT	One or more signals to indicate state of system health.
Unit Identifier LED	OUT	A unit identifier LED to indicate to floor personnel for service reasons.
Unit Identifier Button	IN	A unit identifier button used by floor personnel for service reasons.

This is a list of examples and is not meant as an all-inclusive list. Additional signals and usage of virtual wires are up to the implementor. See the M-PNP.HPM and Peripheral Programmable Device specification for further details regarding M-PESTI target virtual wire requirements.

See supplementary materials for example JSON mockups with M-PESTI virtual wire definitions.

10.5. PCIe Switch Peripheral Requirements and Considerations

This section outlines specific requirements and considerations for PCIe switch peripherals used within M-PNP conformant servers.

10.5.1. Common PCIe Switch Discovery and Configuration Capabilities Requirements

This section outlines any common requirements for all PCIe switch class peripherals.

Table 22: PCIe Switch Peripheral Requirements

Requirement	REQ	OPT	Description
FRU EEPROM / Information Device address at 0xA6	Y		It is required that the PCIe switch provides a FRU EEPROM or EEPROM-like information device at address 0xA6.

FRU EEPROM / Information Device connection via PICPWR I2C bus.	Y		It is required that the FRU EEPROM be on a PICPWR I2C bus.
FRU EEPROM / Information Device not behind a MUX	Y		It is required that the PCIe switch provides a FRU EEPROM or EEPROM-like information device shall not be behind a MUX.
Self-describing JSON file contents in FRU EEPROM / Information Device	Y		The local self-describing file shall be present in the FRU EEPROM. See section 10.5.2, “Common PCIe switch Peripheral Self-Description Requirements”, for details.
Programmable Logic / MCU JSON File if PLD is populated	Y*		If the PCIe switch has programmable logic, then it is required that the PCIe switch provides a JSON self-describing information file for all programmable devices contained on the PCIe switch. Guidance on how this is accomplished is contained within the M-PNP.HPM and Peripheral Programmable Device specification and schema documentation. The location where this file can be collected shall be provided within the FRU EEPROM JSON file.
Source-to-Destination Detection	Y		It is required that the PCIe switch provides Source-to-destination (S2D) routing detection between the PICPWR connector and the MXIO connector(s) CBL_PRES_N signal.
Operational Power State Requirements	Y		It is required that the PCIe switch indicates whether it needs to be operational only in S0 or whether it needs to be powered during S5 or Standby power states. This shall be within the self-describing JSON file.
Maximum Power Usage Requirements	Y		It is required that the PCIe switch provides a maximum power requirement encoded as an attribute within the self-describing JSON file.
M-PESTI Virtual Wires Definition		Y	The use of M-PESTI virtual wires is optional. If the PCIe switch supports M-PESTI virtual wires, then it shall provide a definition of those virtual wires within the riser programmable logic (PLD) JSON file.
M-PESTI MUX / Switch Support		Y	It is not required for peripherals to support M-PESTI MUXes.
Flex IO and Sideband		Y	It is required that all peripherals disclose specific usage of FlexIO and Sideband signaling in the FRU information device JSON file. This is only required where a peripheral of a given device class is not using a predefined default assignment for a given FlexIO or PICPWR sideband signal.

SPDM Support		Y	Peripherals may optionally support SPDM for enhanced security. The JSON file shall contain the I2C / I3C address if SPDM is supported using these buses.

*Note: The JSON file shall be provided only if the peripheral contains one or more programmable devices.

10.5.2. Common PCIe switch Peripheral Self-Description Requirements

This section outlines the required and recommended self-description needed to support PCIe switch peripheral component interoperability. The column, “R”, refers to required attributes, and the column, “L”, refers to local attributes. The allowable values for these columns are Y for yes, N for no, and C for conditional (see descriptions for explanation as to specific conditions).

Table 23: PCIe Switch Peripheral Self-Describing JSON Attributes

Attribute Requirement	R	L	Description	Section Reference
Device Class and Subclass	Y	Y	Classification of peripheral encoded as two attributes: Device Class, and Device Subclass.	10.7
Vendor ID	Y	Y	Required general attributes.	12.2.3
Device	Y	Y	List of all devices present in the peripheral including connections to peripheral buses and connectors.	12.2.5
Buses	Y	C[13] (partial)	Array structure of all buses supported by peripheral if present.	12.2.8
Connectors	Y	C[14] (partial)	Listing of all connectors and bus mappings to connectors supported by the peripheral if present.	12.2.4

[13] Note: These sections of attributes require only a sub-set to be local within the FRU information device.
[14] ibid.

Flex IO Signals Mappings	Y	Y	Present if unique assignments are required.	13.12.4
Sideband Signals Mappings	Y	Y	Present if unique assignments are required.	13.12.5
M-PESTI Virtual Wires Definition	C	Y	It is required that the PLD JSON file provides the M-PESTI virtual wires definition if the peripheral supports the M-PESTI active phase.	13.6
M-PESTI MUX / Switch Support	C	Y	It is not required for peripherals to support M-PESTI MUXes. If supported, it shall be indicated in the JSON file.	13.6
SPDM Support		Y	Peripherals that support SPDM shall indicate which interface supports this security feature.	

It is required that implementors provide the device class and subclass of the peripheral. The peripheral shall provide a JSON file, and it is required to describe all bus interconnections between devices on the peripheral and connectors and device addresses for each bus device. This is needed for the BMC to properly create the bus topology for each bus. This is accomplished using the JSON descriptive attributes within the self-describing JSON file.

10.5.3. Example PCIe switch Block Diagram

The block diagram below contains an example implementation for a common PCIe switch.

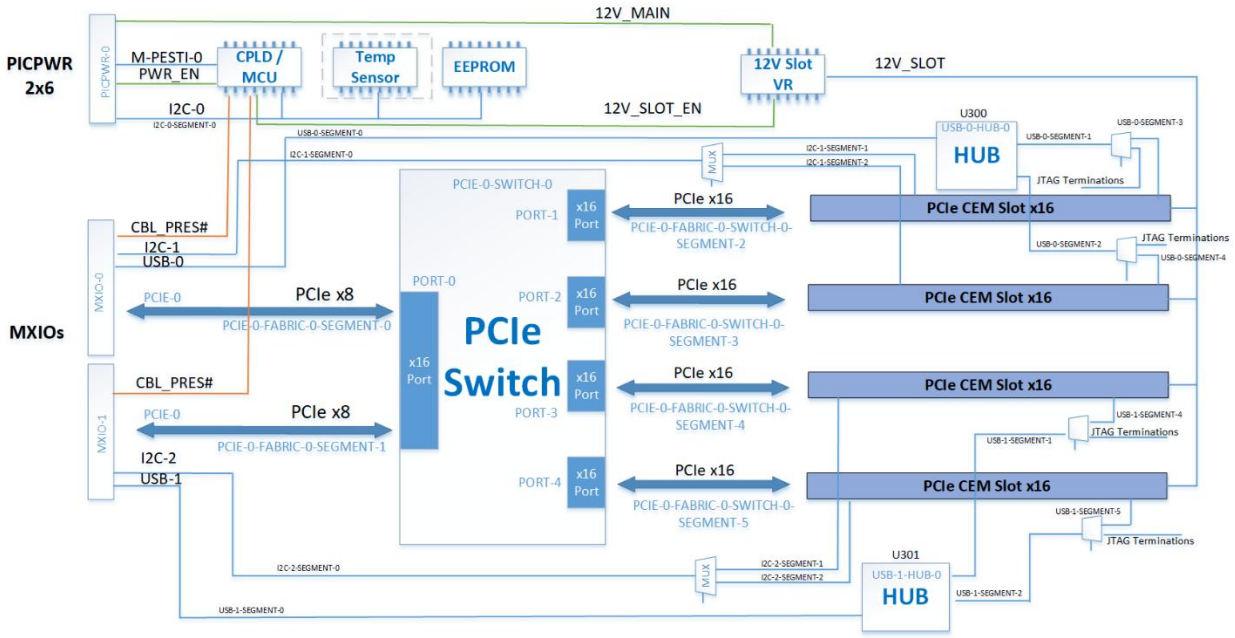


Figure 22: PCIe Switch single x8 slot (MPESTI Version) Example

The example PCIe Switch board block diagram illustrates the basic components for supporting four x16 PCIe CEM slots. The dashed box around the temperature sensing IC indicates that it is an optional component. This example indicates the use programmable logic, such as a CPLD or MCU, however, other embodiments exist that include the use of GPIO expanders or other devices to handle signaling on this peripheral.

This diagram is meant as a simple example, and it is not required for vendors to implement it as shown. Additional examples may be provided in the additional resources package. See section 6.1, “Supplementary Materials List”, for additional resources.

10.6. Remote Cooling Subsystem Peripheral Requirements and Considerations

This section outlines specific requirements and considerations for remote cooling subsystem peripherals used within M-PNP conformant servers. In this specification, a cooling subsystem is meant as an overarching term and is not specific to any one particular cooling technology. This is inclusive of tradition air movers (e.g. fans), direct liquid cooling (DLC), closed loop liquid cooling, and any other cooling technologies.

10.6.1. Common Remote Cooling Subsystem Discovery and Configuration Capabilities Requirements

This section outlines any common requirements for all remote cooling subsystem class peripherals.

Table 24: Remote Cooling Subsystem Peripheral Requirements

Requirement	REQ	OPT	Description
FRU EEPROM / Information Device address at 0xA6	Y		It is required that the remote cooling board provides a FRU EEPROM or EEPROM-like information device at address 0xA6.
FRU EEPROM / Information Device connection via PICPWR I2C bus.	Y		It is required that the FRU EEPROM be on a PICPWR I2C bus.
FRU EEPROM / Information Device not behind a MUX	Y		It is required that the remote cooling board provides a FRU EEPROM or EEPROM-like information device shall not be behind a MUX.
Self-describing JSON file contents in FRU EEPROM / Information Device	Y		The local self-describing file shall be present in the FRU EEPROM. See section 10.5.2, “Common Remote Cooling Peripheral Self-Description Requirements”, for details.
Programmable Logic / MCU JSON File if PLD is populated	Y*		If the remote cooling board has programmable logic, then it is required that the PCIe switch provides a JSON self-describing information file for all programmable devices contained on the remote cooling board. Guidance on how this is accomplished is contained within the M-PNP.HPM and Peripheral Programmable Device specification and schema documentation. The location where this file can be collected shall be provided within the FRU EEPROM JSON file.
Operational Power State Requirements	Y		It is required that the remote cooling board indicate whether it needs to be operational only in S0 or whether it needs to be powered during S5 or Standby power states. This shall be within the self-describing JSON file.

Maximum Power Usage Requirements	Y		It is required that the remote cooling board provides a maximum power requirement encoded as an attribute within the self-describing JSON file.
M-PESTI Virtual Wires Definition		Y	The use of M-PESTI virtual wires is optional. If the remote cooling board supports M-PESTI virtual wires, then it shall provide a definition of those virtual wires within the riser programmable logic (PLD) JSON file.
M-PESTI MUX / Switch Support		Y	It is not required for peripherals to support M-PESTI MUXes.
PICPWR Sideband		Y	It is required that all peripherals disclose specific usage of Sideband signaling in the FRU information device JSON file. This is only required where a peripheral of a given device class is not using a predefined default assignment for a given PICPWR sideband signal.
SPDM Support		Y	Peripherals may optionally support SPDM for enhanced security. The JSON file shall contain the I2C / I3C address if SPDM is supported using these buses.

*Note: The JSON file shall be provided only if the peripheral contains one or more programmable devices.

10.6.2. Common Remote Cooling Subsystem Peripheral Self-Description Requirements

This section outlines the required and recommended self-description needed to support remote cooling subsystem peripheral component interoperability. The column, “R”, refers to required attributes, and the column, “L”, refers to local attributes. The allowable values for these columns are Y for yes, N for no, and C for conditional (see descriptions for explanation as to specific conditions).

Table 25: Remote Cooling Peripheral Self-Describing JSON Attributes

Attribute Requirement	R	L	Description	Section Reference
Device Class and Subclass	Y	Y	Classification of peripheral encoded as two attributes: Device Class, and Device Subclass.	10.7

Vendor ID	Y	Y	Required general attributes.	12.2.3
Device	Y	Y	List of all devices present in the peripheral including connections to peripheral buses and connectors.	12.2.5
Buses	Y	C[15] (partial)	Array structure of all buses supported by peripheral if present.	12.2.8
Connectors	Y	C[16] (partial)	Listing of all connectors and bus mappings to connectors supported by the peripheral if present.	12.2.4
Sideband Signals Mappings	Y	Y	Present if unique assignments are required.	13.12.5
M-PESTI Virtual Wires Definition	C	Y	It is required that the PLD JSON file provides the M-PESTI virtual wires definition if the peripheral supports the M-PESTI active phase.	13.6
M-PESTI MUX / Switch Support	C	Y	It is not required for peripherals to support M-PESTI MUXes. If supported, it shall be indicated in the JSON file.	13.6
SPDM Support		Y	Peripherals that support SPDM shall indicate which interface supports this security feature.	

It is required that implementors provide the device class and subclass of the peripheral. The peripheral shall provide a JSON file, and it is required to describe all bus interconnections between devices on the peripheral and connectors and device addresses for each bus device. This is needed for the BMC to properly create the bus topology for each bus. This is accomplished using the JSON descriptive attributes within the self-describing JSON file.

[15] Note: These sections of attributes require only a sub-set to be local within the FRU information device.
[16] ibid.

10.6.3. Example Remote Cooling Subsystem Peripheral Block Diagram

The block diagram below contains an example implementation for a common remote cooling subsystem board.

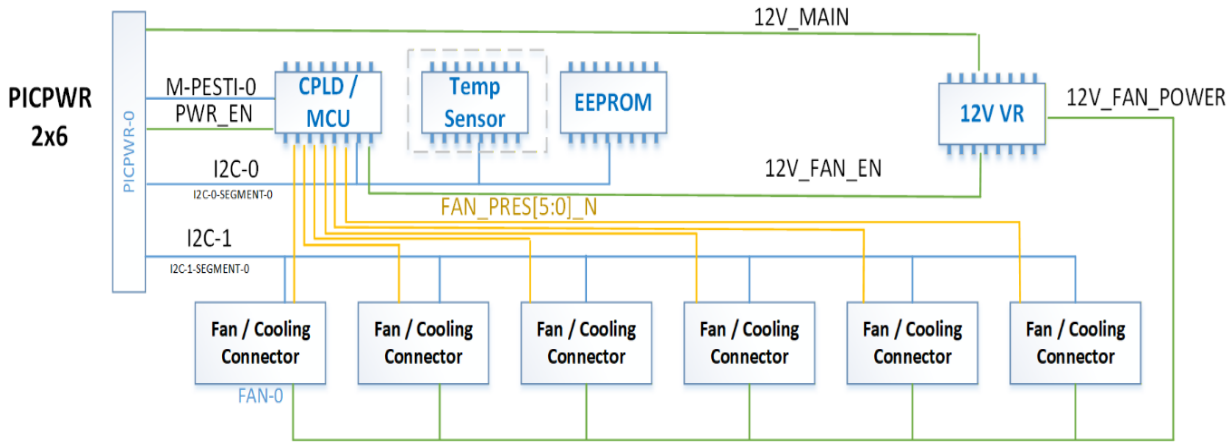


Figure 23: Remote Cooling Subsystem board featuring smart I2C Fan / Cooling Support Example

The example Remote Cooling Subsystem board block diagram illustrates the basic components for supporting air cooling. The dashed box around the temperature sensing IC indicates that it is an optional component. For implementations that support Direct Liquid Cooling (DLC) or other cooling technologies may differ considerably from this example.

This diagram is meant as an example, and it is not required for vendors to implement it as shown. For instance, it is possible that different implementations could have specific fan controller ASIC or programmable logic that the system BMC could interface with. Additionally, a vendor may implement an M-PESTI MUX to provide M-PESTI bus fan-out as described in the M-PESTI specification. Additional examples may be provided in the additional resources package. See section 6.1, “Supplementary Materials List”, for additional resources.

10.7. PCIe Re-timer CEM Passthrough Peripheral Requirements and Considerations

This section outlines specific requirements and considerations for a CEM pass-through PCIe re-timer peripherals used within M-PNP conformant servers.

10.7.1. Common PCIe Re-timer CEM Passthrough Peripheral Discovery and Configuration Capabilities Requirements

This section outlines any common requirements for all PCIe re-timer CEM passthrough device class peripherals.

Table 26: PCIe Re-timer CEM passthrough Peripheral Requirements

Requirement	REQ	OPT	Description
FRU EEPROM / Information Device address at 0xA0	Y		It is required that the PCIe re-timer CEM passthrough peripheral provides a FRU EEPROM or EEPROM-like information device at address 0xA0.
FRU EEPROM / Information Device connection via PICPWR I2C bus.	Y		It is required that the FRU EEPROM be on a PICPWR I2C bus.
FRU EEPROM / Information Device behind a programmable MUX	Y		It is required that the PCIe re-timer CEM passthrough peripheral provides a FRU EEPROM or EEPROM-like information device shall be behind a programmable MUX.
Self-describing JSON file contents in FRU EEPROM / Information Device	Y		The local self-describing file shall be present in the FRU EEPROM. See section 10.5.2, “Common PCIe re-timer CEM passthrough Peripheral Self-Description Requirements”, for details.
Programmable Logic / MCU JSON File if PLD is populated	Y*		If the PCIe re-timer has programmable logic, then it is required that the PCIe re-timer CEM passthrough peripheral provides a JSON self-describing information file for all programmable devices contained on the PCIe re-timer CEM passthrough peripheral. Guidance on how this is accomplished is contained within the M-PNP.HPM and Peripheral Programmable Device specification and schema documentation. The location where this file can be collected shall be provided within the FRU EEPROM JSON file.
Source-to-Destination Detection	Y		It is required that the PCIe re-timer CEM passthrough peripheral provides Source-to-destination (S2D) routing detection between the PICPWR connector and the MXIO connector(s) CBL_PRES_N signal.

Operational Power State Requirements	Y		It is required that the PCIe re-timer CEM passthrough peripheral indicate whether it needs to be operational only in S0 or whether it needs to be powered during S5 or Standby power states. This shall be within the self-describing JSON file.
Maximum Power Usage Requirements	Y		It is required that the PCIe re-timer CEM passthrough peripheral provides a maximum power requirement encoded as an attribute within the self-describing JSON file.
M-PESTI Virtual Wires Definition		Y	The use of M-PESTI virtual wires is optional. If the PCIe switch supports M-PESTI virtual wires, then it shall provide a definition of those virtual wires within the riser programmable logic (PLD) JSON file.
M-PESTI MUX / Switch Support		Y	It is not required for peripherals to support M-PESTI MUXes.
Flex IO and Sideband		Y	It is required that all peripherals disclose specific usage of FlexIO and Sideband signaling in the FRU information device JSON file. This is only required where a peripheral of a given device class is not using a predefined default assignment for a given FlexIO or PICPWR sideband signal.
SPDM Support		Y	Peripherals may optionally support SPDM for enhanced security. The JSON file shall contain the I2C / I3C address if SPDM is supported using these buses.

*Note: The JSON file shall be provided only if the peripheral contains one or more programmable devices.

10.7.2. Common PCIe re-timer CEM passthrough Peripheral Self-Description Requirements

This section outlines the required and recommended self-description needed to support PCIe re-timer CEM passthrough peripheral component interoperability. The column, “R”, refers to required attributes, and the column, “L”, refers to local attributes. The allowable values for these columns are Y for yes, N for no, and C for conditional (see descriptions for explanation as to specific conditions).

Table 27: PCIe Re-timer CEM passthrough Peripheral Self-Describing JSON Attributes

Attribute Requirement	R	L	Description	Section Reference
Device Class and Subclass	Y	Y	Classification of peripheral encoded as two attributes: Device Class, and Device Subclass.	10.7
Vendor ID	Y	Y	Required general attributes.	12.2.3
Device	Y	Y	List of all devices present in the peripheral including connections to peripheral buses and connectors.	12.2.5
Buses	Y	C[17] (partial)	Array structure of all buses supported by peripheral if present.	12.2.8
Connectors	Y	C[18] (partial)	Listing of all connectors and bus mappings to connectors supported by the peripheral if present.	12.2.4
Flex IO Signals Mappings	Y	Y	Present if unique assignments are required.	13.12.4
Sideband Signals Mappings	Y	Y	Present if unique assignments are required.	13.12.5
M-PESTI Virtual Wires Definition	C	Y	It is required that the PLD JSON file provides the M-PESTI virtual wires definition if the peripheral supports the M-PESTI active phase.	13.6
M-PESTI MUX / Switch Support	C	Y	It is not required for peripherals to support M-PESTI MUXes. If supported, it shall be indicated in the JSON file.	13.6
SPDM Support		Y	Peripherals that support SPDM shall indicate which interface supports this security feature.	

[17] Note: These sections of attributes require only a sub-set to be local within the FRU information device.
[18] ibid.

It is required that implementors provide the device class and subclass of the peripheral. The peripheral shall provide a JSON file, and it required to describe all bus interconnections between devices on the peripheral and connectors and device addresses for each bus device. This is needed for the BMC to properly create the bus topology for each bus. This is accomplished using the JSON descriptive attributes within the self-describing JSON file.

10.7.3. Example PCIe Re-timer CEM passthrough peripheral Block Diagram

The block diagram below contains an example implementation for a common PCIe re-timer CEM passthrough peripheral.

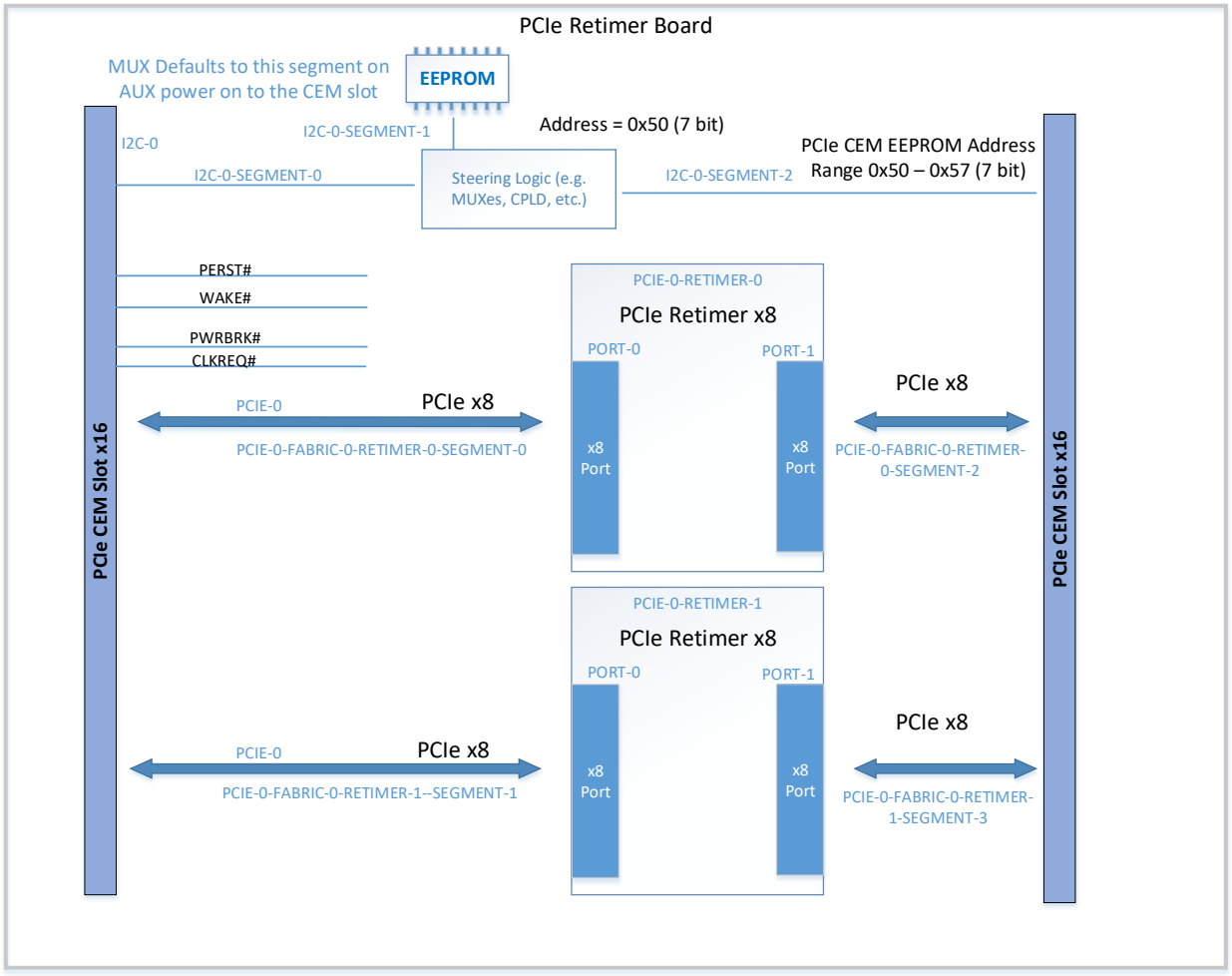


Figure 24: PCIe Retimer CEM passthrough peripheral with x16 slot (MPESTI Version) Example

The example PCIe Retimer CEM passthrough peripheral board block diagram illustrates the basic components for supporting a single x16 PCIe CEM slot. The dashed box around the temperature sensing IC indicates that it is an optional component.

This diagram is meant to provide simple example, and it is not required for vendors to implement it as shown. Additional examples may be provided in the additional resources package. See section 6.1, “Supplementary Materials List”, for additional resources.

10.8. OCP NIC Interposer Peripheral Requirements and Considerations

This section outlines specific requirements and considerations for an OCP NIC interposer 4peripherals used within M-PNP conformant servers.

10.8.1. Common OCP NIC Interposer Peripheral Discovery and Configuration Capabilities Requirements

This section outlines any common requirements for all OCP NIC interposer device class peripherals.

Table 28: OCP NIC Interposer Peripheral Requirements

Requirement	REQ	OPT	Description
FRU EEPROM / Information Device address at 0xA0	Y		It is required that the OCP NIC Interposer peripheral provides a FRU EEPROM or EEPROM-like information device at address 0xA0.
FRU EEPROM / Information Device connection via PICPWR I2C bus.	Y		It is required that the FRU EEPROM be on a PICPWR I2C bus.
FRU EEPROM / Information Device behind a programmable MUX	Y		It is required that the OCP NIC interposer peripheral provides a FRU EEPROM or EEPROM-like information device shall be behind a programmable MUX.
Self-describing JSON file contents in FRU EEPROM / Information Device	Y*		The local self-describing file shall be present in the FRU EEPROM. See section 10.5.2, “Common OCP NIC interposer Peripheral Self-Description Requirements”, for details.
Programmable Logic / MCU JSON File if PLD is populated	Y		If the OCP NIC interposer has programmable logic, then it is required that the OCP NIC interposer peripheral provides a JSON self-describing information file for all programmable devices contained on the OCP NIC interposer. Guidance on how this is accomplished is contained within the M-

			PNP.HPM and Peripheral Programmable Device specification and schema documentation. The location where this file can be collected shall be provided within the FRU EEPROM JSON file.
Source-to-Destination Detection	Y		It is required that the OCP NIC interposer peripheral provides Source-to-destination (S2D) routing detection between the PICPWR connector and the MXIO connector(s) CBL_PRES_N signal.
Operational Power State Requirements	Y		It is required that the OCP NIC interposer peripheral indicate whether it needs to be operational only in S0 or whether it needs to be powered during S5 or Standby power states. This shall be within the self-describing JSON file.
Maximum Power Usage Requirements	Y		It is required that the peripheral provides a maximum power requirement encoded as an attribute within the self-describing JSON file.
M-PESTI Virtual Wires Definition		Y	The use of M-PESTI virtual wires is optional. If the OCP NIC interposer supports M-PESTI virtual wires, then it shall provide a definition of those virtual wires within the riser programmable logic (PLD) JSON file.
M-PESTI MUX / Switch Support		Y	It is not required for peripherals to support M-PESTI MUXes.
Flex IO and Sideband		Y	It is required that all peripherals disclose specific usage of FlexIO and Sideband signaling in the FRU information device JSON file. This is only required where a peripheral of a given device class is not using a predefined default assignment for a given FlexIO or PICPWR sideband signal.
SPDM Support		Y	Peripherals may optionally support SPDM for enhanced security. The JSON file shall contain the I2C / I3C address if SPDM is supported using these buses.

*Note: The JSON file shall be provided only if the peripheral contains one or more programmable devices.

10.8.2. Common OCP NIC interposer Peripheral Self-Description Requirements

This section outlines the required and recommended self-description needed to support the OCP NIC interposer through peripheral component interoperability. The column, “R”, refers to required attributes, and the column, “L”, refers to local attributes. The allowable values for these

columns are Y for yes, N for no, and C for conditional (see descriptions for explanation as to specific conditions).

Table 29: OCP NIC interposer Peripheral Self-Describing JSON Attributes

Attribute Requirement	R	L	Description	Section Reference
Device Class and Subclass	Y	Y	Classification of peripheral encoded as two attributes: Device Class, and Device Subclass.	10.7
Vendor ID	Y	Y	Required general attributes.	12.2.3
Device	Y	Y	List of all devices present in the peripheral including connections to peripheral buses and connectors.	12.2.5
Buses	Y	C[19] (partial)	Array structure of all buses supported by peripheral if present.	12.2.8
Connectors	Y	C[20] (partial)	Listing of all connectors and bus mappings to connectors supported by the peripheral if present.	12.2.4
Flex IO Signals Mappings	Y	Y	Present if unique assignments are required.	13.12.4
Sideband Signals Mappings	Y	Y	Present if unique assignments are required.	13.12.5
M-PESTI Virtual Wires Definition	C	Y	It is required that the PLD JSON file provides the M-PESTI virtual wires definition if the peripheral supports the M-PESTI active phase.	13.6
M-PESTI MUX / Switch Support	C	Y	It is not required for peripherals to support M-PESTI MUXes. If supported, it shall be indicated in the JSON file.	13.6

[19] Note: These sections of attributes require only a sub-set to be local within the FRU information device.
[20] ibid.

SPDM Support		Y	Peripherals that support SPDM shall indicate which interface supports this security feature.	

It is required that implementors provide the device class and subclass of the peripheral. The peripheral shall provide a JSON file, and it required to describe all bus interconnections between devices on the peripheral and connectors and device addresses for each bus device. This is needed for the BMC to properly create the bus topology for each bus. This is accomplished using the JSON descriptive attributes within the self-describing JSON file.

10.8.3. Example OCP NIC interposer peripheral Block Diagram

The block diagram below contains an example implementation for a common OCP NIC interposer peripheral.

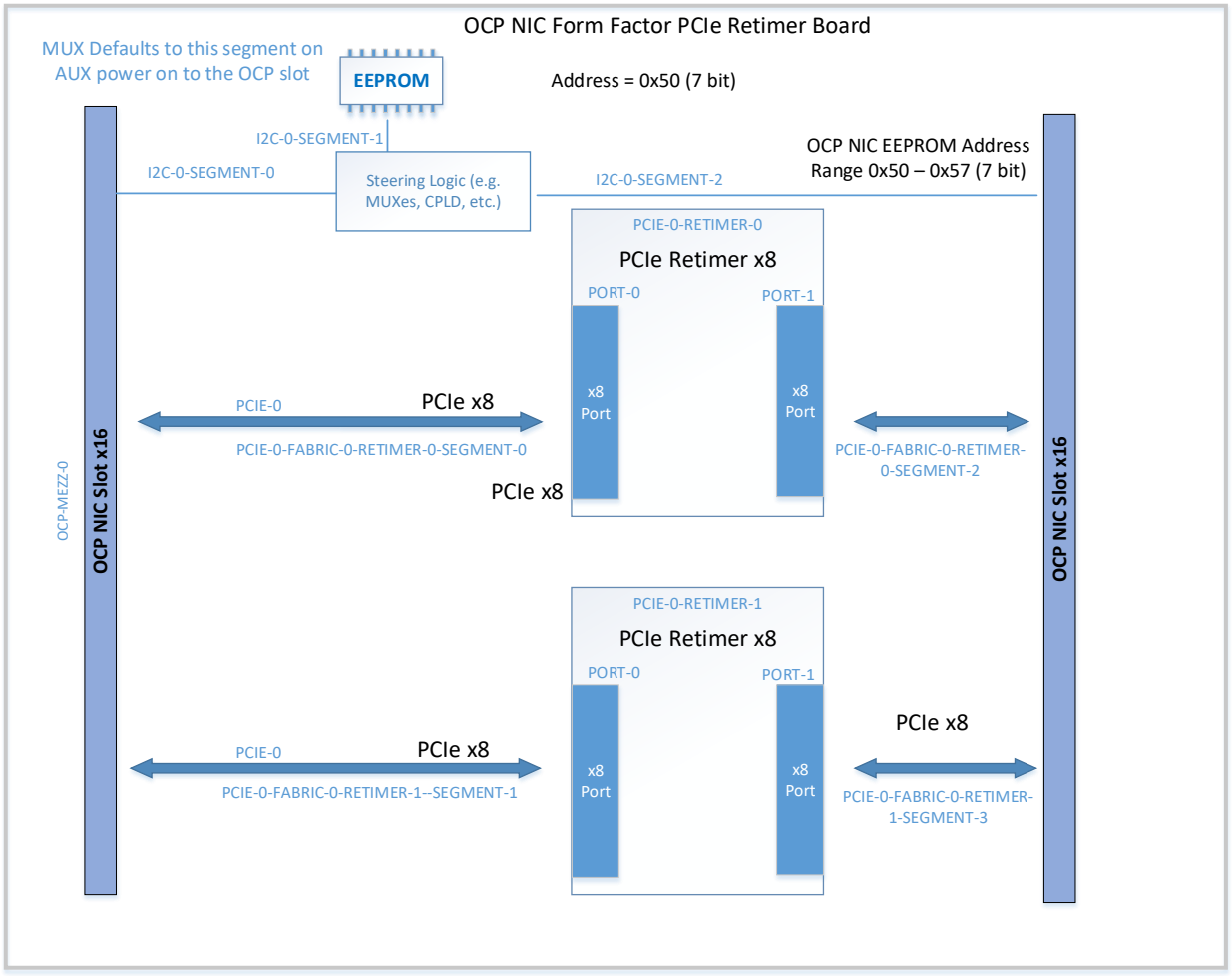


Figure 25: OCP NIC interposer peripheral with x16 slot (MPESTI Version) Example

The example OCP NIC interposer peripheral board block diagram illustrates the basic components for supporting a single x16 OCP NIC slot. The dashed box around the temperature sensing IC indicates that it is an optional component.

The above diagram is meant as a simple example and it is not required for vendors to implement this as shown. Other implementations could substitute MXIO and PICPWR connectors to provide floating riser-based connectivity to an additional OCP NIC connector.

Additional examples may be provided in the additional resources package. See section 6.1, “Supplementary Materials List”, for additional resources.

10.9. General Peripheral Requirements and Considerations

This section outlines overall general requirements and considerations for non-specific components used within M-PNP conformant servers.

Non-endpoint peripheral devices, including OEM / vendor defined non-endpoint peripherals that use a vendor defined class and / or subclass, shall adhere to the general peripheral requirements that are outlined in this section. Certain guidelines do not apply to nested peripheral use cases such as the PCIe re-timer and OCP NIC interposer peripherals. See those sections for details.

10.9.1. Common Discovery and Configuration Capabilities Requirements

This section outlines any common requirements for all peripherals unless otherwise specified for a particular device class.

Table 30: Common Peripheral Discovery Requirements

Requirement	REQ	OPT	Description
FRU EEPROM address at 0xA6	Y		It is required that the peripheral provides a FRU EEPROM or EEPROM-like information device at address 0xA6.
FRU EEPROM connection via PICPWR I2C bus.	Y		It is required that the FRU EEPROM be on a PICPWR I2C bus.
FRU EEPROM not behind a MUX	Y		It is required that the peripheral provides a FRU EEPROM or EEPROM-like information device and it shall not be behind a MUX.
Self-describing JSON file contents in FRU EEPROM	Y*		The local self-describing file shall be present in the FRU EEPROM. See section 10.5.2, “Common Peripheral Self-Description Requirements”, for details.
Programmable Logic / MCU JSON File	Y		It is required that the peripheral provides a JSON self-describing information file for all programmable devices contained on the riser. Guidance on how this is accomplished is contained within the M-PNP.HPM and Peripheral Programmable Device specification and schema documentation. The location where this file can be collected shall be provided within the FRU EEPROM JSON file.

Operational Power State Requirements	Y		It is required that the peripheral indicates whether it needs to be operational only in S0 or whether it needs to be powered during S5 or Standby power states. This shall be within the self-describing JSON file.
Maximum Power Requirements	Y		It is required that the peripheral provides a maximum power requirement encoded as an attribute within the self-describing JSON file.
M-PESTI Virtual Wires Definition		Y	If the peripheral supports PESTI virtual wires, then it shall provide a definition of those virtual wires within the riser programmable logic (PLD) JSON file.
M-PESTI MUX / Switch Support		Y	It is recommended but not required for peripherals to support M-PESTI MUXes.
FlexIO and Sideband	Y		It is required that all peripherals disclose specific usage of FlexIO and Sideband signaling in the FRU information device JSON file. This is only required where a peripheral of a given device class is not using a predefined default assignment for a given FlexIO or PICPWR sideband signal.
SPDM Support		Y	Peripherals may optionally support SPDM for enhanced security. The JSON file shall contain the I2C / I3C address if SPDM is supported using these buses.

*Note: The JSON file shall be provided only if the peripheral contains one or more programmable devices.

10.9.2. Common Peripheral Self-Description Requirements

This section outlines the required self-description needed to support common peripheral component interoperability.

The following table provides the required and optional self-describing data set needed to promote M-PNP peripheral interoperability. The column, “R”, refers to required attributes, and the column, “L”, refers to local attributes. The allowable values for these columns are Y for yes, N for no, and C for conditional (see descriptions for explanation as to specific conditions).

Table 31: Common Peripheral Self-Describing JSON Attributes

Attribute Requirement	R	L	Description	Section Reference

Device Class and Subclass	Y	Y	Classification of peripheral encoded as two attributes: Device Class, and Device Subclass.	10.7
Vendor ID	Y	Y	Required general attributes.	12.2.3
Device	Y	Y	List of all devices present in the peripheral including connections to peripheral buses and connectors.	12.2.5
Buses	Y	C[21] (partial)	Array structure of all buses supported by peripheral if present.	12.2.8
Connectors	Y	C[22] (partial)	Listing of all connectors and bus mappings to connectors supported by the peripheral if present.	12.2.4
Flex IO Signals Mappings	Y	Y	Present if unique, non-default assignments are required. These are captured in the MXIO connectors section.	13.12.4
Sideband Signals Mappings	Y	Y	Present if unique, non-default assignments are required. These are captured in the PICPWR connectors section.	13.12.5
M-PESTI Virtual Wires Definition	C	Y	It is required that the PLD JSON file provides the M-PESTI virtual wires definition if the peripheral supports the M-PESTI active phase.	13.6
M-PESTI MUX / Switch Support	C	Y	JSON file shall indicate if a M-PESTI MUX / Switch is present.	13.6
SPDM Support		Y	Peripherals that support SPDM shall indicate which interface supports this security feature.	

[21] Note: These sections of attributes require only a sub-set to be local within the FRU information device.

[22] ibid.

It is required that implementors provide the device class and subclass of the peripheral. The peripheral shall provide a JSON file, and it is required to describe all bus interconnections between devices on the peripheral and connectors and device addresses for each bus device. This is needed for the BMC to properly create the bus topology for each bus. This is accomplished using the JSON descriptive attributes within the self-describing JSON file.

10.10. Discovery Phase 2: Non-endpoint Peripherals Discovery

This section outlines the second discovery phase targeted towards discovering non-endpoint peripherals. The diagram below outlines the location in the discovery process flow for this specific phase.

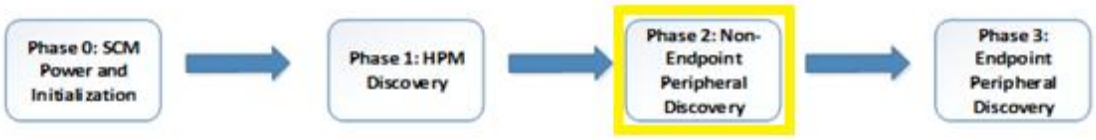


Figure 26: Discovery Phases With Phase 2 Highlighted In Yellow

In Phase 2, the zone of discovery extends out to devices in the system that are not leaf nodes in the overall system topology. Most non-endpoint peripherals are typically backplanes, risers and PDBs, but other intermediary devices could arise in the future. The following block diagram illustrates the interconnections used in the discovery process, and this specific example depicts the interconnection of backplanes and risers to the HPM. Here the HPM primary PD implemented using an FPGA device.

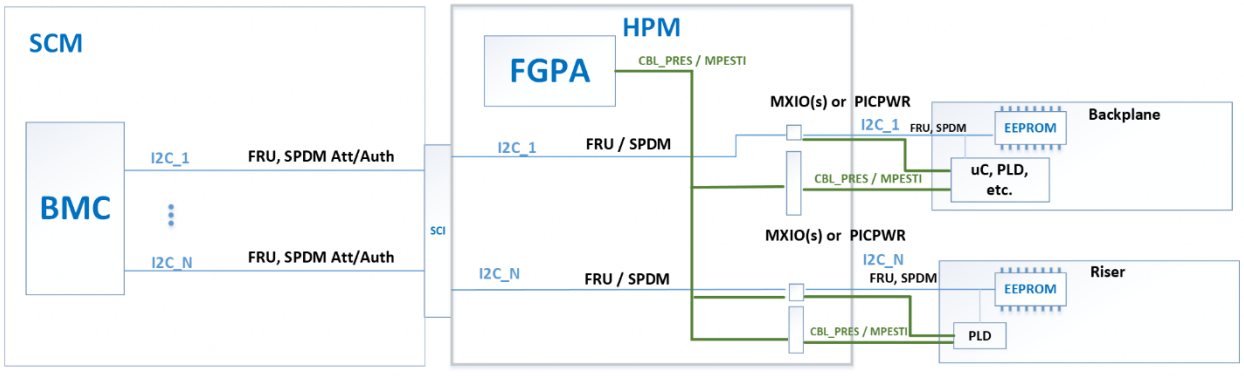


Figure 27: Non-endpoint (Backplane and Riser Example) Discovery Block Diagram

The following diagram outlines the key steps in the process.



Figure 28: Non-endpoint (Backplane and Riser) Discovery Process Diagram

10.10.1. Primary HPM Programmable Device Autonomous System Discovery: Non-Endpoint Peripherals

This section contains the discovery process steps that the Primary HPM Programmable Device will perform autonomously on behalf of the SCM BMC during the discovery process for discovery of non-endpoint peripherals, such as backplanes and risers. This is an action that is independent of any discovery actions performed directly by the BMC.

- [**STBY (Run)**] HPM's Primary Programmable Device uses CBL_PRES_N / M-PESTI to do background discovery.
- Determine presence of non-endpoint peripherals that support CBL_PRES_N / MPESTI[**STBY (Run), S5 (Hold-Off), S0 (Hold-Off)**] the SCM BMC will direct the Primary HPM Programmable Device to perform these autonomous functions based on non-endpoint supporting peripheral power requirements.
    - Discovery payload collection for non-endpoint peripherals if target supports the M-PESTI discovery phase.
    - Determination of target virtual wires (VGPIOs) support and usage (if described within the discovery payload) if the M-PESTI target supports the M-PESTI active phase (see M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification for details)
- [**STBY (Run), S5 (Hold-Off), S0 (Hold-Off)**] Primary HPM Programmable Device will indicate peripheral discovery completion via one or more interrupt signals.
    - M-PESTI IP block interrupt will signal a change in state for one or more PESTI targets.
    - M-PESTI interrupt will also raise the global interrupt that is mapped to LTPI low latency and/or SGPIO bus GPIOs.
    - [**Optional**] individual IP block interrupts may be mapped to LTPI and/or SGPIO bus GPIOs as well if desired.
- Note: If a peripheral loses presence, communications, or fails to complete the discovery process during or after completing the discovery process this shall be considered a hardware failure. The BMC should report and log that this failure occurred. The BMC may continue the platform discovery process according to vendor policy. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.

See the M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification for further details.

10.10.2. Establishing Connection to Non-Endpoint Peripherals

This section contains the process steps for establishing the connection to non-endpoint peripherals, such as backplanes and risers, in the system.

- [**STBY (Run)**] SCM BMC checks for peripheral cable presence, CBL_PRES_N, for each M-PESTI target device
    - Presence is determined as found in the DSTAT [1:0] register (see M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification for details)
- [**STBY (Run)**] SCM BMC collects FRU device information for each non-endpoint peripheral with cable presence detected via I2C as indicated by I2C bus connections to each non-endpoint peripheral connection.
    - It is required that each non-endpoint peripheral supports a FRU information device at I2C address 0xA6, and it shall be powered and reachable (not behind a MUX) during standby / auxiliary power.
- [**STBY (Run), S5 (Hold-Off), S0 (Hold-Off)**] SCM BMC determines the power requirements of the non-endpoint peripheral and may enable power based on that peripheral's requirements when it has entered that power state.
    - Full discovery of all non-endpoint peripherals may occur once BMC determines which minimum power state that the endpoint peripheral needs to operate in and can provide discovery payloads.
- [**STBY (Run), S5 (Hold-Off), S0 (Hold-Off)**] SCM BMC sets the 'PWRON_ALLOW' bit within the M-PESTI IP block PESTI target registers for any peripheral.
    - This will cause the PWR_EN PICPWR sideband signal to assert for the peripheral.
    - The DPEN bit will be enabled automatically which will enable entry to the discovery phase if that peripheral supports it (see M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification for details).
- [**STBY (Run), S5 (Hold-Off), S0 (Hold-Off)**] SCM BMC waits for PESTI target completion indication set in Primary HPM Programmable Device for all non-endpoint peripherals as set in the discovery status DSTAT [1:0] register within the M-PESTI IP block.
- [**STBY (Run), S5 (Hold-Off), S0 (Hold-Off)**] SCM BMC retrieves provisional information about non-endpoint peripherals from Primary HPM Programmable Device M-PESTI discovery payloads if that peripheral supports the M-PESTI discovery phase.
    - This information can be used to provide virtual wire definition guidance for the Primary HPM Programmable Device if used by the Primary HPM Programmable Device in an autonomous fashion if the active phase supports virtual wires.
    - BMC can now set the APEN bit to enable active phase for that M-PESTI target if supported by that target (see M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification for details).

- [**STBY (Run), S5 (Hold-Off), S0 (Hold-Off)**, **Optional**] SCM BMC compares Primary HPM Programmable Device gathered peripheral information to JSON data for validation of peripheral.
    - Uses discovery information received from Primary HPM Programmable Device (cable presence, source-to-destination routing, etc.)
    - Uses information contained in HPM FRU for non-endpoint peripherals as found in the Connectors portion of the JSON data.
    - SCM BMC will also use peripheral JSON data to enable this comparison.
- A validation error detected by the BMC at this stage shall be considered to be a discovery process failure. The BMC should report and log that this failure occurred. The BMC may continue the platform discovery process according to vendor policy. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.
- [**STBY (Run), S5 (Hold-Off), S0 (Hold-Off)**] SCM BMC establishes source/destination routing for certain non-endpoint peripherals, such as backplanes and risers, using a combination of HPM and peripheral FRU information as well as optional M-PESTI discovery information if supported.
    - Source-to-destination routing detection for backplanes and risers (mapping M-PIC PICPWRs and MXIO connector to the devices present on the backplanes and risers – sorting out the overall system topology) will be established here with BMC assistance.
    - A full explanation of source-to-destination routing can be found within the M-PESTI specification.
    - The inability of the BMC to determine source-to-destination routing at this stage shall be considered to be a discovery process failure. The BMC should report and log that this failure occurred. The BMC may continue the platform discovery process according to vendor policy. Any further actions including remediation are policy decisions left to the vendor and are out of scope for this specification.
- [**STBY (Run), S5 (Hold-Off), S0 (Hold-Off), Optional**][23] SCM BMC establishes MCTP connection to non-endpoint peripherals if supported.
- [**STBY (Run), S5 (Hold-Off), S0 (Hold-Off), Optional**] SCM BMC retrieves non-endpoint peripheral FRUs via non-I2C system buses over connections as indicated by HPM and peripheral FRU information.
- [**STBY (Run), S5 (Hold-Off), S0 (Hold-Off), Optional, Security**] SCM BMC establishes SPDM connection to each non-endpoint peripheral (e.g. backplanes, risers, etc.) over system buses (I2C, I3C, PCIe, etc.) (where possible)
- [**STBY (Run), S5 (Hold-Off), S0 (Hold-Off), Optional, Security**] SCM BMC validates information gathered from Primary HPM Programmable Device over MPESTI and FRU via SPDM connection.
    - The inability to establish an SPDM session shall be considered a security failure if required by vendor policy. The BMC should report and log that this failure occurred. The BMC may continue the platform discovery process according to vendor policy.

[23] Note: This typically requires full power for backplanes if connection if PCIe VDM or CXL and may need to be out of reset as well.

- [**STBY (Run), S5 (Hold-Off), S0 (Hold-Off), Optional, Security**] SCM BMC now trusts backplanes and risers.

Note: If a peripheral loses presence, communications, or fails to complete the discovery process during or after completing the discovery process this shall be considered a hardware failure. The BMC should report and log that this failure occurred. The BMC may continue the platform discovery process according to vendor policy. Any further actions including rediscovery and remediation are policy decisions left to the vendor and are out of scope for this specification.

See M-PNP.HPM and Peripheral Programmable Device Design Specification for further details regarding the usage of the M-PESTI IP block and per-target CSRs.

10.10.3. High-level Discovery of Non-Endpoint Peripherals

This section contains the process steps for the high-level discovery of the set of non-endpoint peripherals in the system, such as backplanes and risers.

- [**STBY (Run)**] The BMC parses non-endpoint peripherals JSON FRU information to identify bus presence, topologies, and configuration.
    - Bus topologies for peripheral buses such as I2C, I3C, PCIe, USB, etc. will be provided via the JSON file locally as they are necessary for early peripheral discovery.
- [**STBY (Run), S5 (Hold-Off), S0 (Hold-Off)**] The BMC uses bus-specific techniques to characterize buses based on peripheral power requirements.
    - Some peripherals may not be characterizable until provided main operational power is applied.
- [**STBY (Run), S5 (Hold-Off), S0 (Hold-Off), Optional**] For UBM backplanes, BMC uses the FRU to identify expected backplane and drive and slots.
    - Check backplane FRU to test for UBM support and version.
    - Use UBM commands over I2C[24] to get information:
        - backplane number
        - drive bays
        - drive presence and type.
        - drive status
        - LED type and status
        - power disable presence and status.
- Note: Failure to complete a non-endpoint peripheral inventory process the using the HPM, peripherals, and Primary HPM Programmable Device JSON files by the BMC at this stage shall be considered to be a discovery process failure. The BMC should report and log that this failure occurred. The BMC may continue the platform discovery process according to vendor policy. Any further actions including rediscovery and remediation are policy decisions left to the vendor and are out of scope for this specification.

[24] Note: UBM backplanes are currently specified to require an I2C bus connection to UBM controllers. This connection must be indicated within the backplane JSON file.

- Note: Hot-plug support for non-endpoint peripherals (e.g., backplanes, risers, etc.) is out of scope for this specification.

10.11. Peripheral Device Class Definitions

This section contains details regarding device class assignments for each peripheral device.

10.11.1. Peripheral Device Class Table

The table below contains the device class enumeration assignments for all peripheral device types.

Table 32: Peripheral Device Class Table

Peripheral	Device Class	Class Code	Subclass	Subclass Code
Riser	CEM Riser	00h	Fixed	00h
	CEM Riser	00h	Floating	01h
M-CRPS Power Supply	Power Supplies	01h	Base	00h
Backplane	Backplane	02h	Legacy	00h
		02h	UBM	01h
		02h	Passthrough	02h
Power Distribution Board (PDB)	Power Distribution Board (PDB)	03h	Base	00h
Remote Cooling Subsystem	Remote Fan Controller Board	04h	Air Movers	00h
	DLC controller board	04h	DLC	01h
Front Panel	Front Panel Board	05h	Base	00h
PCIe Switch	PCIe Switch Board	06h	Base	00h
Chassis Manager	Chassis Manager Board	08h	Base	00h
PCIe Re-timer	PCIe Re-timer Board	0Ah	Base	00h
	PCIe Retimer CEM Passthrough Board	0Ah	CEM Passthrough	01h

OCP NIC Interposer	OCP NIC Interposer Board	0Bh	Base	00h
Vendor Defined	Vendor defined range	0xF0 – 0xFD	Vendor Defined	
Reserved	Reserved for M-PESTI snoop target device. This device could be on any peripheral that supports a MUX.	FEh		

10.11.2. M-PESTI Payload and Snoop Device Considerations

The discovery payload contains the same device class code as the device class assignment table in the previous section.

An exception of this is the reserved value of FEh which assigned to the M-PESTI Snoop Device. It is required that the M-PESTI Snoop Device always returns a device class of FEh.

10.12. Peripheral M-XIO FlexIO And M-PIC PICPWR Alternate Functions Definitions and Default Assignments

This section contains details regarding the M-XIO Flex-IO signal and M-PIC PICPWR sideband signal alternate functions definitions and default assignments for each peripheral device.

10.12.1. M-XIO FlexIO Alternate Functions Definitions Table

This table below contains the set of alternate functions defined for each M-XIO FlexIO signal and the support requirements for the HPM.

Table 33: MXIO FlexIO Alternate Functions Table

Alternate Function	Default ALT Assignment	Preferred Assignment	HPM SHALL Support	IF Supported SHALL be on	Power State Valid	Common Peripheral Usage
WAKE_N*	FLEXIO_2A	FLEXIO_2A	Y	FLEXIO_2A	S5	Riser
PWR_BRAKE_N*	FLEXIO_2A	FLEXIO_2A	Y	FLEXIO_2A	S0	Riser
CPRSNT_B_N	NC	FLEXIO_1A	N	FLEXIO_1A	S5 / S0	Backplane
I2C_RESETA_N	NC	FLEXIO_2A	N	FLEXIO_2A	S5 / S0	Backplane

REFCLK_B	NC	FLEXIO_3A	N	FLEXIO_3A	S5 / S0	Backplane
REFCLK_B	NC	FLEXIO_4A	N	FLEXIO_4A	S5 / S0	Backplane
SMSCL_B	NC	FLEXIO_5A	N	FLEXIO_5A	S5 / S0	Backplane
SMSDA_B	NC	FLEXIO_6A	N	FLEXIO_6A	S5 / S0	Backplane

Note: WAKE_N and PWR_BRAKE_ are dual use signals that share the same FlexIO physical signal but change in definition when the system is in a different power state when in either S5 or S0, respectively.

10.12.2. M-PIC PICPWR Sideband Alternate Functions Definitions

This table below contains the set of alternate functions defined for each M-PIC PICPWR sideband signal and the support requirements for the HPM.

Table 34: M-PIC PICPWR Sideband Alternate Functions Definitions Table

Alternate Function	Default ALT Assignment	Preferred Assignment	HPM SHALL Support	IF Supported SHALL be on	Power State Valid	Common Peripheral Usage
PRES_A_N / PESTI_A	SB1_A	-	Y	SB1_A	S5 / S0	Backplane / Riser
PWR_EN_A	SB2_A	-	N	SB2_A	S5 / S0	Backplane / Riser
PWRFAULT_N_A	SB3_A	-	N	SB3_A	S5 / S0	Backplane / Riser
WAKE_N_A	SB4_A	-	N	SB4_A	S5 / S0	Riser
PRES_B_N / PESTI_B	SB1_B	-	Y	SB1_B	S5 / S0	Backplane / Riser
PWR_EN_B	SB2_B	-	N	SB2_B	S5 / S0	Backplane / Riser
PWRFAULT_N_B	SB3_B	-	N	SB3_B	S5 / S0	Backplane / Riser
WAKE_N_B	SB4_B	-	N	SB4_B	S5 / S0	Riser
LOCATION_A	NC	SB4_A	N	SB4_A	S5 / S0	Backplane
LOCATION_B	NC	SB4_B	N	SB4_B	S5 / S0	Backplane

10.12.3. Control Panel Sideband Alternate Functions Definitions Table

This table below contains the set of alternate functions defined for each M-PIC Control Panel sideband signal and the support requirements for the HPM.

Table 35: M-PIC Panel Control Sideband Alternate Functions Definitions Table

Alternate Function	Default ALT Assignment	Preferred Assignment	HPM SHALL Support	IF Supported SHALL be on	Power State Valid
PRES_N	**PCP_SB1**	**-**	**Y**	-	**Standby**
PWREN	**PCP_SB2**	**-**	**Y**	-	**Standby**

Usage of PCP_SB3 and PCP_SB4 are vendor defined. To promote interoperability, it is required that the HPM connect these sideband signals from the PCP connector to the HPM Primary HPM Programmable Device for general usage purposes such that it can be configured as a GPIO. See the M-PNP.HPM and Peripheral Programmable Device specification for details regarding PICPWR sideband configuration.

Please see the MHS M-PIC specification for more details.

10.13. Peripheral Device FRU Information

This section provides a high-level overview of the peripheral FRU information device data. The following figure provides such an overview, and the figure below contains the FRU information expected in an peripheral FRU information device. Solid lines indicate required sections, and dotted lines mean those sections are optional. The blue coloring indicates sections that are specified by the DMTF standards body for the purposes of creating a DMTF specific FRU information device using the DMTF FRU format.

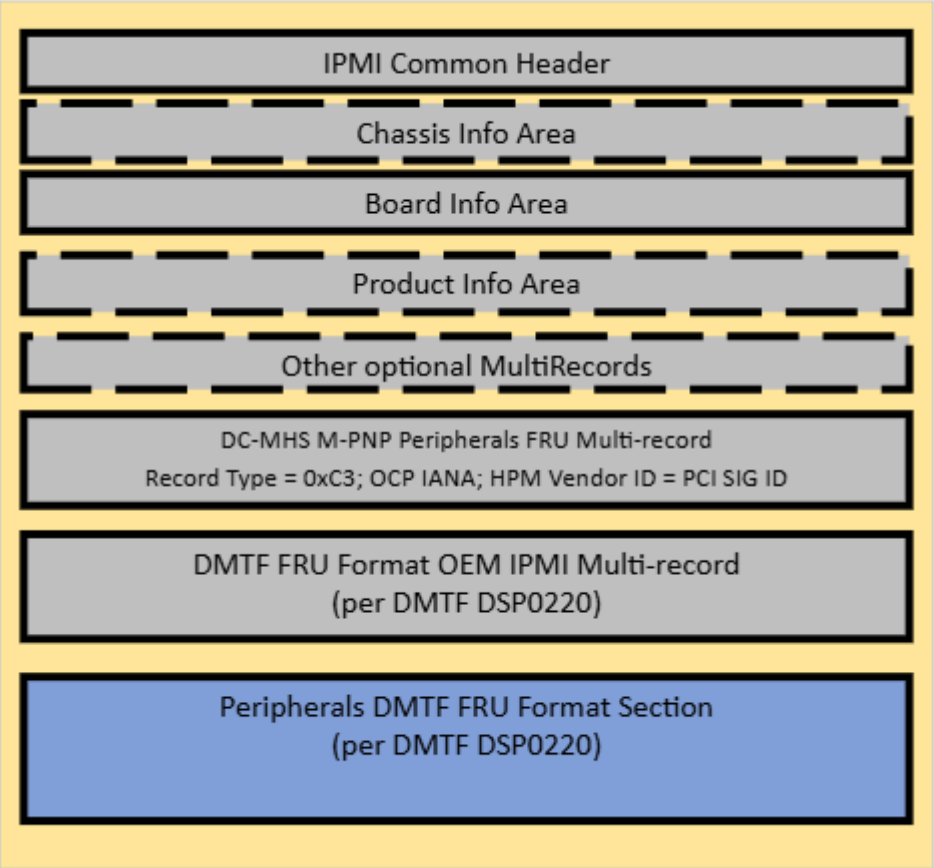


Figure 29: Peripheral FRU Layout

The following sections provide further details regarding the records contained within the block diagram.

10.13.1. M-PNP Peripheral FRU IPMI Common Header

This is the industry standard IPMI header as defined within the *[IPMI Platform Management FRU Information Storage Definition]* specification.

10.13.2. M-PNP Peripheral FRU IPMI Chassis Info Area Record

This is the industry standard IPMI chassis area record as defined within the *[IPMI Platform Management FRU Information Storage Definition]* specification. This is an optional section.

10.13.3. M-PNP Peripheral FRU IPMI Board Info Area Record

This is the industry standard IPMI board area record as defined within the *[IPMI Platform Management FRU Information Storage Definition]* specification. This is a mandatory section.

10.13.4. M-PNP Peripheral FRU IPMI Product Info Area Record

This is the industry standard IPMI product area record as defined within the *[IPMI Platform Management FRU Information Storage Definition]* specification. This is an optional section.

10.13.5. M-PNP Peripheral FRU IPMI Multi-record Area Record

This record defines a basic data set to assist in identifying and classifying the peripheral for discovery. This record shall comply with the IPMI multi-record format as per IPMI specifications. This is a mandatory section and is required to be local to the device on the FRU.

Table 36: Peripheral FRU IPMI Multi-Record Area Record Table

Offset (Dec)	Field Length	Field	Value	Notes
0	1	Record Type ID	0xC3	Assigned in TypeIDs-for-IPMI-FRU-MultiRecord.md owned by HW management project
1	1	7:7 – End of list 6:4 – Reserved, write as 000b 3:0 – Record Format version		
2	1	Record Length	6	
3	1	Record Checksum (zero checksum)	Varies	
4	1	Header Checksum (zero checksum)	Varies	

5	3	Manufacture ID	0x7f	OCP IANA assigned ID 0x00A67F, (LSB first)
			0xA6	
			0x00	
8	1	OCP M-PNP Peripheral FRU Multi-Record Version	0x00 – Reserved 0x01 – OCP M-PNP FRU record released with version 1.0 0x02 – 0xFF - Reserved	
9	2	Vendor ID	Varies per vendor	Specific to vendor. Same as PCIe vendor ID. LSB first format.
11	1	Device Class ID	Varies as per device	See Device Class Table
12	1	Device Subclass ID	Varies as per device	See Device Class Table
13	2	OEM Unique ID	Varies per vendor	Specific to vendor. Optional. LSB first format.

The intention of this multi-record is to provide a simple legacy mechanism to ascertain the peripheral class and sub-class information to assist in early discovery. Detailed information regarding the peripheral will be provided in the JSON file located in the DMTF FRU section of the FRU device image.

10.13.6. M-PNP Peripheral DMTF FRU Format OEM IPMI Multi-Record FRU M-PNP Multi-record Section

This record format is defined within the DMTF as it uses the DMTF FRU Format (see DMTF DSP0220) specification and is used to reference the offset for the DMTF FRU section of the FRU device contents. This is a mandatory section and uses the format outlined in DSP0220. The high-level diagram below outlines the DMTF FRU Format IPMI OEM multi-record.

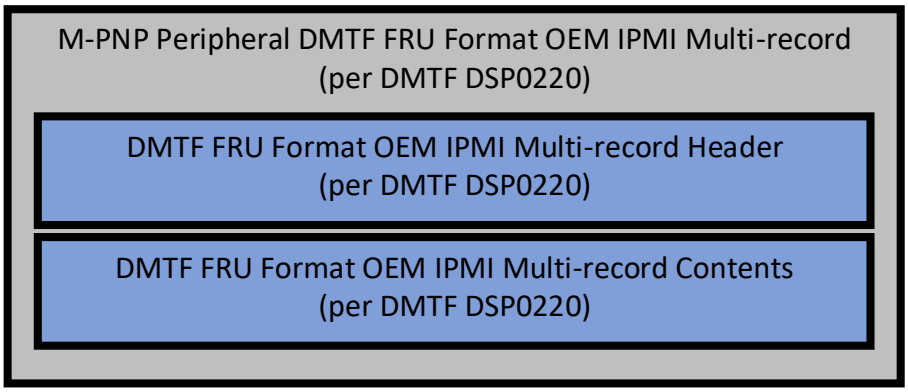


Figure 30: DMTF FRU OEM Multi-record Block Diagram

There are several sub-sections within this multi-record which are detailed below.

- **Peripheral FRU M-PNP DMTF FRU Format OEM Multi-Record Header:** Follows the DMTF FRU format OEM multi-record header format (see DMTF DSP0220 for details).
- **Peripheral FRU M-PNP DMTF FRU Format OEM Multi-Record Contents:** Follows the DMTF FRU format for the OEM multi-record (see DMTF DSP0220 for details).

10.13.7. Peripheral FRU DMTF FRU Format OEM Multi-record Header

This section contains the multi-record header table and description. This format is defined by DSP0220 and follows the same format as the standard IPMI multi-record header.

10.13.8. Peripheral FRU M-PNP FRU Format OEM Multi-record Contents

This format is defined by DSP0220 and contains the pointer to the starting offset for the DMTF FRU Format section of the FRU image.

10.13.9. M-PNP Peripheral FRU OCP M-PNP DMTF FRU Format Section

This section outlines the peripheral FRU Format section. This section follows the DMTF FRU Record format as per DSP0220 and contains M-PNP specific files. An overview of the DMTF FRU record format is shown in the diagram below.

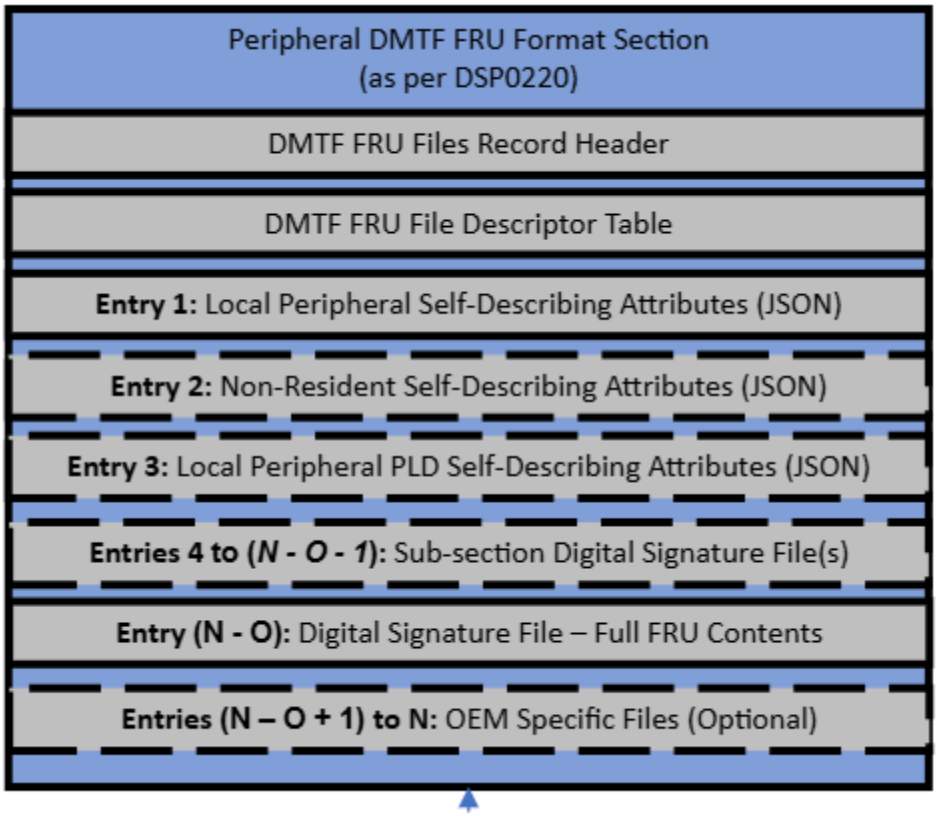


Figure 31: Block diagram of the M-PNP Peripheral DMTF FRU Format Section

Below contains an overview description of each section of the M-PNP Standards Specific Data Block. For these entries, *N* is the total number of files in the File Descriptor Table, *S* is the number of sub-section signature files, and *O* is the number of OEM files. For this table, $N = S + O + 4$.

- **FRU Files Record Header**: Contains the header and the offset to the start of the file descriptor table (see DSP0220 for details).
- **File Descriptor Table:** Contains entries for each file within the multi-record. Each entry has a pointer offset and size of the file. Each entry can have an optional hash in SHA 256 format (see DSP0220 for details).
- **File Entry 1:** Required self-describing FRU local attributes in JSON format for the peripheral. The local self-describing JSON attributes file will contain the specific attributes required to support peripheral discovery.
- **File Entry 2:** Non-resident FRU attributes in externally stored JSON file. The FRU Information Device provides GUID references to the non-resident file and location information associated with how to navigate to the non-resident file..
- **File Entry 3:** Optional resident peripheral programmable device JSON attributes in JSON file stored as initial factory version of the image JSON within the FRU information device applied during the manufacturing process. See M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification for details.
- **File Entries 4 to (*N – O - 1*)**: Contains S signatures for individual data files (optional).
- **File Entry (N - O)**: Contains a mandatory signature that covers the entire FRU contents (static data only) including static IPMI records.
- **File Entry (N – O + 1) to N:** Contains O number of optional data files in any format desired by system implementor. For more information regarding this format, please consult DSP0220.

10.13.10. M-PNP Peripheral DMTF FRU File Entry 1: Local Self-Describing JSON Data

This section contains the description for the M-PNP peripheral local JSON self-describing attributes file. This entry will always contain the minimum required local self-describing JSON data payload. Additional JSON data payloads can be optionally stored and provided within subsequent file entries. Examples of additional payloads could include any extended attributes that a system integrator may want to store locally on the FRU information device.

Section 12, “Self-Describing JSON Attributes,” contains the definition of the JSON self-describing data and the peripheral minimum required data set for local storage in the FRU information device.

10.13.11. M-PNP Peripheral DMTF FRU File Entry 2: Non-Resident Self-Describing JSON Data

This section contains the description for the M-PNP peripheral non-resident JSON self-describing attributes file. This file typically contains the additional extended attributes that a system integrator may want to store outside the FRU information device.

10.13.12. M-PNP Peripheral DMTF FRU File Entry 3: Resident Self-Describing JSON Data For Peripheral PLD Initial Factory Image

This section contains the description for the M-PNP Peripheral PLD JSON self-describing attributes file associated with the initial factory image of the peripheral programmable device. This is intended to provide an initial baseline file of the PLD image which can be used in conjunction with PLD provided patch files to construct the current JSON representation of the PLD image. This can be used as a means to reduce the required PLD memory footprint for JSON file storage. Support for the patch file mechanism is an optional feature.

Patch files may reside within Primary HPM Programmable Device memory and accessible via the memory map directory functionality as specified in the M-PNP.HPM and Peripheral Programmable Device specification. Patch files may be attached as an addendum to the PD image. Patch files may be referenced via a GUID and located as a non-resident file. The specific storage and delivery mechanism is a vendor implementation decision and left as vendor defined.

10.13.13. M-PNP Peripheral FRU M-PNP Multi-record File Entries 4 to (N - O –1): DMTF FRU Files Digital Signatures

This section contains one or more optional digital signatures for files contained within the FRU information device. See section 11, “FRU Digital Signature”, for more details regarding the FRU information device digital signature requirements.

10.13.14. M-PNP Peripheral FRU M-PNP Multi-record File Entry N – O: FRU Digital Signature

This section contains the digital signature for the entire contents of the FRU information device. It is required that the signature represents the static information contained in the FRU image. An implementation may cover dynamic information but any dynamic re-signing of the contents is outside the scope of this specification. See section 11, “FRU Digital Signature”, for more details regarding the FRU information device and the FRU digital signature.

10.13.15. M-PNP Peripheral DMTF FRU File Entries (N – O + 1) to N: Optional OEM Files

Subsequent file sections are optional and may be used to store extended attributes or other OEM data not specified within this document.

10.13.16. Peripheral FRU Self-Describing Attributes Considerations and Guidelines

This section provides general guidance regarding peripheral FRU self-describing attributes.

If any portion of the JSON is present, then it is required to contain identifying information to ensure that the extended attributes file for a given peripheral can be mapped uniquely to that peripheral's local attributes and local FRU information.

10.13.17. Peripheral FRU Information Authenticity Requirements

An optional digital signature is required to detect alteration, impersonation, cloning, or undesired gray market or recycling operations. A method for a BMC to cryptographically verify the authenticity of the FRU Information is required.

- When supplied, a digital signature for the entire FRU contents shall be provided in the FRU information device within a file as per the FRU files format. Details of this signature are outlined in section 11, "FRU Digital Signature".

10.13.18. Peripheral FRU Write Protection Requirements

In many applications, Peripheral FRU information is intentionally not meant to be modifiable by a system BMC or other entities and thus the EEPROM is typically hard strapped for write protection.

In some applications, there is a desire for hybrid FRU, meaning a partial area that is static and write protected and a partial area where the system can write. The primary use case is for carrying the specific FRU isolated failure data with the returned commodity or asset tag information carried in one or more of the traditional IPMI records.

There are many ways to implement FRU EEPROM write protection control and it is out of scope for this specification to provide specific implementation guidance.

10.14. Generalized Nested Peripherals Guidance

For use cases involving nesting non-endpoint peripherals, such as a using an extension board or interposer breakout board, it is required that the component vendor take the necessary steps to prevent incompatibility issues. Examples of incompatibility issues include I2C address collisions. To prevent I2C address collisions, an addressable MUX and bus steering logic shall be utilized. This is analogous to the PCIe CEM to PCIe CEM based re-timer board use case. Each M-PNP supporting peripheral shall have a FRU EEPROM. It is required that the steering logic steers each I2C bus to the FRU EEPROM on the intermediary board on power on reset. Since the HPM is expecting to interface with a standard peripheral, the address of -0xA6 (8-bit) is expected and required for the peripheral extension board.

An example of peripheral nesting is provided in the following diagram. Here the HPM primary Programmable Device is implemented using an FPGA device.

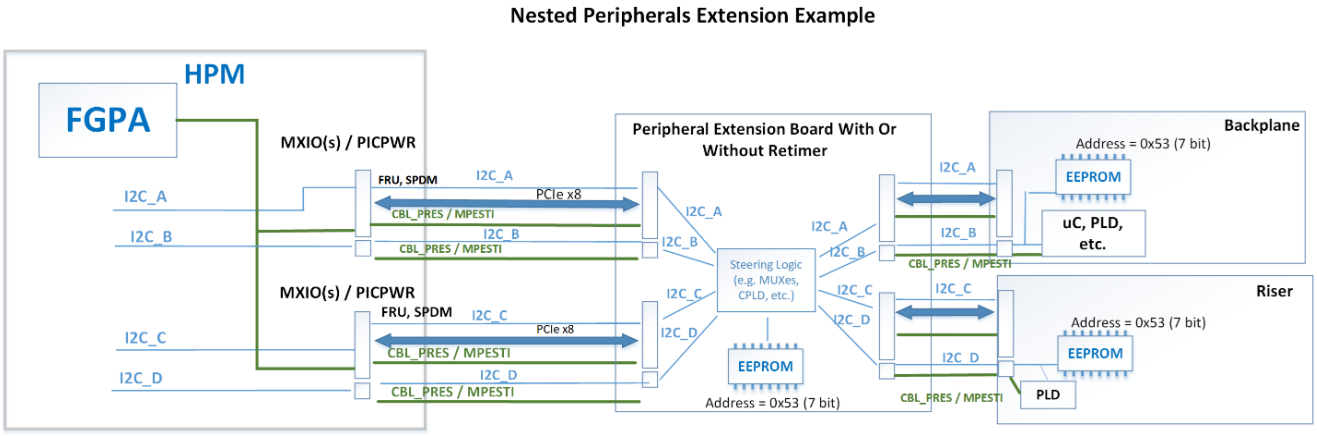


Figure 32: Peripheral Nesting Example Diagram

In this diagram, the component vendor is required to design the I2C bus steering logic such that it is guaranteed to prevent I2C address collisions. The design and implementation of this logic is outside of the scope of this specification and is completely vendor defined. This is but one example of a peripheral nesting use case and is meant for illustrative purposes. Additional use cases of peripheral nesting are outside the scope of this specification.

10.15. Discovery Phase 3: Endpoint Peripheral Discovery Method

Endpoint discovery occurs in Phase 3, and the zone of discovery expands to encompass the leaf nodes in the system topology tree. The following diagram outlines this phase in the discovery process with the specific phase highlighted in yellow.



Figure 33: Discovery Phases With Phase 3 Highlighted In Yellow

To more fully illustrate the hardware interconnections between components, the following example block diagram illustrates component interconnections used within the discovery process.

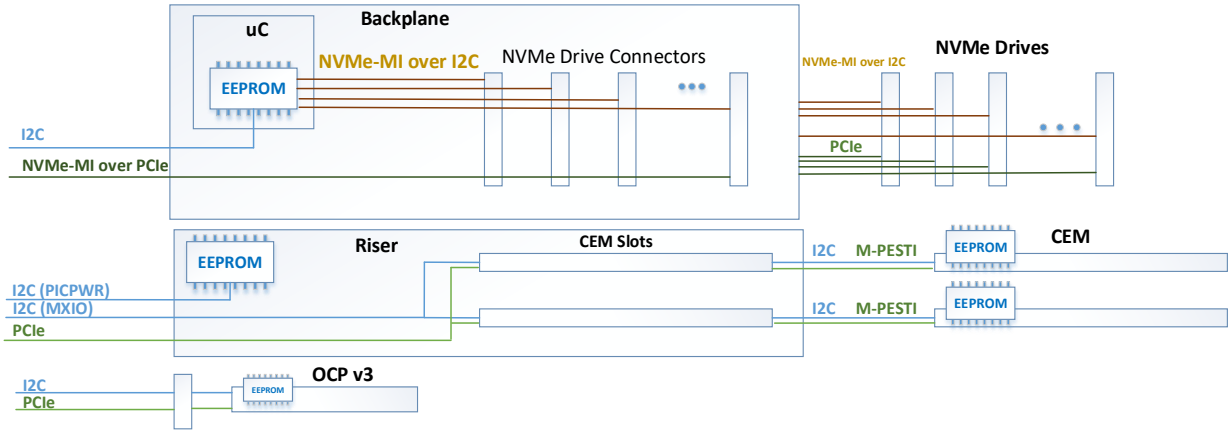


Figure 34: Endpoint Example Discovery Block Diagram

:

The following diagram outlines the key steps in the endpoint discovery process.



Figure 35: Endpoint Discovery Process Flow Diagram

This diagram will be used in the following sub-sections.

10.15.1. Primary HPM Programmable Device Autonomous System Discovery: Endpoint Peripherals

This section contains the process steps for the discovery of the set of directly connected MXIO M-PIC PICPWR, and other directly connected devices as endpoint devices connected within the system.

This essentially follows the same process that was outlined in section 10.6.1, “Primary HPM Programmable Device Autonomous System Discovery: Non-Endpoint Peripherals”. However, there are some key differences that are outlined below.

- Endpoint peripherals directly connected via M-XIO, M-PIC PICPWR, M-PIC Control Panel connectors shall:
    - use CBL_PRES_N / PESTI signal to determine presence of the endpoint device.
    - provide a FRU information device and JSON file within that FRU image to enable discovery.
        - FRU information device shall be accessible via I2C at address 0xA6.
- Endpoint peripherals connected via OCP NIC or PCIe CEM connectors shall:
    - Follow PCIe CEM or OCP NIC specifications for indication of component presence and FRU (see PCIe and OCP NIC specifications for details)
- Endpoint peripherals connected via OEM or other non-standard connectors shall:
    - provide identification of presence via information provided within the HPM or non-endpoint peripheral JSON file.
        - An example of this is usage of a GPIO for presence detection in a non-standard way.
        - Presence needs to be identified within the connector description as indicating presence of a component connected to that connector.
    - provide indication of bus reachability for FRU information device.
        - It shall provide the specific bus, such as I2C, within the JSON description of the connector in the HPM JSON file.
        - It shall provide specific FRU information device address(es) within description of connector in the HPM or non-endpoint peripheral JSON file.

10.15.2. Establishing Connection to Endpoint Peripheral Devices

This section contains the process steps for establishing a connection to the set connected devices as endpoint devices connected within the system.

This section largely follows the same process that was outlined in section 10.6.2, “Establishing Connection to Non-Endpoint Peripherals”. However, there are some key differences that are outlined below.

For endpoint peripherals directly connected via standard M-XIO, M-PIC PICPWR, M-PIC Control Panel connectors, follow the steps outlined in section 10.6.2.

For endpoint peripherals (PCIe CEM, OCP NIC, NVMe drives, etc.) connected via non-endpoint peripherals, such as backplanes and risers, these additional steps may be necessary to perform discovery:

- [**STBY, S5 (Hold-Off), S0 (Hold-Off)**]] PCIe CEM, OCP, NVMe drive, or other standard slot connector connected endpoint peripherals may require additional steps to enable discovery. These steps may include:
    - Selection of I2C MUX channel based on I2C bus topology to select one or more slots to enable FRU collection from the endpoint peripheral.
    - Control of MUXes to enable JTAG or USB bus support in the endpoint peripheral
    - Communications with non-endpoint peripheral MCU or PLD from the SCM BMC for additional discovery process steps (e.g. UBM backplanes)
    - It is required that the non-endpoint peripheral describes all controls necessary to enable discovery and control functions including GPIOs, MUX control signals, etc. within the non-endpoint FRU JSON information file.
- [**STBY, S5 (Hold-Off), S0 (Hold-Off)**, **Optional, Security**] Endpoint security related functionality including:
    - SCM BMC establishes MCTP connection (where possible) to devices if supported.
    - SCM establishes SPDM connection to devices using system buses (I2C, I3C, PCIe VDM, etc.)
    - The inability to establish an SPDM session shall be considered a security failure if required by vendor policy. The BMC should report and log that this failure occurred. The BMC may continue the platform discovery process according to vendor policy.
- Note: If a peripheral loses presence, communications, or fails to complete the discovery process during or after completing the discovery process this shall be considered a hardware failure. The BMC should report and log that this failure occurred. The BMC may continue the platform discovery process according to vendor policy. Any further actions including rediscovery and remediation are policy decisions left to the vendor and are out of scope for this specification.

10.15.3. Endpoint Peripheral High-level Discovery

This section largely follows the same process that was outlined in section 10.6.3, “High-level Discovery of Non-Endpoint Peripherals”. However, there are some key differences that are outlined below.

For endpoint peripherals directly connected via standard M-XIO, M-PIC PICPWR, M-PIC Control Panel connectors, follow the steps outlined in section 10.6.3.

For endpoint peripherals (PCIe CEM, OCP NIC, NVMe drives, etc.) connected via non-endpoint peripherals, such as backplanes and risers, these it is expected that they will use industry standard connectors / slots. Please refer to the appropriate industry standard for details regarding discovery and hot-plug support.

For endpoint peripherals connected via OEM or other non-standard connectors, it is expected that these peripherals will contain FRU data that identifies the device as this is required for M-PNP discoverability. The identification of the location and the reachability of that FRU data shall be provided within the FRU JSON file contained by the component that the endpoint is directly connected to. For a connector that is on the HPM, this means that the HPM JSON information file shall contain reachability information tied to the OEM connector. For backplanes or risers, it shall be contained within their respective FRU JSON file.

The BMC should report and log any peripheral catastrophic failures. Any further actions including rediscovery and remediation are policy decisions left to the vendor and are out of scope for this specification.

10.15.4. Granting Full Operational Power to the System (Transition To S0)

This section outlines the process steps for granting operational power to the HPM and peripherals. This is typically the 12V primary power rail from the PDB but may include additional voltage rails as needed. Operational power may include additional voltage rails to power significant subsystems within the HPM, such as the SoC and memory complex. Additionally, full operational power may be required during the peripherals discovery process in order to facilitate full discovery of certain peripherals. All voltage rail power state and failure indications shall be provided by the Primary HPM Programmable Device and outlined in the Primary HPM Programmable Device JSON file.

The following process steps shall be used to proceed from the S5 power state to the S0 power state:

- [**S5 (Run)**] The SCM BMC will set the power state transition request field to signal to the HPM PD to transition to the [**S0 SoC Ramp**] power state.
    - Setting the Target Power State field to for the power state transition request can be accomplished either by LTPI or SGPIO GPIO bit field OR via direct memory map access to the power management IP block registers (see M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification for details)
    - BMC will set the transition Trigger Flag to indicate that it requires the Primary HPM Programmable Device to being the power state transition process.
    - Primary HPM Programmable Device will set the Transition Flag within the power management status register in the power management IP block to indicate that transition is in process.

    - Primary HPM Programmable Device will indicate completion of power state transition by setting the interrupt flag for the power management IP block and will indicate that transition is complete by clearing the Transition Flag within the power management status register in the power management IP block and will set the power state in to **S0 SoC-Ramp** (see M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification for details).
- [**S0 SoC-Ramp**] The SCM BMC will indicate that it is ready for SoC power on by setting the transition request to proceed to **S0 Hold-Off**
    - BMC sets the Transition Trigger Flag
    - Primary HPM Programmable Device will indicate transition is in process via the Transition Flag
    - When completed, the Primary HPM Programmable Device will set the power management IP block interrupt to indicate completion and clear the Transition Flag) and indicate the power state is in S0 Hold Off
    - At this point, SoC power is operational, and BMC can perform OOB security related activities if desired (see M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification for details)
    - Note: SoC Ramp is a transition state prior to S0 Hold-off which is used to stabilize system power prior to applying main power to the SoC. The BMC may choose to initiate a transition directly from S5 to S0 Hold Off through the Primary HPM Programmable Device if the system does not require stopping at this stage for stability concerns. See the M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification for further details.
- [**S0 (Hold-Off), Optional, Security**] BMC can perform OOB security related activities as per required security policy.
- **[S0 (Hold-Off)] The** SCM BMC will indicate that it is ready for SoC power on by setting the transition request to proceed to S0 (Run-BIOS-Boot)
    - BMC sets the Transition Trigger Flag
    - PD will indicate transition is in process via the Transition Flag
    - When completed, the PD will set the power management IP block interrupt to indicate completion and clear the Transition Flag) and indicate the power state is in S0 (**Run-BIOS-Boot**)
    - At this point, the SoC is out of reset and beginning to execute BIOS code.
- **[S0 Run-BIOS-Boot, Optional, Security] BMC** may perform security related functions, such as attestation, to devices that required BIOS support to be discoverable.
    - Once the BIOS boot process is completed, the BMC indicates to the Primary HPM Programmable Device that it is ready for the system to proceed to the S0 Run-OS power state. This is completed using the same power state transition handshaking steps as prior state configuration processes (see M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification for details).
- **[S0 Run-OS] The** HPM and all platform components are fully powered, configured, and available for the OS boot process.
- Note: Failure of the platform to transition to the required power state shall be considered a hardware failure. The BMC should report and log that this failure occurred. The BMC may continue the platform discovery process according to vendor policy. Any further actions including rediscovery and remediation are policy decisions left to the vendor and are out of scope for this specification.

- Note: Before enabling the S0 power state, the BMC shall determine the platform power requirements through the use of component discovery information, and it shall determine the power supply capacities provided by PSUs. There may be some peripherals that cannot provide this information unless it is powered in S0. If so, the BMC may defer this power calculation until such information can be obtained. If it cannot safely determine that the power requirements of the platform are satisfied, then this shall be considered a hardware failure. The BMC should report and log that this failure occurred. Any further actions including rediscovery and remediation are policy decisions left to the vendor and are out of scope for this specification.

10.15.5. Considerations Regarding Operational Power For Peripherals Connected Via Standard Interfaces

This section provides guidance and process steps for granting operational power to endpoint and non-endpoint peripherals that are connected via standard connectors (M-XIO, M-PIC PICPWR, etc.).

Non-endpoint peripherals, such as backplanes and risers, will follow the processes outlined in section. Main operational power is granted via the PWR_EN sideband signal, and this is controlled via the POWERON_ALLOW bit within the PESTI target register for that peripheral connection. Endpoint peripherals connected via M-XIO, M-PIC PICPWR, and M-PIC Front Panel Control connectors will use the same mechanism.

Endpoint peripherals connected via standard PCIe CEM, OCP NIC, NVMe drive, or other industry standard connectors will follow guidelines within their respective specifications regarding application of operational power.

10.15.6. Considerations Regarding Main Power to Endpoint Peripherals Connected Via Non-Standard / OEM Connectors

This section provides guidance and process steps for granting operational power to endpoint and non-endpoint peripherals that are connected via non-standard, vendor defined connectors.

It is required that the FRU JSON information file contains attributes which outline the specific power requirements for devices connected to this type of connector.

10.15.7. SCM Final Discovery State Checkpoint

This section describes the state of the SCM HPM once the discovery process is completed.

At this stage, the SCM shall meet these requirements to be able to allow the system to boot.

- SCM shall have a full map of system topology.

- SCM shall build a Redfish profile for system.

At this point, the SCM can signal to the HPM that it can being the boot process. This usually means that host processor reset can be de-asserted and can then subsequently fetch code from a CPU vendor defined reset vector.

10.15.8. Runtime Rediscovery Considerations

The process of rediscovery of components during runtime for the HPM or non-endpoint peripherals is outside the scope of this specification. Peripheral rediscovery for endpoint peripherals may be supported by implementor if that peripheral supports hot plug capabilities as part of an industry standard (e.g., NVME SNIA specification) that specifically designs in such support. Consult those industry standards for further guidance.

11. FRU Digital Signature

This section outlines the specific requirements and considerations regarding the FRU information device digital signature for an M-PNP supporting system.

11.1. FRU Information Authenticity Requirements

A method for a BMC to cryptographically verify the authenticity of the FRU information is required.

- When supplied, a digital signature for the FRU contents shall cover all static data in the FRU information device.

- When supplied, a digital signature shall be provided in the FRU information device within a file as per the DMTF FRU files format.

11.2. FRU Digital Signature Requirements

When provided, a FRU digital signature shall adhere to the following requirements.

- Required to use a SHA 384-bit hash for the digital signature, at a minimum.
- It is required that vendors use a industry standard digital signature algorithm.
    - It is recommended that implementors add support for Post Quantum Cryptographic (PQC) signing of the FRU contents.

- It is required to disclose the signature algorithm and SHA hash size.

12. Component Self-Describing JSON Description

This section details the set of self-describing JSON attributes used to create the JSON file. This information is needed for the HPM and peripheral discovery process. This file information may reside in a FRU information device or other local memory storage device, or it may reside in a file maintained by the system BMC. There will be a set of attributes that are required to be local as it will be needed early in that discovery process. Other extended attributes do not need to be locally stored and can be retrieved separately using a different mechanism than the local attributes.

An HPM or peripheral vendor can decide not to describe one or more features, functions, or buses. Anything not described within the JSON file will be assumed as vendor specific and will not be included as part of the M-PNP plug and play framework.

12.1. Self-Describing JSON Overview

This section provides an overview of the self-describing JSON file structure and usage for HPM and peripheral components.

The figure below provides a top-level overview of how the self-describing JSON schema is structured for the HPM and peripherals.

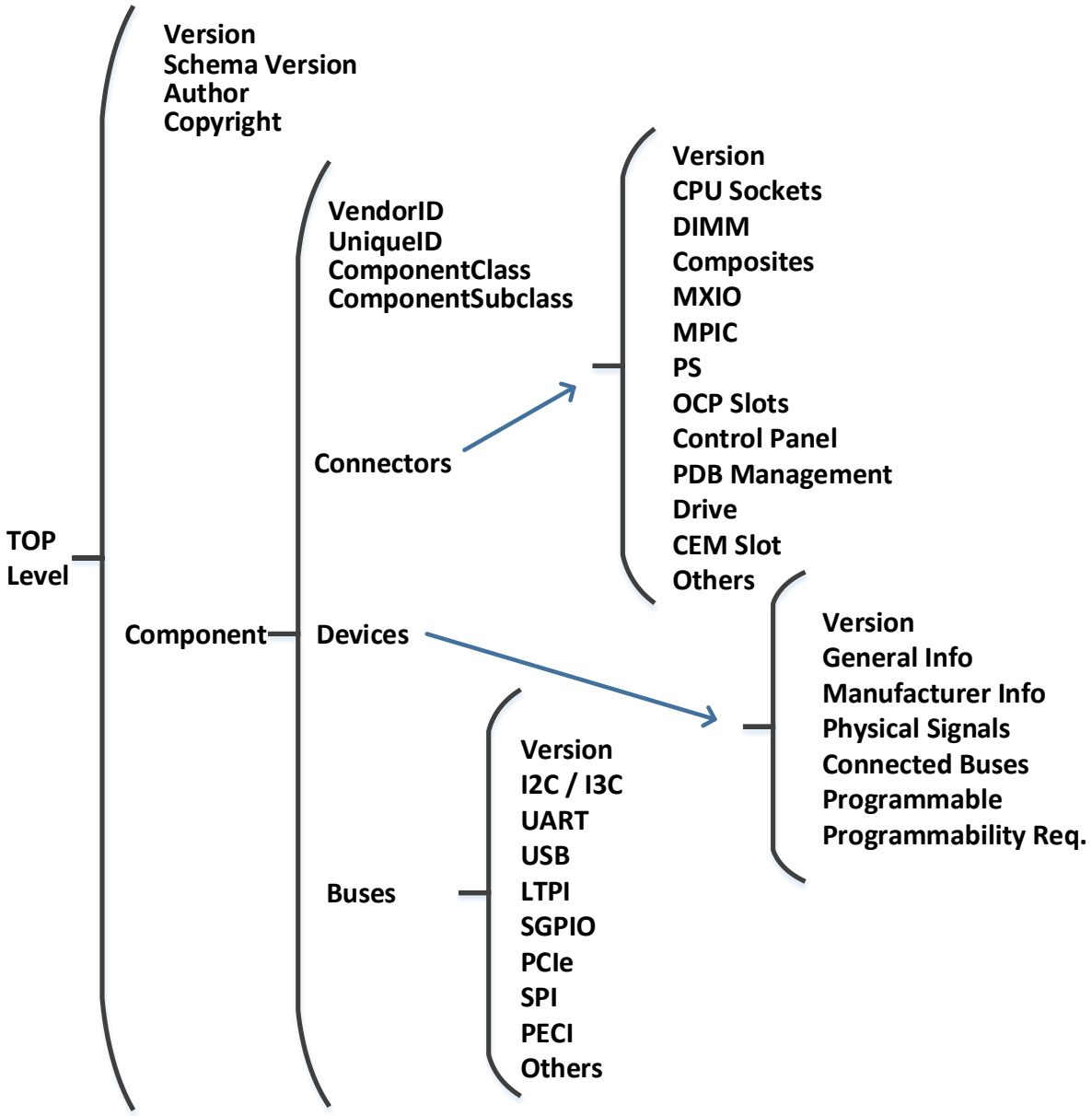


Figure 36: Self-describing JSON Schema Structure – HPM and Peripherals

The figure above contains the architectural outline of the self-describing JSON schema. The next sections provide a high-level description of each section and subsection. Examples of mock-ups are provided in the overall M-PNP specifications package. Peripherals use the same schema as the HPM. It is expected that peripheral JSON files will typically contain a reduced set of attributes needed to describe the component hardware.

Please see the document, “**OCP MHS Modular Plug-and-Play Hardware Component Schema Document”,** for further details regarding the specific JSON attributes.

12.2. JSON Schema Reference Documentation

For details regarding the JSON attributes, please consult the following documents:

- The “**OCP MHS Modular Plug-and-Play HPM and Peripheral Programmable Device Schema Documentation”,** is a document that contains a description of the Primary HPM and Peripheral Programmable Device JSON schema attributes.
- The “**OCP MHS Modular Plug-and-Play Hardware Component Schema Documentation”,** is a document that contains a description of the JSON schema attributes for hardware components.

12.3. Common Bus Reference Diagrams

This section provides some reference diagrams for use when reviewing the JSON schema reference documentation.

12.3.1. I2C Bus Reference Diagrams

The diagram below provides a set of an example I2C tree diagrams used to describe a set of I2C buses that can be described using the JSON schema.

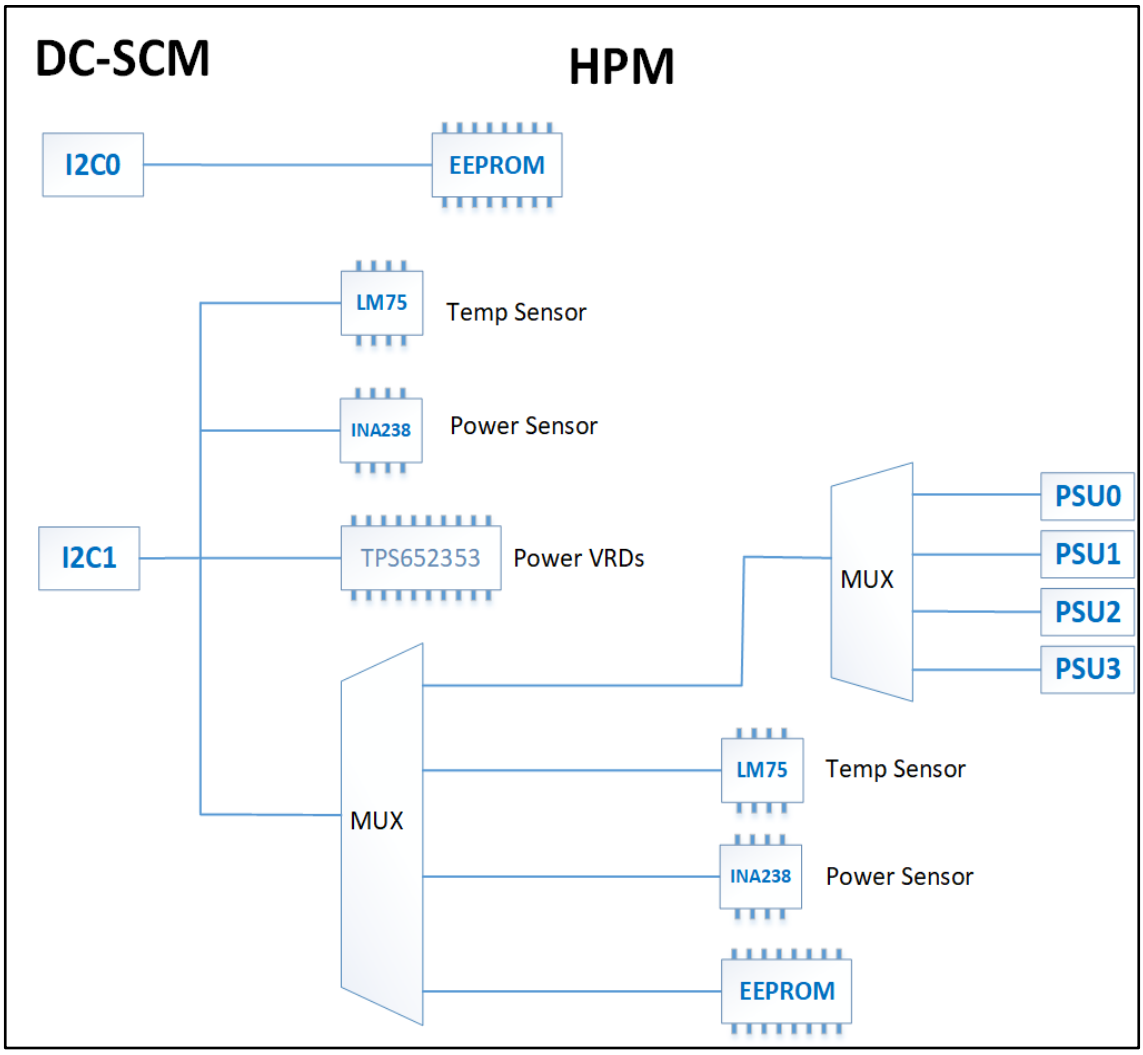


Figure 37: Example I2C device trees.

This diagram contains some example device trees for I2C as implemented on an HPM.

Please see the supplementary materials for additional JSON mock-up examples and JSON schema details.

12.3.2. I3C Bus Reference Diagrams

The diagram below provides a set of an example I3C tree diagrams used to describe a set of I3C buses that can be described using the JSON schema.

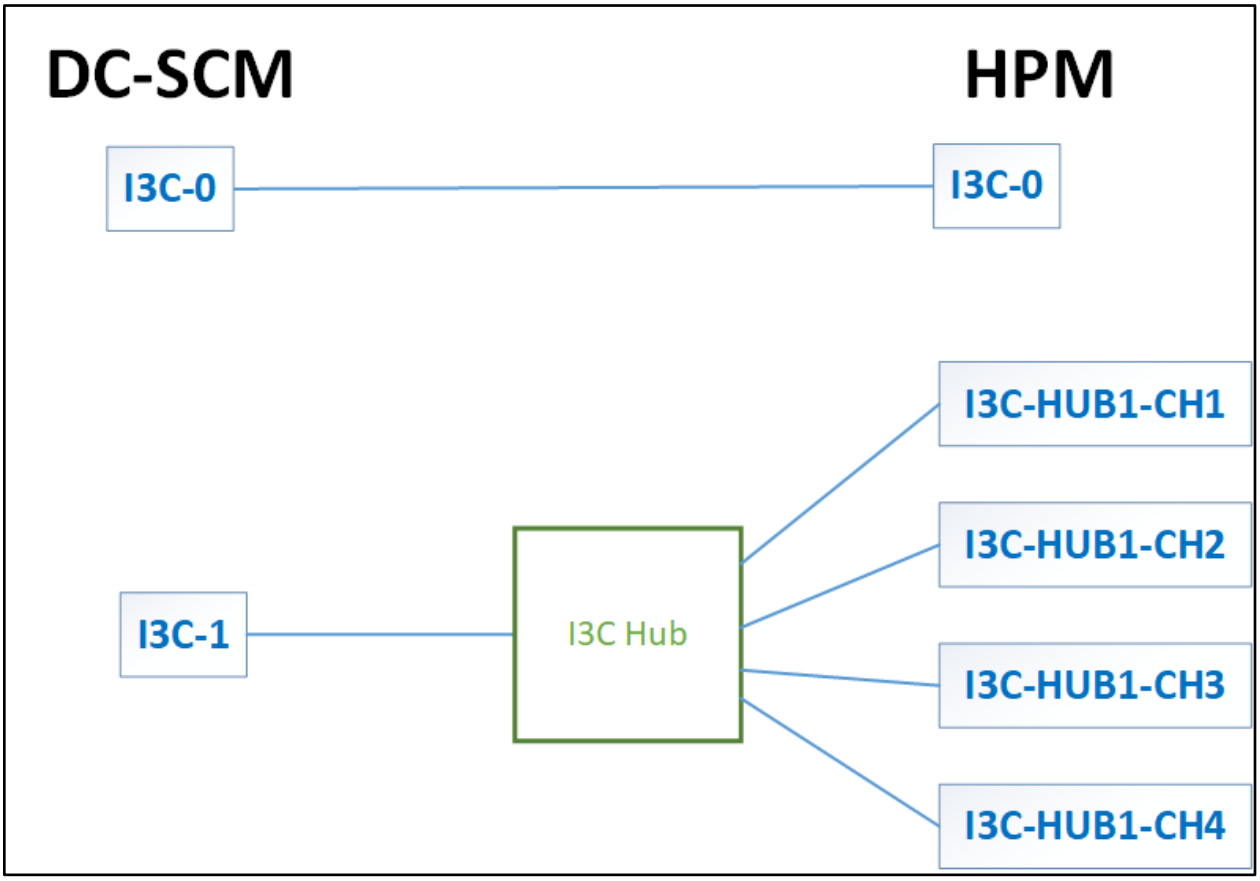
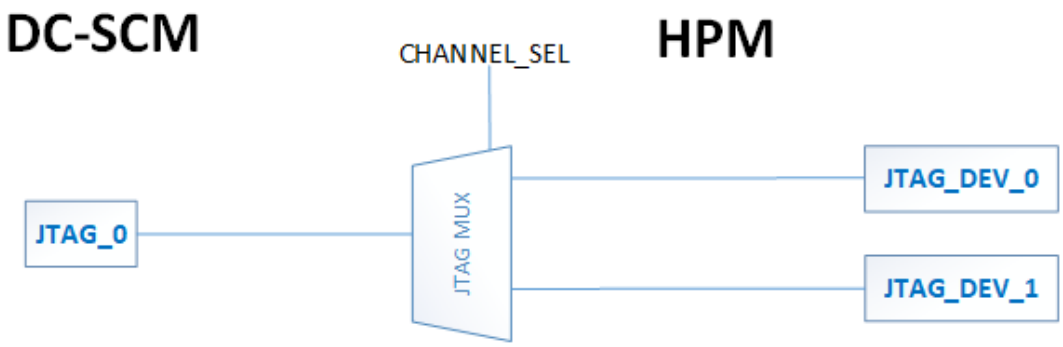


Figure 38: Example I3C device trees.

Please see the supplementary materials for additional JSON mock-up examples and JSON schema details.

12.3.3. JTAG Bus Reference Diagrams

The diagram below provides a set of an example JTAG tree diagrams used to describe a set of JTAG buses that can be described using the JSON schema.



Please see the supplementary materials for additional JSON mock-up examples and JSON schema details.

12.3.4. PCIe Bus Reference Diagrams

The diagram below provides a set of an example PCIe tree diagrams used to describe a set of PCIe buses that can be described using the JSON schema.

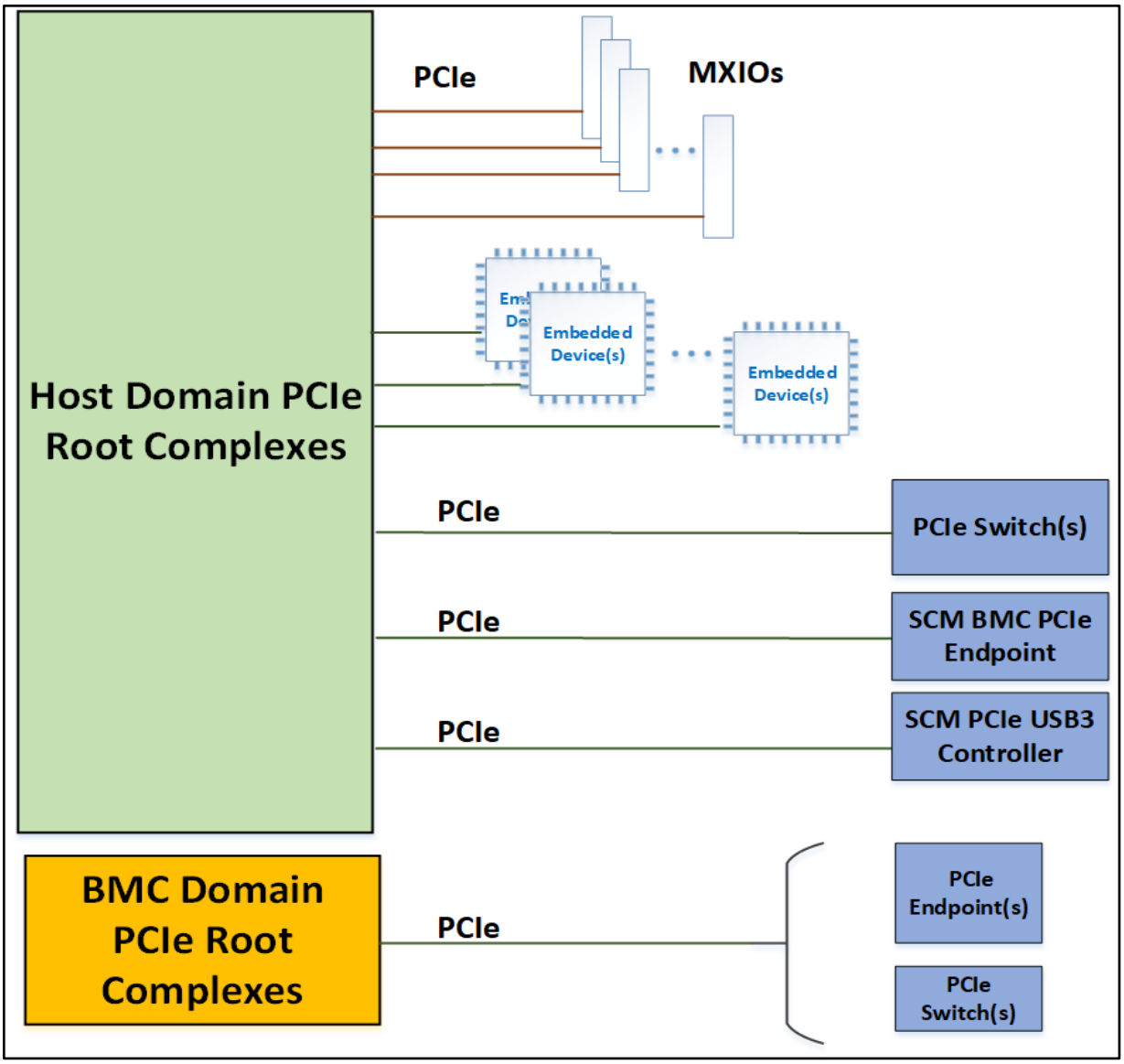


Figure 39: Example PCIe JSON Schema Hierarchy

Please see the supplementary materials for additional JSON mock-up examples and JSON schema details.

12.3.5. USB Reference Diagrams

The diagram below provides a set of example USB tree diagrams used to describe a set of USB buses that can be described using the JSON schema.\

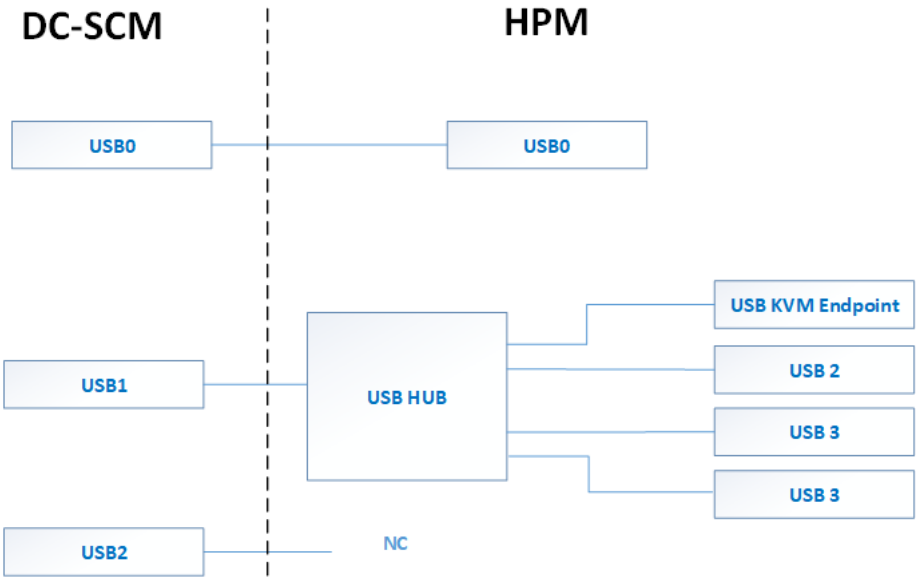


Figure 40: Example USB device topology.

Please see the supplementary materials for additional JSON mock-up examples and JSON schema details.

13. Boot Process Requirements

This section outlines the specific boot process requirements for an M-PNP supporting system.

13.1. Boot Process System-Level Discovery

This section describes the requirements and guidance needed to enable component discovery during the system boot process during the system boot phase.

13.1.1. Secure Discovery Process Enablement

This section describes features that shall be implemented to ensure the system enables a secure boot process during the discovery and configuration of the system.

The system may contain peripherals that cannot be fully discovered and configured prior to the boot phase of the platform. For these peripherals, it is preferred that the peripheral be held in reset until the BMC can authenticate it. The MPNP. HPM and Peripheral Programmable Device specification outlines a set of power management states that are specifically implemented to assist in enabling this functionality. The BMC can control the power management state flow using the Primary HPM Programmable Device via various mechanisms. During the S0 (Hold Off) state, the BMC can attest devices that are main powered but held in reset. Some devices may need to still be held in reset if they cannot be attested to during S0 (Hold-Off). It may be required to reach a certain point in the BIOS boot process so that necessary activities such as PCIe bus training occur to facilitate in-band access to one or more devices that need attesting but cannot until PCIe training has occurred. The BMC may not have an OOB management connection to that device which makes in-band access a dependency. Therefore, the additional state, S0 (Run-BIOS-Boot), was added to enable attestation during the system boot phase. Finally, once all peripherals and devices are attested, the platform will transition to the S0 Run-OS state.

Please consult the M-PNP.HPM AND PERIPHERAL PROGRAMMABLE DEVICE specification for further details regarding the Primary HPM Programmable Device power states.

13.2. BMC Usage of Primary HPM Programmable Device Power State Guidance and Considerations

This section provides example usage and guidance for the BMC consumption of HPM Primary Programmable Device (PD) power states.

The BMC is responsible for keeping track of the current system power state and is responsible for using the PD to facilitate changes in system power state. The PD will hold the center of truth for the system power state once the PD is out of reset. This occurs when the system is in Standby (Run) power state.

When the BMC requires a power state transition change, it will write to the power state transition request CSRs in the Primary HPM Programmable Device and then can initiate the transition change via the transition state request flag in those CSRs. The Primary HPM Programmable Device will indicate that a power state transition is in progress and provide a power state transition interrupt signal once it has completed. Details concerning the definition of the power management CSRs is found in the M-PNP.HPM and Peripheral Programmable Device specification.

See the power state table and transition diagram for specific details on power states and transitions in the M-PNP.HPM and Peripheral Programmable Device specification.

13.3. BMC – BIOS Communications Guidance

This section describes any requirements that the system shall adhere to for enabling component discovery and configuration during the system boot process and provides guidance as to how this can be accomplished.

13.3.1. BMC – BIOS Communications Requirements

This section describes any requirements and / or guidance that the system shall adhere regarding BMC to BIOS communications.

Table 37: BIOS - BMC Communications Requirements

Requirement	REQ	OPT	Description
Memory Mapped Buffer Interface (MMBI) Support	Y		It is required BMC and BIOS support the DMTF MMBI specification for communication during the early boot phase. See the DMTF DSP0282 specification for details.
Early Boot Data Transport	Y		The set of payloads support via a request/response protocol over MMBI.

13.3.2. Early Boot Communications Data Payloads Transport Guidance

This section describes any requirements and / or guidance that the system shall adhere regarding BMC to BIOS communications of data and payload information.

It is required that MMBI support MCTP transport binding. This is outlined in the MCTP over MMBI Transport Binding (DSP0284) specification. It is required that PLDM provide the necessary transport for Redfish payloads. Additionally, PLDM support a generalized file transfer protocol as outlined in the Platform Level Data Model (PLDM) for File Transfer Specification (DSP0242).

Table 38: Early Boot Payload Requirements

Payload	REQ	OPT	Description
Redfish Inventory Payload Set	Y		It is required BMC provide the BIOS with the Redfish payloads that represent the inventory of the platform. This payload typically includes data as per the Computer System, Chassis, and other key platform representational schemas. See Redfish specifications for details. This could be a partial inventory if the platform has not been booted at least once.
M-PNP Self Describing JSON Files	Y		It is required that the BMC provide the BIOS the set of M-PNP self-describing JSON files as collected by the BMC during the discovery process upon request.

To facilitate this, it is required that the BMC service as the File Host which contains a set of Platform Descriptor Records (PDRs) that contain specialized File Descriptor PDRs. Each File Descriptor PDR describes a specific file or directory and contains the metadata necessary along with a unique File Identifier.

Further information regarding PLDM and PDRs support can be found in the PLDM for Monitor and Control Specification (DSP0248).

It is required that a each JSON self-describing file be represented with a unique file name and file identifier. The overall set of JSON self-describing files shall be contained within a unique root directory.

The table below is not an exhaustive list and can contain many additional JSON files and extended JSON files that represent a varied set of platform components.

Table 39: Required and Optional JSON Files Exposed Via PLDM File Transfer

JSON File	Filename	REQ	OPT	Description
HPM JSON File(s)	hpm_node_0.json … hpm_node_n.json	Y		HPM JSON for HPM node 0. For other HPM nodes in the chassis that are physically separate HPMs, the node number shall be incremented. Additional HPM JSON files may be present for multi-node supporting platforms.
Primary HPM Programmable Device File(s)	Primary_hpm_pd_node_0.json … Primary_hpm_pd_node_n.json	Y		Primary HPM PD JSON for HPM node 0. For other HPM nodes in the chassis that are physically separate HPMs, the node number shall be incremented. This shall represent the primary programmable logic for the HPM. Additional HPM PD JSON files may be present for muliti-node supporting platforms.
Riser JSON File(s)	riser_0.json … riser_n.json		Y	One or more JSON files representing risers populated in the platform.
Riser PLD JSON File(s)	riser_pld_0.json … riser_pld_1.json		Y	One or more JSON files representing riser programmable logic populated in the platform.
Backplane JSON File(s)	backplane_0.json … backplane_n.json		Y	One or more JSON files representing backplanes populated in the platform.
Backplane PLD JSON File(s)	backplane _pld_0.json … backplane _pld_1.json		Y	One or more JSON files representing backplane programmable logic populated in the platform.
PDB JSON File(s)	pdb_0.json … pdb_n.json		Y	One or more JSON files representing PDBs populated in the platform.
PDB PLD JSON File(s)	pdb _pld_0.json … pdb _pld_1.json		Y	One or more JSON files representing pdb programmable logic populated in the platform.
Other Optional Files	varies		Y	One or more JSON files representing other optional components within the platform.

The file attributes field, FileAttributes, shall be set to Read-Only. All files shall have the FRU Data File classification.

Redfish Inventory and all other Redfish payload transport details are out of the scope of this specification.

14. Specification Compliance Checklist

The following table is intended to summarize the list of attributes and requirements for a design to be MHS M-PNP FRU Discovery Boot Specification compliant. Compliance requirements throughout this document can be identified by underlined text including the word “shall”.

Table 40: Compliance checklist table.

#	Technical Specification	Document Reference
1		

15. Trademarks

Names and brands may be claimed as trademarks by their respective companies. I3C is a trademark of MIPI Alliance. PMBus name and logo are trademarks of SMIF, Incorporated. PCIe® and PCI Express® are the registered trademarks of PCI-SIG.

16. Future Material

Note: This section is for reference only and intends to move into the main document body when further refined by the workgroups after Release 1.0. The implementation is highly subject to change.